数据结构面试

100 道真题 / 高频原型深度解析

从暴力解到最优数据结构 · 从不变量到面试追问

Professional Interview Handbook

2026 Edition

原创整理 · 参考《剑指 Offer》的问题驱动式教学思想，不复制其原文、插图或版式

使用说明与版权边界

定位 这不是“100 个答案”的堆叠，而是一套面向编码面试的结构化训练系统。每题统一从题意澄清、基线方案、关键洞察、不变量、参考实现、复杂度、失分点和追问展开。

本书借鉴《剑指 Offer》“围绕问题组织知识、逐步优化解法、强调代码质量与面试沟通”的教学思想，但全文、代码、图表和版式均为重新设计的原创内容，不复刻该书文字或页面。

题目来源口径：一部分是长期高频的经典数据结构原型；少量题目标记为“公开面经变体”，来自候选人公开分享，仅用于观察题型趋势，不代表任何公司的官方题库，也不保证未来重复出现。

建议使用方式：第一次只看“题目 + 题意澄清”，自己讲出基线方案；第二次读“关键洞察 + 不变量”；第三次手写代码并覆盖边界；最后只看追问，训练结构替换能力。

能力地图：100 题不是 100 种算法

高质量准备的目标不是记住 100 份代码，而是把问题快速压缩到有限的结构模式。可以把整套题库近似理解为：

核心公式 100 道题 ≈ 20 个核心 Pattern × 多种边界/约束/追问变体。真正的面试能力来自“识别 Pattern → 建立不变量 → 用约束证明复杂度”。

Pattern	代表题	识别信号
Hash Lookup	Q001/Q016-Q023	把重复搜索变成平均 O(1) 查询
Two Pointers	Q002/Q009/Q031	利用单调性或长度对齐消除嵌套循环
Sliding Window	Q003/Q004/Q008	连续区间 + 左右边界单调移动
Prefix Sum	Q007	把区间和变成两个前缀状态之差
Monotonic Structure	Q008/Q038-Q040	删除被新元素永久支配的候选
Binary Search	Q010/Q015/Q095	答案空间/有序空间每次排除一半
Fast-Slow Pointer	Q025/Q026/Q029/Q030/Q033	链表距离与环结构
Tree Postorder DP	Q048/Q049/Q056	子树返回状态 + 当前节点合并
Heap / Top-K	Q059-Q066	持续维护极值边界
Trie	Q067-Q072/Q099	字符串/路径公共前缀共享
BFS/DFS	Q073-Q081	连通、层级、无权最短路
DSU	Q075/Q076/Q084/Q085	动态等价类与连通性
Topological Sort	Q077/Q078	先后依赖与 DAG
Dijkstra/MST	Q082/Q083	带权最短路与最小连通成本
Range Structure	Q087-Q092	动态更新 + 区间聚合
Composite Design	Q093-Q100	用多种结构共同满足 API 复杂度

复习优先级与面试方法

截至 2026 年，Tech Interview Handbook 的最新学习清单仍把 Array、String、Sorting/Search、Matrix、Tree、Graph 列为 High Priority；Hash Table、Linked List、Queue、Stack、Heap、Trie、Interval 列为 Mid Priority。本书因此将前 86 题作为主战区，区间树与复合设计用于冲击更高难度和高级岗位。

每道题建议按以下 7 步口述：① 澄清输入/输出与边界；② 给出能工作的基线；③ 指出瓶颈；④ 识别可利用的结构；⑤ 写出不变量；⑥ 编码并自测；⑦ 报复杂度并主动回答 follow-up。

- 输入是否有序？是否有重复/负数/空值？
- 允许修改输入吗？需要稳定顺序吗？
- 数据规模多大？目标复杂度是否暗示特殊结构？
- 先说基线，避免“背答案感”。
- 优化时明确“减少了哪类重复工作”。
- 写代码时用 invariant 驱动而非凭感觉。
- 最后主动覆盖空输入、单元素、极端重复、极端偏斜等边界。

目录

第一章 数组、字符串、矩阵与区间 Q001-Q015

把连续空间问题压缩成 Hash、双指针、滑动窗口、前缀和、二分与 Sweep Line。

第二章 HashMap / HashSet Q016-Q023

理解 O(1) lookup 的价值，也理解它只是平均复杂度；从应用走到底层冲突与扩容。

第三章 链表 Q024-Q033

核心不是 API，而是指针不变量：保存 next、Dummy、快慢指针、局部反转。

第四章 Stack / Queue / Deque Q034-Q043

嵌套结构、最近未完成状态、单调关系、FIFO 层级搜索。

第五章 Binary Tree / BST Q044-Q058

从递归定义出发，区分“向父返回的状态”和“当前节点形成的全局答案”。

第六章 Heap / Priority Queue Q059-Q066

Top-K、动态最值、K 路归并、在线中位数与调度。

第七章 Trie / Prefix Tree Q067-Q072

把字符串集合压缩成公共前缀图；理解空间换查询与前缀剪枝。

第八章 Graph / BFS / DFS / Union-Find Q073-Q086

先建模图，再决定遍历、最短路、拓扑、连通性还是最小生成树。

第九章 Fenwick / Segment Tree / Range Structures Q087-Q092

处理“动态更新 + 区间查询”，理解数据结构如何缓存可组合的区间信息。

第十章 复合数据结构设计 Q093-Q100

把单一结构的短板通过组合补齐，并用 invariant 驱动 API 复杂度设计。

第一章

数组、字符串、矩阵与区间

Q001 - Q015

把连续空间问题压缩成 Hash、双指针、滑动窗口、前缀和、二分与 Sweep Line。

Q001 Two Sum：两数之和

难度 E	核心 数组 / HashMap	类别 数组、字符串、矩阵与区间

题目

给定整数数组 nums 和目标值 target，返回两个不同下标 i、j，使 nums[i] + nums[j] = target。通常假设恰有一组解。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

双重循环枚举所有二元组，时间 O(n^2)。若先排序再双指针可做到 O(n log n)，但原下标需要额外保存。

关键洞察：为什么这个数据结构合适

把“我现在需要哪个数”转成 O(1) 查询：扫描 x 时只需询问 complement=target-x 是否已经出现。先查后写可自然避免同一元素被使用两次。

不变量（Invariant） 扫描到位置 i 时，哈希表只包含 [0,i) 的元素；若 complement 在表中，那么得到的一定是两个不同下标。

参考实现（Python ）

```
def two_sum(nums, target):
    pos = {}
    for i, x in enumerate(nums):
        y = target - x
        if y in pos:
            return [pos[y], i]
        pos[x] = i
    return []
```

复杂度

时间 O(n)，额外空间 O(n)。

高频失分点

- 把值直接放进 set，却忘了题目需要下标
- 先写入再查询会在 target=2*x 时错误复用当前元素

- 没有向面试官确认是否保证有解/是否允许多个答案

面试官追问

- 如果数组已经有序，能否做到 O(1) 额外空间?
- 如果要返回所有不重复数值对，如何去重?
- 数据流场景下如何在线判断?

60 秒口述模板

我会先给出一个可工作的基线方案：双重循环枚举所有二元组，时间 O(n^2)。若先排序再双指针可做到 O(n log n)，但原下标需要额外保存。 真正的瓶颈在重复工作，因此优化的核心是：把“我现在需要哪个数”转成 O(1) 查询：扫描 x 时只需询问 complement=target-x 是否已经出现。先查后写可自然避免同一元素被使用两次。 正确性依赖的不变量是：扫描到位置 i 时，哈希表只包含 [0,i) 的元素；若 complement 在表中，那么得到的一定是两个不同下标。 最终复杂度为：时间 O(n)，额外空间 O(n)。

Q002 3Sum：三数之和

难度 M	核心 数组 / 排序	类别 数组、字符串、矩阵与区间

题目

给定整数数组，返回所有和为 0 的不重复三元组。答案中三元组顺序通常不重要。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

三重循环是 O(n^3)；固定两个数、用哈希找第三个可到 O(n^2)，但去重逻辑更繁琐。

关键洞察：为什么这个数据结构合适

排序后固定 nums[i]，剩余两数之和目标为 -nums[i]，用左右指针线性收缩。排序同时提供了稳定的去重方式。

不变量（Invariant） 对固定 i，若当前和过小只能增大 left；过大只能减小 right。找到答案后跳过所有相同的 left/right 值，保证输出唯一。

参考实现（Python ）

```
def three_sum(nums):
    nums.sort()
    ans = []
    n = len(nums)
    for i in range(n - 2):
        if i and nums[i] == nums[i - 1]:
            continue
        if nums[i] > 0:
            break
        l, r = i + 1, n - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s < 0:
                l += 1
            elif s > 0:
                r -= 1
            else:
                ans.append([nums[i], nums[l], nums[r]])
                l += 1; r -= 1
                while l < r and nums[l] == nums[l - 1]: l += 1
                while l < r and nums[r] == nums[r + 1]: r -= 1
```

```
    return ans
```

复杂度

排序 O(n log n)，双指针主循环 O(n^2)，额外空间取决于排序实现。

高频失分点

- 只对 i 去重，没有对 l/r 去重
- 忘记 nums[i]>0 时可以提前结束
- 把复杂度误报成 O(n log n)

面试官追问

- 改成 4Sum 如何泛化?
- 如何写一个通用 k-Sum?
- 若只要求是否存在而不是列举所有答案，能否更早退出?

60 秒口述模板

我会先给出一个可工作的基线方案：三重循环是 O(n^3)；固定两个数、用哈希找第三个可到 O(n^2)，但去重逻辑更繁琐。 真正的瓶颈在重复工作，因此优化的核心是：排序后固定 nums[i]，剩余两数之和目标为 -nums[i]，用左右指针线性收缩。排序同时提供了稳定的去重方式。 正确性依赖的不变量是：对固定 i，若当前和过小只能增大 left；过大只能减小 right。找到答案后跳过所有相同的 left/right 值，保证输出唯一。 最终复杂度为：排序 O(n log n)，双指针主循环 O(n^2)，额外空间取决于排序实现。

Q003 最长无重复字符子串

难度 M	核心 字符串 / 滑动窗口	类别 数组、字符串、矩阵与区间

题目

给定字符串 s，求不含重复字符的最长连续子串长度。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举每个左端点并向右扩展直到重复，最坏 O(n^2)。

关键洞察：为什么这个数据结构合适

维护窗口 [left,right] 中字符均唯一。记录每个字符最近出现位置；遇到重复字符 c 时，把 left 跳到 max(left,last[c]+1)。

不变量（Invariant） 每次迭代结束后，窗口内没有重复字符；left 单调不减，因此每个位置最多被跨过一次。

参考实现（Python ）

```
def length_of_longest_substring(s):
    last = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in last:
            left = max(left, last[ch] + 1)
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

复杂度

时间 O(n)，空间 O(min(n, 字符集大小))。

高频失分点

- left 直接赋 last[ch]+1，导致窗口左边界倒退
- 混淆“子串”和“子序列”
- Unicode 字符集下仍错误假设 128 大小数组

面试官追问

- 如果最多允许 K 个不同字符怎么办?
- 如果最多允许重复一次怎么办?
- 若输入是无限字符流，状态如何维护?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举每个左端点并向右扩展直到重复，最坏 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：维护窗口 [left,right] 中字符均唯一。记录每个字符最近出现位置；遇到重复字符 c 时，把 left 跳到 max(left,last[c]+1)。 正确性依赖的不变量是：每次迭代结束后，窗口内没有重复字符；left 单调不减，因此每个位置最多被跨过一次。 最终复杂度为：时间 O(n)，空间 O(min(n, 字符集大小))。

Q004 最小覆盖子串

难度 H	核心 字符串 / 滑动窗口	类别 数组、字符串、矩阵与区间

题目

给定 s 与 t，找 s 中包含 t 全部字符及其 multiplicity 的最短连续子串；不存在则返回空串。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举所有子串并统计字符需要 O(n^2) 甚至更高。

关键洞察：为什么这个数据结构合适

右指针扩张直到窗口满足需求，再不断移动左指针压缩；当某个必要字符数量跌破要求时，窗口重新变为无效。关键是只跟踪“满足要求的字符种类数”。

不变量（Invariant） 窗口有效当且仅当 formed == required。左指针只在窗口有效时推进，因此被记录的候选都是合法覆盖。

参考实现（Python ）

```
from collections import Counter, defaultdict

def min_window(s, t):
    if not t: return ''
    need = Counter(t)
    have = defaultdict(int)
    required = len(need)
    formed = 0
    left = 0
    best = (float('inf'), 0, 0)
    for right, ch in enumerate(s):
        have[ch] += 1
        if ch in need and have[ch] == need[ch]: formed += 1
        while formed == required:
            if right-left+1 < best[0]: best = (right-left+1,left,right)
            c = s[left]; have[c] -= 1; left += 1
            if c in need and have[c] < need[c]: formed -= 1
    return '' if best[0] == float('inf') else s[best[1]:best[2]+1]
```

复杂度

时间 O(|s|+|t|)，空间 O(字符集)。

高频失分点

- 只比较 distinct 字符而忽略 t 中重复次数
- 在字符数刚好等于 need 时 formed 加一，减少时也必须对称处理
- 空 t、无解、s 比 t 短的边界

面试官追问

- 如果要求覆盖 t 的任意排列，本题为何仍适用?
- 如果要求最短子序列而非子串，结构会怎样变化?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举所有子串并统计字符需要 O(n^2) 甚至更高。 真正的瓶颈在重复工作，因此优化的核心是：右指针扩张直到窗口满足需求，再不断移动左指针压缩；当某个必要字符数量跌破要求时，窗口重新变为无效。关键是只跟踪“满足要求的字符种类数”。 正确性依赖的不变量是：窗口有效当且仅当 formed == required。左指针只在窗口有效时推进，因此被记录的候选都是合法覆盖。 最终复杂度为：时间 O(|s|+|t|)，空间 O(字符集)。

Q005 除自身以外数组乘积

难度 M	核心 数组 / 前缀/后缀	类别 数组、字符串、矩阵与区间

题目

返回 answer[i] = nums 中除 nums[i] 外所有元素之积；不允许使用除法。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

对每个 i 重新乘其余元素为 O(n^2)。分别构造 prefix 与 suffix 数组可以 O(n) 时间但需要 O(n) 额外空间。

关键洞察：为什么这个数据结构合适

answer[i] 先保存 i 左侧所有数的乘积，再用一个滚动变量 suffix 从右向左乘入右侧乘积；输出数组不计额外空间时可做到 O(1)。

不变量（Invariant） 第二遍处理 i 时，answer[i] 已等于左侧积，suffix 等于严格右侧积，两者乘积恰好排除 nums[i]。

参考实现（Python ）

```
def product_except_self(nums):
    n = len(nums)
    ans = [1] * n
    prefix = 1
    for i in range(n):
        ans[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n-1, -1, -1):
        ans[i] *= suffix
        suffix *= nums[i]
    return ans
```

复杂度

时间 O(n)，除输出外额外空间 O(1)。

高频失分点

- 使用除法后被 0 卡住

- 第二遍先更新 suffix 再乘，错误地包含自己
- 没有说明输出数组是否计入空间复杂度

面试官追问

- 如果数据会溢出怎么办?
- 如果只允许单遍扫描，能否做到同样复杂度？为什么困难?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个 i 重新乘其余元素为 O(n^2)。分别构造 prefix 与 suffix 数组可以 O(n) 时间但需要 O(n) 额外空间。 真正的瓶颈在重复工作，因此优化的核心是：answer[i] 先保存 i 左侧所有数的乘积，再用一个滚动变量 suffix 从右向左乘入右侧乘积；输出数组不计额外空间时可做到 O(1)。 正确性依赖的不变量是：第二遍处理 i 时，answer[i] 已等于左侧积，suffix 等于严格右侧积，两者乘积恰好排除 nums[i]。 最终复杂度为：时间 O(n)，除输出外额外空间 O(1)。

Q006 最大连续子数组和

难度 M	核心 数组 / 动态规划	类别 数组、字符串、矩阵与区间

题目

给定整数数组，求和最大的非空连续子数组之和。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举左右边界 O(n^2)，若每次重新求和甚至 O(n^3)。

关键洞察：为什么这个数据结构合适

定义 cur 为“必须以当前位置结束”的最大和；新元素 x 到来时，要么从 x 重新开始，要么接在此前 cur 后面：cur=max(x,cur+x)。全局 best 记录所有结束位置的最大值。

不变量（Invariant） cur 始终是以 i 结尾的最优子数组，因此 best=max_{j<=i} cur[j]。

参考实现（Python ）

```
def max_subarray(nums):
    cur = best = nums[0]
    for x in nums[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 把 cur 初始为 0 会让全负数组返回 0
- 把“连续”问题错误理解为挑选任意正数
- 若要返回区间下标，需要额外维护起点

面试官追问

- 如何返回具体区间?

- 若允许删除一个元素，状态如何增加?
- 二维矩阵最大子矩形如何由此扩展?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举左右边界 O(n^2)，若每次重新求和甚至 O(n^3)。 真正的瓶颈在重复工作，因此优化的核心是：定义 cur 为“必须以当前位置结束”的最大和；新元素 x 到来时，要么从 x 重新开始，要么接在此前 cur 后面：cur=max(x,cur+x)。全局 best 记录所有结束位置的最大值。 正确性依赖的不变量是：cur 始终是以 i 结尾的最优子数组，因此 best=max_{j<=i} cur[j]。 最终复杂度为：时间 O(n)，空间 O(1)。

Q007 和为 K 的子数组数量

难度 M	核心 数组 / 前缀和	类别 数组、字符串、矩阵与区间

题目

给定整数数组 nums 和整数 k，统计和恰好为 k 的连续子数组数量。数组可包含负数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举所有起点和终点可在前缀和帮助下做到 O(n^2)。双指针/普通滑窗在含负数时失效。

关键洞察：为什么这个数据结构合适

设当前前缀和为 S_j。若存在历史前缀 S_i=S_j-k，则 (i,j] 的和为 k。哈希表存“某个前缀和出现过多少次”。

不变量（Invariant） 扫描到 j 时，freq 只统计 j 之前的前缀；先查询 freq[prefix-k] 再把当前 prefix 加入表。初始 freq[0]=1 表示从数组开头开始的子数组。

参考实现（Python ）

```
from collections import defaultdict

def subarray_sum(nums, k):
    freq = defaultdict(int)
    freq[0] = 1
    prefix = ans = 0
    for x in nums:
        prefix += x
        ans += freq[prefix - k]
        freq[prefix] += 1
    return ans
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 忘记 freq[0]=1
- 误用滑动窗口

- 只存前缀和是否出现而非出现次数，漏掉多解

面试官追问

- 若只求最长和为 k 子数组，哈希表应存什么?
- 二维矩阵中统计和为 target 的子矩形怎样降维?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举所有起点和终点可在前缀和帮助下做到 O(n^2)。双指针/普通滑窗在含负数时失效。 真正的瓶颈在重复工作，因此优化的核心是：设当前前缀和为 S_j。若存在历史前缀 S_i=S_j-k，则 (i,j] 的和为 k。哈希表存“某个前缀和出现过多少次”。 正确性依赖的不变量是：扫描到 j 时，freq 只统计 j 之前的前缀；先查询 freq[prefix-k] 再把当前 prefix 加入表。初始 freq[0]=1 表示从数组开头开始的子数组。 最终复杂度为：时间 O(n)，空间 O(n)。

Q008 滑动窗口最大值

难度 H	核心 Deque / 单调队列	类别 数组、字符串、矩阵与区间

题目

给定数组与窗口大小 k，窗口从左到右滑动，每一步输出窗口最大值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每个窗口扫描 k 个元素为 O(nk)；用 heap 可到 O(n log n) 或配合延迟删除。

关键洞察：为什么这个数据结构合适

维护一个下标 deque，使对应值严格/非严格单调递减。新元素进入时，把尾部所有不可能再成为最大值的更小元素弹出；队首若已经离开窗口则弹出。

不变量（Invariant） deque 中下标递增、值递减，且都属于当前窗口；因此队首永远是窗口最大值。每个下标至多进队出队一次。

参考实现（Python ）

```
from collections import deque

def max_sliding_window(nums, k):
    q, ans = deque(), []
    for i, x in enumerate(nums):
        while q and q[0] <= i-k:
            q.popleft()
        while q and nums[q[-1]] <= x:
            q.pop()
        q.append(i)
        if i >= k-1:
            ans.append(nums[q[0]])
    return ans
```

复杂度

时间 O(n)，空间 O(k)。

高频失分点

- deque 存值导致无法判断元素是否过期
- 维护成递增队列却取错端
- 把 while 写成 if 导致多个过期/劣质元素残留

面试官追问

- 如何同时输出 min 和 max?
- 如果窗口大小动态变化怎么办?
- 为什么这是一种摊销 O(1) 操作?

60 秒口述模板

我会先给出一个可工作的基线方案：每个窗口扫描 k 个元素为 O(nk)；用 heap 可到 O(n log n) 或配合延迟删除。 真正的瓶颈在重复工作，因此优化的核心是：维护一个下标 deque，使对应值严格/非严格单调递减。新元素进入时，把尾部所有不可能再成为最大值的更小元素弹出；队首若已经离开窗口则弹出。 正确性依赖的不变量是：deque 中下标递增、值递减，且都属于当前窗口；因此队首永远是窗口最大值。每个下标至多进队出队一次。 最终复杂度为：时间 O(n)，空间 O(k)。

Q009 接雨水

难度 H	核心 数组 / 双指针	类别 数组、字符串、矩阵与区间

题目

给定非负高度数组，柱宽均为 1，计算下雨后可接多少单位水。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

对每个位置分别向左右扫描最高柱，O(n^2)。预计算 leftMax/rightMax 可 O(n) 时间、O(n) 空间。

关键洞察：为什么这个数据结构合适

双指针把额外空间降到 O(1)：当 leftMax <= rightMax 时，左侧当前位置的水量只由 leftMax 决定，因为右侧至少已有一个不低于 rightMax 的边界；右侧对称。

不变量（Invariant） 每次移动较低边界一侧时，该位置的另一侧存在足够高的已知边界，因此其水量可以立即且永久结算。

参考实现（Python ）

```
def trap(height):
    l, r = 0, len(height)-1
    lmax = rmax = water = 0
    while l <= r:
        if lmax <= rmax:
            lmax = max(lmax, height[l])
            water += lmax - height[l]
            l += 1
        else:
            rmax = max(rmax, height[r])
            water += rmax - height[r]
            r -= 1
    return water
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 用当前 height[l] 与 height[r] 比较却没有正确维护 max
- 边界条件导致重复计算
- 只会背代码，说不清为何可以移动较小侧

面试官追问

- 用单调栈如何做?
- 二维高度图接雨水为什么需要最小堆?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个位置分别向左右扫描最高柱，O(n^2)。预计算 leftMax/rightMax 可 O(n) 时间、O(n) 空间。 真正的瓶颈在重复工作，因此优化的核心是：双指针把额外空间降到 O(1)：当 leftMax <= rightMax 时，左侧当前位置的水量只由 leftMax 决定，因为右侧至少已有一个不低于 rightMax 的边界；右侧对称。 正确性依赖的不变量是：每次移动较低边界一侧时，该位置的另一侧存在足够高的已知边界，因此其水量可以立即且永久结算。 最终复杂度为：时间 O(n)，空间 O(1)。

Q010 旋转有序数组查找

难度 M	核心 数组 / 二分查找	类别 数组、字符串、矩阵与区间

题目

一个严格递增数组在未知位置旋转，给定 target，要求 O(log n) 返回其下标，不存在返回 -1。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

线性扫描 O(n)。先找 pivot 再二分可 O(log n)，但可以合并成一次二分。

关键洞察：为什么这个数据结构合适

任意时刻 mid 会把区间分成两半，其中至少一半保持有序。先判断哪半有序，再判断 target 是否落在该有序值域，决定舍弃另一半。

> **不变量（Invariant）** 每次迭代都能依据“至少一半有序”排除 target 不可能所在的一半，搜索区间严格缩小。

参考实现（Python ）

```
def search_rotated(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        m = (l+r)//2
        if nums[m] == target: return m
        if nums[l] <= nums[m]:
            if nums[l] <= target < nums[m]: r = m-1
            else: l = m+1
        else:
            if nums[m] < target <= nums[r]: l = m+1
            else: r = m-1
    return -1
```

复杂度

时间 O(log n)，空间 O(1)。

高频失分点

- 不区分闭区间/半开区间模板

- 有重复元素时“哪一半有序”可能无法判断
- 边界比较符号写错造成死循环

面试官追问

- 允许重复元素后最坏复杂度为何可能退化为 O(n)?
- 如何找旋转数组最小值?

60 秒口述模板

我会先给出一个可工作的基线方案：线性扫描 O(n)。先找 pivot 再二分可 O(log n)，但可以合并成一次二分。 真正的瓶颈在重复工作，因此优化的核心是：任意时刻 mid 会把区间分成两半，其中至少一半保持有序。先判断哪半有序，再判断 target 是否落在该有序值域，决定舍弃另一半。 正确性依赖的不变量是：每次迭代都能依据“至少一半有序”排除 target 不可能所在的一半，搜索区间严格缩小。 最终复杂度为：时间 O(log n)，空间 O(1)。

Q011 合并重叠区间

难度 M	核心 区间 / 排序	类别 数组、字符串、矩阵与区间

题目

给定若干闭区间 [start,end]，合并所有相互重叠的区间。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 区间采用闭区间还是半开区间？端点相等时是否视为相交?

基线方案与瓶颈

反复扫描并合并任意相交区间，容易达到 O(n^2)。

关键洞察：为什么这个数据结构合适

按 start 升序排序后，当前区间只需要与结果列表最后一个区间比较：若 start <= last_end 就扩展 last_end，否则当前区间一定与此前所有结果都不相交。

不变量（Invariant） 结果列表始终按 start 排序且互不重叠；处理新区间时，只有最后一个结果区间可能与它相交。

参考实现（Python ）

```
def merge_intervals(intervals):
    if not intervals: return []
    intervals.sort(key=lambda x: x[0])
    out = [intervals[0][:]]
    for s, e in intervals[1:]:
        if s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

复杂度

时间 O(n log n)，排序外额外空间 O(n) 用于输出。

高频失分点

- 未明确端点接触 [1,2] 与 [2,3] 是否算重叠

- 直接修改输入却没确认是否允许
- 错误地只和原数组前一个区间比较，而非“当前合并结果”

面试官追问

- 插入一个新区间并合并如何做到 O(n)?
- 若区间来自在线数据流，使用什么有序结构?

60 秒口述模板

我会先给出一个可工作的基线方案：反复扫描并合并任意相交区间，容易达到 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：按 start 升序排序后，当前区间只需要与结果列表最后一个区间比较：若 start <= last_end 就扩展 last_end，否则当前区间一定与此前所有结果都不相交。 正确性依赖的不变量是：结果列表始终按 start 排序且互不重叠；处理新区间时，只有最后一个结果区间可能与它相交。 最终复杂度为：时间 O(n log n)，排序外额外空间 O(n) 用于输出。

Q012 员工 Shift 时间线：输出每段时间的在岗人员

难度 H	核心 Sweep Line / 有序集合	类别 数组、字符串、矩阵与区间

来源标记 公开候选人面经变体：Google L5，2025；候选人自述，非官方题库。

题目

输入若干 [name,start,end]，输出所有人员集合保持不变的最大时间段，以及该时间段内的在岗人员。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

把所有时间点切成最小单位并逐点检查不可行；对每个区间分别扫描所有 shift 是 O(n^2)。

关键洞察：为什么这个数据结构合适

把每个 start/end 转成事件，在时间轴上排序并扫描。相同时间的事件必须批量处理：先输出 [prev,t) 使用旧 active 集合，再统一应用 t 时刻的离开/进入事件。

不变量（Invariant） 在任意相邻事件时间之间，没有 shift 状态变化，所以 active 集合保持常量；因此这些区间正是答案的最细自然分段。

参考实现（Python ）

```
from collections import defaultdict

def shift_timeline(shifts):
    events = defaultdict(list)
    for name, s, e in shifts:
        events[s].append((1, name))
        events[e].append((-1, name))
    active, ans = set(), []
    prev = None
    for t in sorted(events):
        if prev is not None and prev < t and active:
            ans.append([prev, t, sorted(active)])
        for typ, name in events[t]:
            if typ == -1: active.discard(name)
        for typ, name in events[t]:
            if typ == 1: active.add(name)
        prev = t
    return ans
```

复杂度

排序 O(n log n)；若复制每段人员列表，输出成本还需计入总输出规模。

高频失分点

- 相同时间 start/end 的顺序没有定义清楚
- 把区间端点语义混成闭区间导致重复计入
- 忽略输出人员列表本身可能很大

面试官追问

- 若需要按员工优先级输出怎么办?
- 若输入持续在线到达，如何支持增量查询?
- 如果只要求人数，结构可简化成什么?

60 秒口述模板

我会先给出一个可工作的基线方案：把所有时间点切成最小单位并逐点检查不可行；对每个区间分别扫描所有 shift 是 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：把每个 start/end 转成事件，在时间轴上排序并扫描。相同时间的事件必须批量处理：先输出 [prev,t) 使用旧 active 集合，再统一应用 t 时刻的离开/进入事件。 正确性依赖的不变量是：在任意相邻事件时间之间，没有 shift 状态变化，所以 active 集合保持常量；因此这些区间正是答案的最细自然分段。 最终复杂度为：排序 O(n log n)；若复制每段人员列表，输出成本还需计入总输出规模。

Q013 矩阵螺旋遍历

难度 M	核心 矩阵 / 模拟	类别 数组、字符串、矩阵与区间

题目

按从外到内的顺时针螺旋顺序输出 m×n 矩阵元素。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

按“方向+visited”模拟可行，但需要额外 O(mn) visited，并容易出现方向切换 bug。

关键洞察：为什么这个数据结构合适

维护 top/bottom/left/right 四条未访问边界，每轮按上、右、下、左遍历一圈，完成后收缩对应边界。

不变量（Invariant） 四个边界始终围住所有尚未访问元素；每完成一条边后立即收缩，使任何单元格最多访问一次。

参考实现（Python ）

```
def spiral_order(mat):
    if not mat: return []
    top, bottom = 0, len(mat)-1
    left, right = 0, len(mat[0])-1
    ans = []
    while top <= bottom and left <= right:
        for j in range(left, right+1): ans.append(mat[top][j])
        top += 1
        for i in range(top, bottom+1): ans.append(mat[i][right])
        right -= 1
        if top <= bottom:
            for j in range(right, left-1, -1): ans.append(mat[bottom][j])
            bottom -= 1
        if left <= right:
            for i in range(bottom, top-1, -1): ans.append(mat[i][left])
            left += 1
    return ans
```

复杂度

时间 O(mn)，额外空间 O(1)（不含输出）。

高频失分点

- 单行/单列矩阵重复访问
- 收缩边界顺序与遍历顺序不匹配
- 空矩阵直接访问 mat[0]

面试官追问

- 逆时针如何改?
- 如果要求原地把矩阵按螺旋顺序重排怎么办?

60 秒口述模板

我会先给出一个可工作的基线方案：按“方向+visited”模拟可行，但需要额外 O(mn) visited，并容易出现方向切换 bug。 真正的瓶颈在重复工作，因此优化的核心是：维护 top/bottom/left/right 四条未访问边界，每轮按上、右、下、左遍历一圈，完成后收缩对应边界。 正确性依赖的不变量是：四个边界始终围住所有尚未访问元素；每完成一条边后立即收缩，使任何单元格最多访问一次。 最终复杂度为：时间 O(mn)，额外空间 O(1)（不含输出）。

Q014 第一个缺失的正整数

难度 H	核心 数组 / 原地哈希	类别 数组、字符串、矩阵与区间

题目

给定未排序整数数组，在 O(n) 时间和 O(1) 额外空间内找最小缺失正整数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

排序 O(n log n)；用 HashSet 可 O(n) 时间但需 O(n) 空间。

关键洞察：为什么这个数据结构合适

答案一定在 [1,n+1]。利用数组自身充当哈希表：只要 1<=x<=n，就尝试把 x 交换到下标 x-1；结束后第一个 nums[i]!=i+1 的位置即答案。

不变量（Invariant） 交换循环结束时，任何值 x∈[1,n] 若存在于数组，都被放到位置 x-1；扫描首个错位即得到最小缺失值。

参考实现（Python ）

```
def first_missing_positive(nums):
    n = len(nums)
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i]-1] != nums[i]:
            j = nums[i] - 1
            nums[i], nums[j] = nums[j], nums[i]
    for i, x in enumerate(nums):
        if x != i + 1:
            return i + 1
    return n + 1
```

复杂度

时间 O(n)：每次交换至少让一个有效值就位；额外空间 O(1)。

高频失分点

- 重复值导致无限交换，必须判断目标位置是否已经是同值
- 把 0 或负数也参与索引

- 没有先证明答案上界 n+1

面试官追问

- 如果数组只读，最优空间/时间 trade-off 是什么?
- 类似技巧还能解决 Find All Missing Numbers 吗?

60 秒口述模板

我会先给出一个可工作的基线方案：排序 O(n log n)；用 HashSet 可 O(n) 时间但需 O(n) 空间。 真正的瓶颈在重复工作，因此优化的核心是：答案一定在 [1,n+1]。利用数组自身充当哈希表：只要 1<=x<=n，就尝试把 x 交换到下标 x-1；结束后第一个 nums[i]!=i+1 的位置即答案。 正确性依赖的不变量是：交换循环结束时，任何值 x∈[1,n] 若存在于数组，都被放到位置 x-1；扫描首个错位即得到最小缺失值。 最终复杂度为：时间 O(n)：每次交换至少让一个有效值就位；额外空间 O(1)。

Q015 两个有序数组的中位数

难度 H+	核心 二分查找 / 分割	类别 数组、字符串、矩阵与区间

题目

给定两个有序数组，总长度为 m+n，要求 O(log(min(m,n))) 求联合中位数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

直接归并到中位位置需要 O(m+n)；双指针仍是线性。

关键洞察：为什么这个数据结构合适

在较短数组上二分切分位置 i，由总左半元素个数推得另一个数组切分 j。寻找满足 A[i-1]<=B[j] 且 B[j-1]<=A[i] 的分割，此时左右两边已经全局有序分离。

不变量（Invariant） 二分变量 i 过大时 A[i-1]>B[j]，必须左移；过小时 B[j-1]>A[i]，必须右移。合法分割唯一地确定中位数。

参考实现（Python ）

```
def find_median_sorted_arrays(a, b):
    if len(a) > len(b): return find_median_sorted_arrays(b, a)
    m, n = len(a), len(b)
    total_left = (m+n+1)//2
    lo, hi = 0, m
    while lo <= hi:
        i = (lo+hi)//2
        j = total_left - i
        al = float('-inf') if i == 0 else a[i-1]
        ar = float('inf') if i == m else a[i]
        bl = float('-inf') if j == 0 else b[j-1]
        br = float('inf') if j == n else b[j]
        if al <= br and bl <= ar:
            if (m+n)%2: return max(al, bl)
            return (max(al,bl)+min(ar,br))/2
        if al > br: hi = i-1
        else: lo = i+1
```

复杂度

时间 O(log(min(m,n)))，空间 O(1)。

高频失分点

- 没有强制在较短数组上二分
- 偶数长度时忘记取左右边界平均
- 大量边界 if 导致代码失控，±inf 能显著简化

面试官追问

- 如何求两个有序数组联合第 K 小?
- 推广到多个有序数组，中位数还能保持对数复杂度吗?

60 秒口述模板

我会先给出一个可工作的基线方案：直接归并到中位位置需要 O(m+n)；双指针仍是线性。 真正的瓶颈在重复工作，因此优化的核心是：在较短数组上二分切分位置 i，由总左半元素个数推得另一个数组切分 j。寻找满足 A[i-1]<=B[j] 且 B[j-1]<=A[i] 的分割，此时左右两边已经全局有序分离。 正确性依赖的不变量是：二分变量 i 过大时 A[i-1]>B[j]，必须左移；过小时 B[j-1]>A[i]，必须右移。合法分割唯一地确定中位数。 最终复杂度为：时间 O(log(min(m,n)))，空间 O(1)。

第二章

HashMap / HashSet

Q016 - Q023

理解 O(1) lookup 的价值，也理解它只是平均复杂度；从应用走到底层冲突与扩容。

Q016 有效字母异位词

难度 E	核心 Hash / 计数	类别 HashMap / HashSet

题目

判断两个字符串是否由完全相同的字符及次数组成。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

排序两个字符串后比较，时间 O(n log n)。

关键洞察：为什么这个数据结构合适

用频次表统计字符净变化；固定小字符集时直接使用数组比 HashMap 更轻量。

> **不变量（Invariant）** 处理完所有字符后，每个字符计数必须回到 0。

参考实现（Python ）

```
def is_anagram(s, t):
    if len(s) != len(t): return False
    cnt = {}
    for a, b in zip(s, t):
        cnt[a] = cnt.get(a, 0) + 1
        cnt[b] = cnt.get(b, 0) - 1
    return all(v == 0 for v in cnt.values())
```

复杂度

时间 O(n)，空间 O(字符集)。

高频失分点

- 只比较字符集合，忘记 multiplicity
- ASCII/Unicode 假设未澄清

面试官追问

- 若字符仅为 a-z，怎样避免 HashMap?
- 海量字符串外存场景如何做?

60 秒口述模板

我会先给出一个可工作的基线方案：排序两个字符串后比较，时间 O(n log n)。 真正的瓶颈在重复工作，因此优化的核心是：用频次表统计字符净变化；固定小字符集时直接使用数组比 HashMap 更轻量。 正确性依赖的不变量是：处理完所有字符后，每个字符计数必须回到 0。 最终复杂度为：时间 O(n)，空间 O(字符集)。

Q017 按字母异位词分组

难度 M	核心 Hash / 字符串签名	类别 HashMap / HashSet

题目

把字符串数组按“互为字母异位词”分组。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每对字符串比较会 O(n^2*k)。排序每个字符串作为 key 是常见 O(n*k log k) 方案。

关键洞察：为什么这个数据结构合适

若字符集固定，可用 26 维计数元组作为 canonical signature，把单字符串建 key 降为 O(k)。

不变量（Invariant） 两个字符串互为异位词，当且仅当它们的字符频次向量完全相同。

参考实现（Python ）

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        cnt = [0]*26
        for c in w:
            cnt[ord(c)-97] += 1
        groups[tuple(cnt)].append(w)
    return list(groups.values())
```

复杂度

固定字符集时 O(总字符数)，空间 O(n*k) 用于输出与 key。

高频失分点

- 列表不能作为 Python dict key，需要转 tuple
- 字符集不是 a-z 时计数向量方案需要调整

面试官追问

- Unicode 文本最合适的 signature 是什么?
- 如果要保持每组原输入顺序怎么办?

60 秒口述模板

我会先给出一个可工作的基线方案：每对字符串比较会 O(n^2*k)。排序每个字符串作为 key 是常见 O(n*k log k) 方案。 真正的瓶颈在重复工作，因此优化的核心是：若字符集固定，可用 26 维计数元组作为 canonical signature，把单字符串建 key 降为 O(k)。 正确性依赖的不变量是：两个字符串互为异位词，当且仅当它们的字符频次向量完全相同。 最终复杂度为：固定字符集时 O(总字符数)，空间 O(n*k) 用于输出与 key。

Q018 最长连续整数序列

难度 M	核心 HashSet / 集合	类别 HashMap / HashSet

题目

给定未排序整数数组，求最长“值连续”序列长度，要求 O(n)。例如 [100,4,200,1,3,2] 返回 4。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

排序后线性扫描是 O(n log n)。从每个元素向两侧扩展会重复工作到 O(n^2)。

关键洞察：为什么这个数据结构合适

只有当 x-1 不在集合中时，x 才可能是某段连续序列的起点。只从起点向右扩展，因此每个整数最多参与一次有效扩展。

不变量（Invariant） 每条连续链只会由其唯一最小元素启动一次。

参考实现（Python ）

```
def longest_consecutive(nums):
    s = set(nums)
    best = 0
    for x in s:
        if x-1 not in s:
            y = x
            while y in s: y += 1
            best = max(best, y-x)
    return best
```

复杂度

期望时间 O(n)，空间 O(n)。

高频失分点

- 对所有 x 都向右扩展，退化 O(n^2)
- 重复元素未去重
- 把“连续”误解为数组下标连续

面试官追问

- 如果数据量大到内存放不下怎么办?
- 若要动态 insert 并实时查询最长连续段，如何用 DSU?

60 秒口述模板

我会先给出一个可工作的基线方案：排序后线性扫描是 O(n log n)。从每个元素向两侧扩展会重复工作到 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：只有当 x-1 不在集合中时，x 才可能是某段连续序列的起点。只从起点向右扩展，因此每个整数最多参与一次有效扩展。 正确性依赖的不变量是：每条连续链只会由其唯一最小元素启动一次。 最终复杂度为：期望时间 O(n)，空间 O(n)。

Q019 K 距离内是否存在重复值

难度 E/M	核心 HashMap / 滑动集合	类别 HashMap / HashSet

题目

判断是否存在 i!=j，使 nums[i]=nums[j] 且 |i-j|<=k。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举所有距离不超过 k 的下标对，最坏 O(nk)。

关键洞察：为什么这个数据结构合适

可记录每个值最后出现下标，也可维护一个只包含最近 k 个元素的 HashSet。前者更直接。

不变量（Invariant） 扫描到 i 时，如果 last[x] 存在，它就是离 i 最近的同值位置；若该距离仍大于 k，更早位置也不可能满足。

参考实现（Python ）

```
def contains_nearby_duplicate(nums, k):
    last = {}
    for i, x in enumerate(nums):
        if x in last and i-last[x] <= k:
            return True
        last[x] = i
    return False
```

复杂度

时间 O(n)，空间 O(min(n,不同值数))。

高频失分点

- 存第一次出现位置而非最近位置
- k=0 的定义没处理清楚

面试官追问

- 如果再要求 |nums[i]-nums[j]|<=t，需要什么有序结构/桶化技巧?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举所有距离不超过 k 的下标对，最坏 O(nk)。 真正的瓶颈在重复工作，因此优化的核心是：可记录每个值最后出现下标，也可维护一个只包含最近 k 个元素的 HashSet。前者更直接。 正确性依赖的不变量是：扫描到 i 时，如果 last[x] 存在，它就是离 i 最近的同值位置；若该距离仍大于 k，更早位置也不可能满足。 最终复杂度为：时间 O(n)，空间 O(min(n,不同值数))。

Q020 O(1) Insert/Delete/GetRandom 集合

难度 M	核心 设计 / HashMap	类别 HashMap / HashSet

题目

设计集合，支持 insert、remove、getRandom，三者平均 O(1)。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

只用数组 getRandom O(1) 但 remove O(n)；只用 HashSet 无法按均匀概率随机访问。

关键洞察：为什么这个数据结构合适

组合数组和 value→index 哈希。删除任意元素时，将数组尾元素交换到待删位置，再 pop 尾部，同时修正尾元素 index。

不变量（Invariant） 哈希表与数组始终一一对应：pos[val] 等于 val 在数组中的真实位置。交换删除保持数组紧凑，因此 random choice 可 O(1)。

参考实现（Python ）

```
import random
class RandomizedSet:
    def __init__(self): self.a, self.pos = [], {}
    def insert(self, x):
        if x in self.pos: return False
        self.pos[x] = len(self.a); self.a.append(x); return True
    def remove(self, x):
        if x not in self.pos: return False
        i = self.pos[x]; last = self.a[-1]
        self.a[i] = last; self.pos[last] = i
        self.a.pop(); del self.pos[x]
        return True
    def getRandom(self): return random.choice(self.a)
```

复杂度

三种操作平均 O(1)，空间 O(n)。

高频失分点

- 删除的恰好是尾元素时仍要保证更新顺序安全
- 数组出现空洞会破坏均匀随机与 O(1) 访问

面试官追问

- 允许重复元素怎样改？见 Q97
- 如何按权重随机抽样?

60 秒口述模板

我会先给出一个可工作的基线方案：只用数组 getRandom O(1) 但 remove O(n)；只用 HashSet 无法按均匀概率随机访问。 真正的瓶颈在重复工作，因此优化的核心是：组合数组和 value→index 哈希。删除任意元素时，将数组尾元素交换到待删位置，再 pop 尾部，同时修正尾元素 index。 正确性依赖的不变量是：哈希表与数组始终一一对应：pos[val] 等于 val 在数组中的真实位置。交换删除保持数组紧凑，因此 random choice 可 O(1)。 最终复杂度为：三种操作平均 O(1)，空间 O(n)。

Q021 从零实现 HashMap

难度 M	核心 HashMap / 底层实现	类别 HashMap / HashSet

题目

不使用语言内置 dict/map，实现 put/get/remove，并讨论扩容与冲突处理。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

最简单可以使用固定桶数组 + 链表 separate chaining，但固定容量会随着 n 增长导致冲突严重。

关键洞察：为什么这个数据结构合适

将 key 通过 hash 映射到 bucket；桶内线性查找同 key。负载因子超过阈值时把 capacity 扩大并 rehash 所有键。面试重点是理解“平均 O(1)”来自桶分布与受控负载。

不变量（Invariant） 每个键只存在于由当前容量和 hash 决定的唯一 bucket；resize 后必须对所有键重新取模，不能仅复制桶数组。

参考实现（Python ）

```
class MyHashMap:
    def __init__(self, cap=8):
        self.cap, self.n = cap, 0
        self.b = [[] for _ in range(cap)]
    def _idx(self, k): return hash(k) % self.cap
    def get(self, k, default=None):
        for key, val in self.b[self._idx(k)]:
            if key == k: return val
        return default
    def put(self, k, v):
        bucket = self.b[self._idx(k)]
        for i, (key, _) in enumerate(bucket):
            if key == k: bucket[i] = (k, v); return
        bucket.append((k, v)); self.n += 1
        if self.n / self.cap > 0.75: self._resize()
    def remove(self, k):
        bucket = self.b[self._idx(k)]
        for i, (key, _) in enumerate(bucket):
            if key == k: bucket.pop(i); self.n -= 1; return True
        return False
    def _resize(self):
```

```
        old = [item for bucket in self.b for item in bucket]
        self.cap *= 2; self.b = [[] for _ in range(self.cap)]
        self.n = 0
        for k, v in old: self.put(k, v)
```

复杂度

平均 put/get/remove O(1)，最坏 O(n)；resize 一次 O(n) 但摊销后 insert 仍 O(1)。

高频失分点

- resize 后不 rehash
- 用 mutable key 却忽略 hash 稳定性
- 把最坏 O(n) 错说成严格 O(1)

面试官追问

- Open Addressing 如何处理 tombstone?
- 为什么容量常选 2 的幂或质数?
- 如何抵御恶意 hash collision?

60 秒口述模板

我会先给出一个可工作的基线方案：最简单可以使用固定桶数组 + 链表 separate chaining，但固定容量会随着 n 增长导致冲突严重。 真正的瓶颈在重复工作，因此优化的核心是：将 key 通过 hash 映射到 bucket；桶内线性查找同 key。负载因子超过阈值时把 capacity 扩大并 rehash 所有键。面试重点是理解“平均 O(1)”来自桶分布与受控负载。 正确性依赖的不变量是：每个键只存在于由当前容量和 hash 决定的唯一 bucket；resize 后必须对所有键重新取模，不能仅复制桶数组。 最终复杂度为：平均 put/get/remove O(1)，最坏 O(n)；resize 一次 O(n) 但摊销后 insert 仍 O(1)。

Q022 Hash Collision、负载因子与复杂度退化

难度 M	核心 HashMap / 理论	类别 HashMap / HashSet

题目

解释哈希冲突的来源、常见处理策略、负载因子如何影响性能，以及为什么 HashMap 的 O(1) 通常只是平均/期望复杂度。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

若把每个键都放进一个线性列表，则查询 O(n)，完全失去哈希价值。

关键洞察：为什么这个数据结构合适

Separate chaining 在每个 bucket 保存链/树；open addressing 通过探测序列寻找空位。负载因子 α=n/m 越高，冲突和探测长度通常越大，因此需要扩容。

不变量（Invariant） 良好 hash 与合理 α 使键近似均匀落桶，期望桶长度维持常数；若大量 key 落到同桶，则查询退化。

参考实现（Python ）

```
def bucket_index(key, capacity):
    # 概念示意：真实实现还会处理哈希扰动、容量策略等
    return hash(key) % capacity
```

复杂度

平均 O(1)，极端/攻击性输入可能 O(n)；具体取决于实现是否树化、随机化 hash 等。

高频失分点

- 把“无冲突”当成 HashMap 的前提
- 不区分 hash 相同与取模后 bucket 相同
- 忽略 resize 的摊销分析

面试官追问

- Separate chaining 与 linear probing 的 cache locality 谁更好?

- 删除在 open addressing 中为何需要 tombstone?
- Java HashMap 树化思想解决了什么最坏情况?

60 秒口述模板

我会先给出一个可工作的基线方案：若把每个键都放进一个线性列表，则查询 O(n)，完全失去哈希价值。 真正的瓶颈在重复工作，因此优化的核心是：Separate chaining 在每个 bucket 保存链/树；open addressing 通过探测序列寻找空位。负载因子 α=n/m 越高，冲突和探测长度通常越大，因此需要扩容。 正确性依赖的不变量是：良好 hash 与合理 α 使键近似均匀落桶，期望桶长度维持常数；若大量 key 落到同桶，则查询退化。 最终复杂度为：平均 O(1)，极端/攻击性输入可能 O(n)；具体取决于实现是否树化、随机化 hash 等。

Q023 稀疏向量点积

难度 M	核心 HashMap / 双指针	类别 HashMap / HashSet

题目

两个高维向量绝大多数元素为 0，计算点积；应利用稀疏性而不是扫描完整维度。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

直接遍历所有维度 O(D)，当 D 很大而非零元素很少时浪费。

关键洞察：为什么这个数据结构合适

把非零元素存成 (index,value)。若双方都按 index 有序，可双指针 O(nnz1+nnz2)；若一方很稀疏，也可把较小方建哈希并遍历另一方。

> **不变量（Invariant）** 只有 index 相同的非零项对点积有贡献；双指针每步至少推进一方，因此线性于非零项总数。

参考实现（Python ）

```
def sparse_dot(a, b):  # a,b: sorted list of (idx,val)
    i = j = ans = 0
    while i < len(a) and j < len(b):
        if a[i][0] == b[j][0]:
            ans += a[i][1]*b[j][1]; i += 1; j += 1
        elif a[i][0] < b[j][0]: i += 1
        else: j += 1
    return ans
```

复杂度

排序表示已给定时 O(nnz1+nnz2)，额外 O(1)。

高频失分点

- 仍扫描完整维度
- 忽略构建稀疏表示本身的成本
- 未根据双方 sparsity 选择 Hash 或双指针

面试官追问

- 一边极稀疏、一边较稠密时最优表示是什么?
- 矩阵乘法中的 CSR/CSC 与这个问题有什么联系?

60 秒口述模板

我会先给出一个可工作的基线方案：直接遍历所有维度 O(D)，当 D 很大而非零元素很少时浪费。 真正的瓶颈在重复工作，因此优化的核心是：把非零元素存成 (index,value)。若双方都按 index 有序，可双指针 O(nnz1+nnz2)；若一方很稀疏，也可把较小方建哈希并遍历另一方。 正确性依赖的不变量是：只有 index 相同的非零项对点积有贡献；双指针每步至少推进一方，因此线性于非零项总数。 最终复杂度为：排序表示已给定时 O(nnz1+nnz2)，额外 O(1)。

第三章

链表

Q024 - Q033

核心不是 API，而是指针不变量：保存 next、Dummy、快慢指针、局部反转。

Q024 反转单链表

难度 E	核心 链表 / 指针	类别 链表

题目

给定单链表头节点，原地反转链表并返回新头。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

把节点值复制到数组后逆序再写回并没有真正反转链接；递归可行但会使用 O(n) 调用栈。

关键洞察：为什么这个数据结构合适

迭代维护 prev/curr/nxt 三个角色。修改 curr.next 前必须先保存 nxt，否则剩余链表会丢失。

不变量（Invariant） 每轮结束后，prev 指向已经完全反转的前缀，curr 指向尚未处理的后缀头。

参考实现（Python ）

```
def reverse_list(head):
    prev, cur = None, head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev, cur = cur, nxt
    return prev
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 先改 cur.next 再取 next，导致丢链
- 返回 head 而不是 prev
- 空链/单节点边界

面试官追问

- 递归版本如何写?
- 如何反转区间 [m,n]?
- K 个一组反转如何复用这个 primitive?

60 秒口述模板

我会先给出一个可工作的基线方案：把节点值复制到数组后逆序再写回并没有真正反转链接；递归可行但会使用 O(n) 调用栈。 真正的瓶颈在重复工作，因此优化的核心是：迭代维护 prev/curr/nxt 三个角色。修改 curr.next 前必须先保存 nxt，否则剩余链表会丢失。 正确性依赖的不变量是：每轮结束后，prev 指向已经完全反转的前缀，curr 指向尚未处理的后缀头。 最终复杂度为：时间 O(n)，空间 O(1)。

Q025 判断链表是否有环

难度 E/M	核心 链表 / Floyd	类别 链表

题目

给定单链表，判断是否存在节点通过 next 最终再次访问到已访问节点。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

用 HashSet 记录所有节点可 O(n) 时间、O(n) 空间。

关键洞察：为什么这个数据结构合适

Floyd 快慢指针：slow 每次一步、fast 每次两步。若无环 fast 会到 null；若有环，进入环后 fast 相对 slow 每轮追近一步，最终相遇。

> **不变量（Invariant）** 在有限长度环上，两个指针的相对位置模环长每轮增加 1，因此必然出现差值 0。

参考实现（Python ）

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 比较节点值而非节点身份
- while 条件忘记 fast.next
- 说“fast 一定追上 slow”却解释不出相对速度

面试官追问

- 如何找到环入口？见 Q26
- 如何计算环长度?

60 秒口述模板

我会先给出一个可工作的基线方案：用 HashSet 记录所有节点可 O(n) 时间、O(n) 空间。 真正的瓶颈在重复工作，因此优化的核心是：Floyd 快慢指针：slow 每次一步、fast 每次两步。若无环 fast 会到 null；若有环，进入环后 fast 相对 slow 每轮追近一步，最终相遇。 正确性依赖的不变量是：在有限长度环上，两个指针的相对位置模环长每轮增加 1，因此必然出现差值 0。 最终复杂度为：时间 O(n)，空间 O(1)。

Q026 寻找链表环入口

难度 M	核心 链表 / Floyd	类别 链表

题目

已知链表可能含环，若有环返回环的第一个节点，否则返回 None。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

用 HashSet 第一个重复节点即入口，空间 O(n)。

关键洞察：为什么这个数据结构合适

先用快慢指针求一个相遇点。设头到入口距离 μ、环长 λ；相遇时 slow 走过 μ+x，fast 走过 2(μ+x)，差值是 λ 的整数倍，因此 μ 与从相遇点继续到入口的距离同余。把一个指针放回 head，两者每次一步，相遇点就是入口。

不变量（Invariant） 第二阶段两指针分别走 μ 步：头指针到入口；环内指针也恰好从相遇点走到入口。

参考实现（Python ）

```
def detect_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            p = head
            while p is not slow:
                p, slow = p.next, slow.next
            return p
    return None
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 只会背“相遇后回头”但不能给出距离关系

- 第二阶段 fast 仍走两步
- 无环时未返回 None

面试官追问

- 证明可用模运算还是路程方程?
- 如果链表结构允许多个 outgoing edge，这个技巧为何失效?

60 秒口述模板

我会先给出一个可工作的基线方案：用 HashSet 第一个重复节点即入口，空间 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：先用快慢指针求一个相遇点。设头到入口距离 μ、环长 λ；相遇时 slow 走过 μ+x，fast 走过 2(μ+x)，差值是 λ 的整数倍，因此 μ 与从相遇点继续到入口的距离同余。把一个指针放回 head，两者每次一步，相遇点就是入口。 正确性依赖的不变量是：第二阶段两指针分别走 μ 步：头指针到入口；环内指针也恰好从相遇点走到入口。 最终复杂度为：时间 O(n)，空间 O(1)。

Q027 合并两个有序链表

难度 E	核心 链表 / 归并	类别 链表

题目

合并两个按升序排列的单链表，返回一个新的升序链表，可复用原节点。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

把所有值取出、排序、重建节点为 O((m+n)log(m+n))。

关键洞察：为什么这个数据结构合适

与归并排序 merge 步骤完全相同。使用 dummy 头节点消除“结果链表第一个节点”的特殊分支。

不变量（Invariant） 每轮 tail 后方已经是全部已消费元素的有序合并结果；比较两个当前头只可能选择更小者。

参考实现（Python ）

```
def merge_two_lists(a, b):
    dummy = tail = ListNode(0)
    while a and b:
        if a.val <= b.val: tail.next, a = a, a.next
        else: tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

复杂度

时间 O(m+n)，空间 O(1)（复用节点）。

高频失分点

- 忘记连接剩余链
- 不用 dummy 导致大量 head 特判
- 同时改动 a.next 和 a 的顺序错误

面试官追问

- 递归版本复杂度如何?
- 如果节点不可修改，需要怎样复制?

60 秒口述模板

我会先给出一个可工作的基线方案：把所有值取出、排序、重建节点为 O((m+n)log(m+n))。 真正的瓶颈在重复工作，因此优化的核心是：与归并排序 merge 步骤完全相同。使用 dummy 头节点消除“结果链表第一个节点”的特殊分支。正确性依赖的不变量是：每轮 tail 后方已经是全部已消费元素的有序合并结果；比较两个当前头只可能选择更小者。 最终复杂度为：时间 O(m+n)，空间 O(1)（复用节点）。

Q028 合并 K 个有序链表

难度 H	核心 链表 / Heap	类别 链表

题目

给定 k 个升序链表，总节点数 N，合并为一条升序链表。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

逐个把当前结果与下一条链表合并，最坏可接近 O(kN)；把所有节点放数组排序 O(N log N)。

关键洞察：为什么这个数据结构合适

Heap 方案只维护 k 个链表当前头节点，每取出最小节点后放入它的 next，复杂度 O(N log k)。分治两两归并也可达到同阶。

> **不变量（Invariant）** 堆中始终包含每条未耗尽链表的最小剩余元素，因此全局最小剩余节点一定在堆顶。

参考实现（Python ）

```
import heapq

def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node: heapq.heappush(heap, (node.val, i, node))
    dummy = tail = ListNode(0)
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node; tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

复杂度

时间 O(N log k)，堆空间 O(k)。

高频失分点

- Python heap 元组值相同会继续比较 node 而报错，因此加入唯一 i
- 把所有 N 个节点都放 heap，空间与常数变大
- 没比较分治方案

面试官追问

- 若 k 极大而每条链很短，分治与 heap 的 cache 行为如何?
- 外存 K 路归并为什么同样用 heap?

60 秒口述模板

我会先给出一个可工作的基线方案：逐个把当前结果与下一条链表合并，最坏可接近 O(kN)；把所有节点放数组排序 O(N log N)。 真正的瓶颈在重复工作，因此优化的核心是：Heap 方案只维护 k 个链表当前头节点，每取出最小节点后放入它的 next，复杂度 O(N log k)。分治两两归并也可达到同阶。 正确性依赖的不变量是：堆中始终包含每条未耗尽链表的最小剩余元素，因此全局最小剩余节点一定在堆顶。 最终复杂度为：时间 O(N log k)，堆空间 O(k)。

Q029 删除倒数第 N 个节点

难度 M	核心 链表 / 双指针	类别 链表

题目

删除单链表倒数第 n 个节点并返回头。常见要求一趟扫描。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

先统计长度 L，再删除第 L-n 个节点，需要两趟。

关键洞察：为什么这个数据结构合适

使用 dummy 节点。fast 先比 slow 多走 n+1 步，然后同步移动，fast 到 null 时 slow 正好位于待删节点的前驱。

> **不变量（Invariant）** fast 与 slow 始终相差 n+1 个链接；fast 到链尾之后，slow.next 距离尾部正好 n。

参考实现（Python ）

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n+1): fast = fast.next
    while fast:
        fast, slow = fast.next, slow.next
    slow.next = slow.next.next
    return dummy.next
```

复杂度

时间 O(n_nodes)，空间 O(1)。

高频失分点

- 不用 dummy 导致删除 head 特判
- 快指针领先 n 还是 n+1 的模板混淆
- 未确认 n 一定合法

面试官追问

- 如果 n 可能非法如何健壮处理?
- 单向链表为什么不能从尾部直接访问?

60 秒口述模板

我会先给出一个可工作的基线方案：先统计长度 L，再删除第 L-n 个节点，需要两趟。 真正的瓶颈在重复工作，因此优化的核心是：使用 dummy 节点。fast 先比 slow 多走 n+1 步，然后同步移动，fast 到 null 时 slow 正好位于待删节点的前驱。 正确性依赖的不变量是：fast 与 slow 始终相差 n+1 个链接；fast 到链尾之后，slow.next 距离尾部正好 n。 最终复杂度为：时间 O(n_nodes)，空间 O(1)。

Q030 重排链表 L0→Ln→L1→Ln-1...

难度 M	核心 链表 / 快慢指针	类别 链表

题目

在不改变节点值的前提下，把 L0→L1→…→Ln 重排成 L0→Ln→L1→Ln-1→…。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

把节点放数组后双指针重连可 O(n) 时间、O(n) 空间。

关键洞察：为什么这个数据结构合适

三段式：快慢指针找到中点；反转后半链表；前后两半交错 merge。这个题本质上是三种链表 primitive 的组合。

不变量（Invariant） 后半反转后，两个链头分别按所需次序提供“前端”和“后端”节点；每轮各消费一个即可保持目标排列。

参考实现（Python ）

```
def reorder_list(head):
    if not head or not head.next: return
    slow = fast = head
    while fast.next and fast.next.next:
        slow, fast = slow.next, fast.next.next
    second = slow.next; slow.next = None
    prev = None
    while second:
        nxt = second.next; second.next = prev
        prev, second = second, nxt
    a, b = head, prev
    while b:
        an, bn = a.next, b.next
        a.next = b; b.next = an
        a, b = an, bn
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 找中点后没断开前半导致形成环
- merge 时覆盖 next 丢链
- 奇偶长度边界

面试官追问

- 能否把三阶段写成可复用 helper?
- 如何证明最终不会产生 cycle?

60 秒口述模板

我会先给出一个可工作的基线方案：把节点放数组后双指针重连可 O(n) 时间、O(n) 空间。 真正的瓶颈在重复工作，因此优化的核心是：三段式：快慢指针找到中点；反转后半链表；前后两半交错 merge。这个题本质上是三种链表 primitive 的组合。 正确性依赖的不变量是：后半反转后，两个链头分别按所需次序提供“前端”和“后端”节点；每轮各消费一个即可保持目标排列。 最终复杂度为：时间 O(n)，空间 O(1)。

Q031 两个单链表的交点

难度 E/M	核心 链表 / 双指针	类别 链表

题目

给定两个无环单链表，返回它们第一个共享的节点对象；若不相交返回 None。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

用 set 存一条链的所有节点，空间 O(m)。先求长度差再对齐也可 O(1) 空间。

关键洞察：为什么这个数据结构合适

更简洁的双指针：p 走完 A 后转到 B，q 走完 B 后转到 A。两者都走了 m+n 距离，因此在共同尾部起点同步，或同时到 None。

> **不变量（Invariant）** p 和 q 在最多 m+n 步后经过相同总路程，长度差被“交换链表”自动抵消。

参考实现（Python ）

```
def get_intersection_node(a, b):
    p, q = a, b
    while p is not q:
        p = b if p is None else p.next
        q = a if q is None else q.next
    return p
```

复杂度

时间 O(m+n)，空间 O(1)。

高频失分点

- 比较节点值而非 identity
- 若链表有环，本算法前提改变
- 写成 p.next if p else b 时注意空指针

面试官追问

- 若两链各自可能有环，交点情况怎样分类?

60 秒口述模板

我会先给出一个可工作的基线方案：用 set 存一条链的所有节点，空间 O(m)。先求长度差再对齐也可 O(1) 空间。 真正的瓶颈在重复工作，因此优化的核心是：更简洁的双指针：p 走完 A 后转到 B，q 走完 B 后转到 A。两者都走了 m+n 距离，因此在共同尾部起点同步，或同时到 None。 正确性依赖的不变量是：p 和 q 在最多 m+n 步后经过相同总路程，长度差被“交换链表”自动抵消。 最终复杂度为：时间 O(m+n)，空间 O(1)。

Q032 复制带 Random Pointer 的链表

难度 M/H	核心 链表 / HashMap	类别 链表

题目

每个节点除了 next 还有 random，可指向任意节点或 None。深拷贝整条链。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

最直观两遍 HashMap：第一遍建 old→new，第二遍填 next/random，时间 O(n)、空间 O(n)。

关键洞察：为什么这个数据结构合适

进阶 O(1) 额外空间：把复制节点插入原节点之后 old→copy→oldNext。此时 old.random.next 就是 copy.random。最后再拆分两条链。

> **不变量（Invariant）** 插入阶段建立局部映射 copy(old)=old.next；因此不需要显式哈希就能根据任意 old.random 定位其复制节点。

参考实现（Python ）

```
def copy_random_list(head):
    if not head: return None
    cur = head
    while cur:
        cp = Node(cur.val); cp.next = cur.next; cur.next = cp; cur = cp.next
    cur = head
    while cur:
        cur.next.random = cur.random.next if cur.random else None
        cur = cur.next.next
    new_head = head.next; cur = head
    while cur:
        cp = cur.next; cur.next = cp.next
        cp.next = cp.next.next if cp.next else None
        cur = cur.next
    return new_head
```

复杂度

时间 O(n)，额外空间 O(1)（不计新链表节点）。

高频失分点

- 拆分阶段破坏原链
- random=None 时空指针
- 把空间复杂度误说成 O(1) 却忽略输出本身；应注明“不计输出”

面试官追问

- 如果节点还有多个任意引用边，本质上是否变成 Clone Graph?

60 秒口述模板

我会先给出一个可工作的基线方案：最直观两遍 HashMap：第一遍建 old→new，第二遍填 next/random，时间 O(n)、空间 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：进阶 O(1) 额外空间：把复制节点插入原节点之后 old→copy→oldNext。此时 old.random.next 就是 copy.random。最后再拆分两条链。 正确性依赖的不变量是：插入阶段建立局部映射 copy(old)=old.next；因此不需要显式哈希就能根据任意 old.random 定位其复制节点。 最终复杂度为：时间 O(n)，额外空间 O(1)（不计新链表节点）。

Q033 回文链表

难度 M	核心 链表 / 快慢指针	类别 链表

题目

判断单链表节点值序列是否为回文，目标 O(n) 时间、O(1) 额外空间。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 链表是否保证无环？节点比较按值还是节点身份?

基线方案与瓶颈

复制到数组后双指针最简单，时间 O(n)、空间 O(n)。

关键洞察：为什么这个数据结构合适

快慢指针找到中点，反转后半部分，再从头与后半头逐个比较。工程上可在比较后再次反转恢复原链。

> **不变量（Invariant）** 反转后半后，前半从左到右与原序列后半从右到左一一对应。

参考实现（Python ）

```
def is_palindrome(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
    prev = None
    while slow:
        nxt = slow.next; slow.next = prev; prev, slow = slow, nxt
    p, q = head, prev
    while q:
        if p.val != q.val: return False
        p, q = p.next, q.next
    return True
```

复杂度

时间 O(n)，空间 O(1)。

高频失分点

- 奇数长度中点处理混乱
- 忘记“是否允许修改输入”这一面试澄清

- 比较长度不一致

面试官追问

- 如何恢复原链表?
- 递归方案的隐含空间复杂度是多少?

60 秒口述模板

我会先给出一个可工作的基线方案：复制到数组后双指针最简单，时间 O(n)、空间 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：快慢指针找到中点，反转后半部分，再从头与后半头逐个比较。工程上可在比较后再次反转恢复原链。正确性依赖的不变量是：反转后半后，前半从左到右与原序列后半从右到左一一对应。 最终复杂度为：时间 O(n)，空间 O(1)。

第四章

Stack / Queue / Deque

Q034 - Q043

嵌套结构、最近未完成状态、单调关系、FIFO 层级搜索。

Q034 有效括号

难度 E	核心 Stack / 字符串	类别 Stack / Queue / Deque

题目

给定只包含括号字符的字符串，判断括号是否以正确类型、正确嵌套顺序闭合。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

反复删除 "()"、"[]"、"{}" 虽可工作但可能 O(n^2)。

关键洞察：为什么这个数据结构合适

左括号入栈；右括号必须与当前最近未匹配的左括号即栈顶配对。嵌套结构天然对应 LIFO。

> **不变量（Invariant）** 栈从底到顶保存所有尚未闭合的左括号，且顺序就是未来必须被关闭的逆序。

参考实现（Python ）

```
def is_valid_parentheses(s):
    match = {')':'(', ']':'[', '}':'{'}
    st = []
    for c in s:
        if c in '([{': st.append(c)
        elif not st or st.pop() != match[c]: return False
    return not st
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 右括号到来时空栈
- 结束后忘记检查栈是否为空
- 只计数左右括号数量而不检查嵌套类型

面试官追问

- 若括号字符串中夹杂普通字符如何处理?

- 如何返回第一个错误位置?

60 秒口述模板

我会先给出一个可工作的基线方案：反复删除 "()"、"[]"、"{}" 虽可工作但可能 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：左括号入栈；右括号必须与当前最近未匹配的左括号即栈顶配对。嵌套结构天然对应 LIFO。 正确性依赖的不变量是：栈从底到顶保存所有尚未闭合的左括号，且顺序就是未来必须被关闭的逆序。 最终复杂度为：时间 O(n)，空间 O(n)。

Q035 Min Stack

难度 M	核心 Stack / 设计	类别 Stack / Queue / Deque

题目

设计栈，支持 push、pop、top、getMin，所有操作 O(1)。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次 getMin 扫描整个栈是 O(n)。

关键洞察：为什么这个数据结构合适

最直接使用两个栈：数据栈保存所有值，最小栈在每次 push 时保存“截至当前深度的最小值”。这样 pop 同步弹出即可。

不变量（Invariant） mins[i] 等于 data[0..i] 的最小值，因此当前最小值永远是 mins[-1]。

参考实现（Python ）

```
class MinStack:
    def __init__(self): self.data, self.mins = [], []
    def push(self, x):
        self.data.append(x)
        self.mins.append(x if not self.mins else min(x, self.mins[-1]))
    def pop(self): self.mins.pop(); return self.data.pop()
    def top(self): return self.data[-1]
    def getMin(self): return self.mins[-1]
```

复杂度

每个操作 O(1)，空间 O(n)。

高频失分点

- min 栈只在新最小值时入栈，却没处理重复最小值
- 空栈操作语义未说明

面试官追问

- 只用一个栈如何编码旧最小值?
- 进一步支持 getMax 怎么办?

60 秒口述模板

我会先给出一个可工作的基线方案：每次 getMin 扫描整个栈是 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：最直接使用两个栈：数据栈保存所有值，最小栈在每次 push 时保存“截至当前深度的最小值”。这样 pop 同步弹出即可。 正确性依赖的不变量是：mins[i] 等于 data[0..i] 的最小值，因此当前最小值永远是 mins[-1]。 最终复杂度为：每个操作 O(1)，空间 O(n)。

Q036 两个栈实现队列

难度 E/M	核心 Stack / Queue	类别 Stack / Queue / Deque

题目

只使用栈操作实现 FIFO 队列。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次 dequeue 都把所有元素来回倒腾会 O(n) 每次。

关键洞察：为什么这个数据结构合适

一个 in 栈负责接收新元素；一个 out 栈负责输出。当 out 为空时才把 in 全部倒入 out，此时顺序被反转成 FIFO。

不变量（Invariant） out 非空期间其栈顶永远是最老元素；每个元素最多经历 in push、in pop、out push、out pop 四个常数操作。

参考实现（Python ）

```
class MyQueue:
    def __init__(self): self.ins, self.out = [], []
    def push(self, x): self.ins.append(x)
    def _move(self):
        if not self.out:
            while self.ins: self.out.append(self.ins.pop())
    def pop(self): self._move(); return self.out.pop()
    def peek(self): self._move(); return self.out[-1]
    def empty(self): return not self.ins and not self.out
```

复杂度

push O(1)；pop/peek 摊销 O(1)，最坏单次 O(n)；空间 O(n)。

高频失分点

- 每次 peek 都搬运 in，即使 out 非空
- 不会解释“摊销 O(1)”

面试官追问

- 用势能法如何证明摊销复杂度?

60 秒口述模板

我会先给出一个可工作的基线方案：每次 dequeue 都把所有元素来回倒腾会 O(n) 每次。 真正的瓶颈在重复工作，因此优化的核心是：一个 in 栈负责接收新元素；一个 out 栈负责输出。当 out 为空时才把 in 全部倒入 out，此时顺序被反转成 FIFO。 正确性依赖的不变量是：out 非空期间其栈顶永远是最老元素；每个元素最多经历 in push、in pop、out push、out pop 四个常数操作。 最终复杂度为：push O(1)；pop/peek 摊销 O(1)，最坏单次 O(n)；空间 O(n)。

Q037 一个/两个队列实现栈

难度 E	核心 Queue / Stack	类别 Stack / Queue / Deque

题目

只用队列操作实现 LIFO 栈。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

两个队列可在 pop 时把 n-1 个元素搬到辅助队列，O(n) pop。

关键洞察：为什么这个数据结构合适

若使用一个 deque/queue，可选择在 push 后旋转此前所有元素，让最新元素始终位于队首；于是 pop O(1)，push O(n)。

> **不变量（Invariant）** 每次 push 结束后，队列顺序等于栈从顶到底的顺序。

参考实现（Python ）

```
from collections import deque
class MyStack:
    def __init__(self): self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q)-1): self.q.append(self.q.popleft())
    def pop(self): return self.q.popleft()
    def top(self): return self.q[0]
    def empty(self): return not self.q
```

复杂度

一种实现 push O(n)、pop/top O(1)，空间 O(n)。

高频失分点

- 没有说明把成本放在 push 还是 pop 是设计选择
- 把 deque 的双端能力当成题目允许的 queue API 使用

面试官追问

- 如果严格只允许 enqueue/dequeue/front，代码如何调整?

60 秒口述模板

我会先给出一个可工作的基线方案：两个队列可在 pop 时把 n-1 个元素搬到辅助队列，O(n) pop。 真正的瓶颈在重复工作，因此优化的核心是：若使用一个 deque/queue，可选择在 push 后旋转此前所有元素，让最新元素始终位于队首；于是 pop O(1)，push O(n)。 正确性依赖的不变量是：每次 push 结束后，队列顺序等于栈从顶到底的顺序。 最终复杂度为：一种实现 push O(n)、pop/top O(1)，空间 O(n)。

Q038 每日温度：下一个更高温度

难度 M	核心 单调栈 / 数组	类别 Stack / Queue / Deque

题目

对每一天，返回还要等待多少天才出现更高温度；不存在则 0。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每个位置向右扫描，最坏 O(n^2)。

关键洞察：为什么这个数据结构合适

维护一个“尚未找到更高温度”的下标栈，对应温度单调递减。新温度更高时，它会一次性解决栈顶的一批旧问题。

不变量（Invariant） 栈内所有下标都还没有遇到右侧更高温度；当 i 弹出 j 时，i 是 j 右侧第一个更高温度，因为中间所有元素都未能使 j 出栈。

参考实现（Python ）

```
def daily_temperatures(t):
    ans = [0]*len(t); st = []
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop(); ans[j] = i-j
        st.append(i)
    return ans
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 栈存温度而不是下标，无法计算距离
- 相等温度是否算“更高”判断错误

面试官追问

- 如何求下一个更小元素?
- 循环数组的 Next Greater 如何处理?

60 秒口述模板

我会先给出一个可工作的基线方案：每个位置向右扫描，最坏 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：维护一个“尚未找到更高温度”的下标栈，对应温度单调递减。新温度更高时，它会一次性解决栈顶的一批旧问题。 正确性依赖的不变量是：栈内所有下标都还没有遇到右侧更高温度；当 i 弹出 j 时，i 是 j 右侧第一个更高温度，因为中间所有元素都未能使 j 出栈。 最终复杂度为：时间 O(n)，空间 O(n)。

Q039 Next Greater Element 通用模板

难度 M	核心 单调栈	类别 Stack / Queue / Deque

题目

给定数组，为每个元素找到其右侧第一个严格更大的元素（或下标）。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

逐个向右扫描 O(n^2)。

关键洞察：为什么这个数据结构合适

可从右向左维护严格递减栈：弹出所有 <= 当前值的元素，剩余栈顶就是最近的更大元素；再把当前元素压栈。

不变量（Invariant） 处理 i 时，栈保存 i 右侧仍可能成为“最近更大值”的候选，并按值递减；任何 <= nums[i] 的候选对更左元素也会被 nums[i] 支配。

参考实现（Python ）

```
def next_greater(nums):
    ans = [-1]*len(nums); st = []
    for i in range(len(nums)-1, -1, -1):
        while st and st[-1] <= nums[i]: st.pop()
        if st: ans[i] = st[-1]
        st.append(nums[i])
    return ans
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 严格大于 vs 大于等于的 pop 条件
- 若要返回下标就必须存下标

面试官追问

- 为什么单调栈经常能把 O(n^2) 的“向左/右找第一个”降成 O(n)?

60 秒口述模板

我会先给出一个可工作的基线方案：逐个向右扫描 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：可从右向左维护严格递减栈：弹出所有 <= 当前值的元素，剩余栈顶就是最近的更大元素；再把当前元素压栈。 正确性依赖的不变量是：处理 i 时，栈保存 i 右侧仍可能成为“最近更大值”的候选，并按值递减；任何 <= nums[i] 的候选对更左元素也会被 nums[i] 支配。 最终复杂度为：时间 O(n)，空间 O(n)。

Q040 柱状图最大矩形

难度 H	核心 单调栈 / 数组	类别 Stack / Queue / Deque

题目

给定各柱高度，求由连续柱形成的最大矩形面积。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举每个区间并取最小高度太慢；对每个柱向左右扫描首个更矮也可能 O(n^2)。

关键洞察：为什么这个数据结构合适

维护单调递增高度栈。遇到更矮柱时，栈顶高度的“右边第一个更矮”已经确定；弹出后新的栈顶就是其左边第一个更矮，因此宽度可立即计算。末尾加入高度 0 的 sentinel 清空栈。

不变量（Invariant） 柱子 h 被弹出时，其能延展的最大连续区间边界已经同时确定，因此此时计算以 h 为最矮柱的最大矩形不会遗漏。

参考实现（Python ）

```
def largest_rectangle(heights):
    st = []  # (start, height)
    best = 0
    for i, h in enumerate(heights + [0]):
        start = i
        while st and st[-1][1] > h:
            j, old = st.pop()
            best = max(best, old*(i-j)); start = j
        st.append((start, h))
    return best
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 宽度 off-by-one
- 没有 sentinel 导致最后递增部分未结算

- 相等高度的处理不统一

面试官追问

- 最大矩形在二进制矩阵中如何复用本题?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举每个区间并取最小高度太慢；对每个柱向左右扫描首个更矮也可能 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：维护单调递增高度栈。遇到更矮柱时，栈顶高度的“右边第一个更矮”已经确定；弹出后新的栈顶就是其左边第一个更矮，因此宽度可立即计算。末尾加入高度 0 的 sentinel 清空栈。 正确性依赖的不变量是：柱子 h 被弹出时，其能延展的最大连续区间边界已经同时确定，因此此时计算以 h 为最矮柱的最大矩形不会遗漏。 最终复杂度为：时间 O(n)，空间 O(n)。

Q041 Decode String：解码嵌套重复串

难度 M	核心 Stack / 递归下降	类别 Stack / Queue / Deque

题目

解码形如 k[encoded] 的字符串，支持多位数字和嵌套，如 3[a2[c]]。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

反复寻找最内层括号替换可行但频繁构造字符串，复杂度和代码都较差。

关键洞察：为什么这个数据结构合适

遇到 [时保存“外层字符串+重复次数”，进入新层；遇到] 时完成当前层并回到上一层。也可用递归下降 parser。

不变量（Invariant） 栈中每一项代表一个尚未闭合的外层上下文；当前 cur 始终是最内层正在构造的字符串。

参考实现（Python ）

```
def decode_string(s):
    st=[]; cur=''; num=0
    for c in s:
        if c.isdigit(): num = num*10 + int(c)
        elif c == '[':
            st.append((cur, num)); cur=''; num=0
        elif c == ']':
            prev, k = st.pop(); cur = prev + cur*k
        else: cur += c
    return cur
```

复杂度

忽略输出长度时输入扫描 O(n)；实际至少要 O(输出长度) 构造结果。

高频失分点

- 数字只读取一位
- 没有把 num 在进入新层后清零

- 复杂度分析忽略巨大输出

面试官追问

- 若输出可能指数级巨大但只询问第 K 个字符，如何避免完整展开?

60 秒口述模板

我会先给出一个可工作的基线方案：反复寻找最内层括号替换可行但频繁构造字符串，复杂度和代码都较差。 真正的瓶颈在重复工作，因此优化的核心是：遇到 [时保存“外层字符串+重复次数”，进入新层；遇到] 时完成当前层并回到上一层。也可用递归下降 parser。 正确性依赖的不变量是：栈中每一项代表一个尚未闭合的外层上下文；当前 cur 始终是最内层正在构造的字符串。 最终复杂度为：忽略输出长度时输入扫描 O(n)；实际至少要 O(输出长度) 构造结果。

Q042 碰撞模拟：Asteroid Collision

难度 M	核心 Stack / 模拟	类别 Stack / Queue / Deque

题目

一维数组表示物体方向与大小，只有相向而行的一对可能碰撞；按规则保留幸存者。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

按时间步反复扫描所有相邻冲突对，可能多轮 O(n^2)。

关键洞察：为什么这个数据结构合适

从左到右扫描，栈保存已稳定的左侧物体。新物体只有在“栈顶向右、新物体向左”时可能继续碰撞，其余情况直接入栈。

不变量（Invariant） 栈内序列内部不存在尚未处理的碰撞；新物体只可能与栈顶形成最邻近冲突，因此逐个弹栈即可。

参考实现（Python ）

```
def asteroid_collision(a):
    st=[]
    for x in a:
        alive=True
        while alive and x<0 and st and st[-1]>0:
            if st[-1] < -x: st.pop()
            elif st[-1] == -x: st.pop(); alive=False
            else: alive=False
        if alive: st.append(x)
    return st
```

复杂度

时间 O(n)，空间 O(n)。

高频失分点

- 所有异号都当成碰撞，忽略方向顺序
- 相等大小时忘记双方消失

面试官追问

- 如果速度不同，为什么栈模型可能不再足够?

60 秒口述模板

我会先给出一个可工作的基线方案：按时间步反复扫描所有相邻冲突对，可能多轮 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：从左到右扫描，栈保存已稳定的左侧物体。新物体只有在“栈顶向右、新物体向左”时可能继续碰撞，其余情况直接入栈。 正确性依赖的不变量是：栈内序列内部不存在尚未处理的碰撞；新物体只可能与栈顶形成最邻近冲突，因此逐个弹栈即可。 最终复杂度为：时间 O(n)，空间 O(n)。

Q043 Basic Calculator：表达式求值

难度 H	核心 Stack / Parser	类别 Stack / Queue / Deque

题目

解析包含整数、+/-、括号（可扩展到 */）的表达式并求值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

直接 eval 不允许且绕过考点；先转后缀表达式再算可行但代码较长。

关键洞察：为什么这个数据结构合适

对只有 +/- 与括号的版本，可维护当前 result、sign。遇到 (时把外层 result/sign 入栈并从 0 开始；遇到) 时把当前层结果乘外层 sign 后加回。

不变量（Invariant） 扫描位置之前的表达式都已经折算进 result；栈保存每个未闭合括号外的累积结果和作用在括号整体上的符号。

参考实现（Python ）

```
def calculate(s):
    res=0; num=0; sign=1; st=[]
    for c in s+'+':
        if c.isdigit(): num=num*10+int(c)
        elif c in '+-':
            res += sign*num; num=0; sign = 1 if c=='+' else -1
        elif c=='(':
            st.append((res,sign)); res=0; sign=1
        elif c==')':
            res += sign*num; num=0
            prev, outer = st.pop(); res = prev + outer*res
    return res
```

复杂度

时间 O(n)，空间 O(嵌套深度)。

高频失分点

- 一元负号与二元减法语义

- 空格处理
- 扩展乘除时优先级不能沿用当前简单状态机

面试官追问

- 加入 * / 后用双栈还是递归下降?
- 如何构建 AST 而不只是求值?

60 秒口述模板

我会先给出一个可工作的基线方案：直接 eval 不允许且绕过考点；先转后缀表达式再算可行但代码较长。 真正的瓶颈在重复工作，因此优化的核心是：对只有 +/- 与括号的版本，可维护当前 result、sign。遇到 (时把外层 result/sign 入栈并从 0 开始；遇到) 时把当前层结果乘外层 sign 后加回。 正确性依赖的不变量是：扫描位置之前的表达式都已经折算进 result；栈保存每个未闭合括号外的累积结果和作用在括号整体上的符号。 最终复杂度为：时间 O(n)，空间 O(嵌套深度)。

第五章

Binary Tree / BST

Q044 - Q058

从递归定义出发，区分“向父返回的状态”和“当前节点形成的全局答案”。

Q044 二叉树最大深度

难度 E	核心 Tree / DFS	类别 Binary Tree / BST

题目

求二叉树根到最深叶子的节点数/层数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

树本身必须访问所有节点，O(n) 已是下界。

关键洞察：为什么这个数据结构合适

递归定义最自然：depth(node)=1+max(depth(left),depth(right))；也可 BFS 按层计数。

不变量（Invariant） 函数返回值就是“当前子树的最大深度”，空树返回 0。

参考实现（Python ）

```
def max_depth(root):
    if not root: return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

复杂度

时间 O(n)，递归栈 O(h)。

高频失分点

- 把边数深度与节点数深度混淆
- 极端偏斜树递归深度 O(n)

面试官追问

- 如何迭代实现?
- 最小深度为何不能直接把 max 改 min?

60 秒口述模板

我会先给出一个可工作的基线方案：树本身必须访问所有节点，O(n) 已是下界。 真正的瓶颈在重复工作，因此优化的核心是：递归定义最自然：depth(node)=1+max(depth(left),depth(right))；也可 BFS 按层计数。 正确性依赖的不变量是：函数返回值就是“当前子树的最大深度”，空树返回 0。 最终复杂度为：时间 O(n)，递归栈 O(h)。

Q045 翻转二叉树

难度 E	核心 Tree / 递归	类别 Binary Tree / BST

题目

把二叉树每个节点的左右子树交换。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

无需额外复杂结构。

关键洞察：为什么这个数据结构合适

对每个节点交换 left/right，然后递归处理两棵子树；先交换还是递归后交换均可，只要访问全部节点。

> **不变量（Invariant）** 处理结束后，当前节点及其左右子树都已经成为原树的镜像。

参考实现（Python ）

```
def invert_tree(root):
    if root:
        root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 交换后继续按旧引用递归造成混乱
- 没有说明是否原地修改

面试官追问

- 如何用 BFS 原地翻转?

60 秒口述模板

我会先给出一个可工作的基线方案：无需额外复杂结构。 真正的瓶颈在重复工作，因此优化的核心是：对每个节点交换 left/right，然后递归处理两棵子树；先交换还是递归后交换均可，只要访问全部节点。 正确性依赖的不变量是：处理结束后，当前节点及其左右子树都已经成为原树的镜像。 最终复杂度为：时间 O(n)，栈 O(h)。

Q046 二叉树层序遍历

难度 M	核心 Tree / BFS	类别 Binary Tree / BST

题目

按从上到下、每层从左到右输出节点值二维列表。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

DFS 也能按 depth 收集，但 BFS 更直接。

关键洞察：为什么这个数据结构合适

队列保存当前 frontier。每轮先记录当前 queue size，只弹出这固定数量的节点作为一层；它们的孩子进入下一层。

不变量（Invariant） 每轮开始时队列恰好包含当前层全部节点。

参考实现（Python ）

```
from collections import deque

def level_order(root):
    if not root: return []
    q=deque([root]); ans=[]
    while q:
        level=[]
        for _ in range(len(q)):
            node=q.popleft(); level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        ans.append(level)
    return ans
```

复杂度

时间 O(n)，队列空间 O(w)，w 为最大层宽。

高频失分点

- for 循环中动态使用 len(q) 而不是提前固定
- 把下一层节点混进当前层

面试官追问

- Zigzag 层序如何改?
- 如何只用 O(1) 额外空间遍历 perfect tree 的 next 指针?

60 秒口述模板

我会先给出一个可工作的基线方案：DFS 也能按 depth 收集，但 BFS 更直接。 真正的瓶颈在重复工作，因此优化的核心是：队列保存当前 frontier。每轮先记录当前 queue size，只弹出这固定数量的节点作为一层；它们的孩子进入下一层。 正确性依赖的不变量是：每轮开始时队列恰好包含当前层全部节点。 最终复杂度为：时间 O(n)，队列空间 O(w)，w 为最大层宽。

Q047 二叉树右视图

难度 M	核心 Tree / BFS	类别 Binary Tree / BST

题目

从树右侧观察，每一层能看到最右节点，返回这些节点值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

层序遍历后取每层最后一个即可。

关键洞察：为什么这个数据结构合适

BFS 是最直观；DFS 也可按 root-right-left 顺序访问，并在第一次到达某 depth 时记录节点。

> **不变量（Invariant）** DFS 右优先保证同深度第一个被访问的节点就是最右可见节点。

参考实现（Python ）

```
def right_side_view(root):
    ans=[]
    def dfs(node, d):
        if not node: return
        if d == len(ans): ans.append(node.val)
        dfs(node.right, d+1); dfs(node.left, d+1)
    dfs(root,0); return ans
```

复杂度

时间 O(n)，空间 O(h) 或 BFS O(w)。

高频失分点

- DFS 顺序写成左优先
- 用 depth 索引但没有判断首次访问

面试官追问

- 左视图？顶部视图需要额外什么坐标信息?

60 秒口述模板

我会先给出一个可工作的基线方案：层序遍历后取每层最后一个即可。 真正的瓶颈在重复工作，因此优化的核心是：BFS 是最直观；DFS 也可按 root-right-left 顺序访问，并在第一次到达某 depth 时记录节点。 正确性依赖的不变量是：DFS 右优先保证同深度第一个被访问的节点就是最右可见节点。 最终复杂度为：时间 O(n)，空间 O(h) 或 BFS O(w)。

Q048 二叉树直径

难度 M	核心 Tree / Postorder	类别 Binary Tree / BST

题目

二叉树直径定义为任意两个节点间最长路径的边数，路径可经过或不经过根。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

对每个节点重新计算左右高度是 O(n^2)。

关键洞察：为什么这个数据结构合适

一次后序 DFS：函数向父节点返回当前子树高度，同时用 leftHeight+rightHeight 更新全局直径。注意“返回值”和“全局候选”不是同一量。

不变量（Invariant） 当处理 node 时左右子树高度已准确；经过 node 的最长路径只能由左侧最长向下链 + 右侧最长向下链组成。

参考实现（Python ）

```
def diameter(root):
    best=0
    def height(node):
        nonlocal best
        if not node: return 0
        l,r=height(node.left),height(node.right)
        best=max(best,l+r)
        return 1+max(l,r)
    height(root); return best
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 把节点数和边数差 1
- 错误返回 l+r+1 给父节点，导致形成分叉路径

面试官追问

- 如何返回直径对应的实际路径?
- N 叉树直径如何改?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个节点重新计算左右高度是 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：一次后序 DFS：函数向父节点返回当前子树高度，同时用 leftHeight+rightHeight 更新全局直径。注意“返回值”和“全局候选”不是同一量。 正确性依赖的不变量是：当处理 node 时左右子树高度已准确；经过 node 的最长路径只能由左侧最长向下链 + 右侧最长向下链组成。 最终复杂度为：时间 O(n)，栈 O(h)。

Q049 判断平衡二叉树

难度 E/M	核心 Tree / Postorder	类别 Binary Tree / BST

题目

若每个节点左右子树高度差不超过 1，则称高度平衡；判断给定树是否平衡。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

对每个节点调用独立 height 会重复遍历子树，最坏 O(n^2)。

关键洞察：为什么这个数据结构合适

后序函数同时承担“计算高度”和“报告失败”。用 -1 作为不平衡 sentinel；一旦子树返回 -1 立即向上传播。

> **不变量（Invariant）** 若函数返回非负值，它就是准确高度且该子树已验证平衡；若返回 -1，则整棵上层子树都无需继续高度计算。

参考实现（Python ）

```
def is_balanced(root):
    def h(node):
        if not node: return 0
        l=h(node.left)
        if l<0: return -1
        r=h(node.right)
        if r<0 or abs(l-r)>1: return -1
        return 1+max(l,r)
    return h(root)>=0
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 只检查 root 的高度差
- 先写独立 max_depth 导致重复计算

面试官追问

- AVL 树如何在插入/删除后维持这个约束?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个节点调用独立 height 会重复遍历子树，最坏 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：后序函数同时承担“计算高度”和“报告失败”。用 -1 作为不平衡 sentinel；一旦子树返回 -1 立即向上传播。 正确性依赖的不变量是：若函数返回非负值，它就是准确高度且该子树已验证平衡；若返回 -1，则整棵上层子树都无需继续高度计算。 最终复杂度为：时间 O(n)，栈 O(h)。

Q050 验证二叉搜索树

难度 M	核心 BST / 递归边界	类别 Binary Tree / BST

题目

判断二叉树是否满足：任一节点左子树所有值都小于它，右子树所有值都大于它。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

只比较 node.left.val < node.val < node.right.val 是错误的，因为违规节点可能在更深层。

关键洞察：为什么这个数据结构合适

递归时把祖先约束传下去：节点值必须落在 (low,high)；去左子树时 high=node.val，去右时 low=node.val。

不变量（Invariant） 区间 (low,high) 精确编码了从根到当前节点路径上所有祖先共同施加的限制。

参考实现（Python ）

```
def is_valid_bst(root):
    def dfs(node, low, high):
        if not node: return True
        if not (low < node.val < high): return False
        return dfs(node.left,low,node.val) and dfs(node.right,node.val,high)
    return dfs(root,float('-inf'),float('inf'))
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 只比较父子
- 重复值是否允许没有澄清
- 使用固定 int 最小值可能溢出/不适用于任意数值类型

面试官追问

- 用 inorder 如何验证?
- 如果 BST 允许重复值且规定重复都放右边，边界如何改变?

60 秒口述模板

我会先给出一个可工作的基线方案：只比较 node.left.val < node.val < node.right.val 是错误的，因为违规节点可能在更深层。 真正的瓶颈在重复工作，因此优化的核心是：递归时把祖先约束传下去：节点值必须落在 (low,high)；去左子树时 high=node.val，去右时 low=node.val。 正确性依赖的不变量是：区间 (low,high) 精确编码了从根到当前节点路径上所有祖先共同施加的限制。 最终复杂度为：时间 O(n)，栈 O(h)。

Q051 BST 第 K 小元素

难度 M	核心 BST / Inorder	类别 Binary Tree / BST

题目

在 BST 中返回第 k 小值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

收集全部值排序是 O(n log n)，忽略了 BST 有序性质。

关键洞察：为什么这个数据结构合适

BST 的 inorder 正好按升序访问。迭代 inorder 在访问第 k 个节点时即可提前结束。

> **不变量（Invariant）** 栈保存尚未访问的祖先；每次 pop 的节点是当前所有未访问节点中的最小值。

参考实现（Python ）

```
def kth_smallest(root,k):
    st=[]; cur=root
    while True:
        while cur: st.append(cur); cur=cur.left
        cur=st.pop(); k-=1
        if k==0: return cur.val
        cur=cur.right
```

复杂度

最坏 O(h+k)，空间 O(h)。

高频失分点

- 把 preorder 当成有序
- k 越界未定义

面试官追问

- 如果频繁查询 kth，如何给每个节点维护 subtree_size 使单次查询 O(h)?

60 秒口述模板

我会先给出一个可工作的基线方案：收集全部值排序是 O(n log n)，忽略了 BST 有序性质。 真正的瓶颈在重复工作，因此优化的核心是：BST 的 inorder 正好按升序访问。迭代 inorder 在访问第 k 个节点时即可提前结束。 正确性依赖的不变量是：栈保存尚未访问的祖先；每次 pop 的节点是当前所有未访问节点中的最小值。 最终复杂度为：最坏 O(h+k)，空间 O(h)。

Q052 BST 最近公共祖先

难度 M	核心 BST / LCA	类别 Binary Tree / BST

题目

给定 BST 中两个节点 p、q，找最低公共祖先。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

通用 LCA 递归可 O(n)，但没有利用 BST 排序。

关键洞察：为什么这个数据结构合适

若 p、q 都小于 root，则 LCA 一定在左子树；都大于则在右子树；否则当前 root 正好是分叉点，也就是 LCA。

不变量（Invariant） 在到达第一个无法把 p、q 同时放入同一侧的节点之前，它们仍共享更深祖先；该分叉点即最低公共祖先。

参考实现（Python ）

```
def lca_bst(root,p,q):
    lo,hi=sorted((p.val,q.val))
    while root:
        if root.val < lo: root=root.right
        elif root.val > hi: root=root.left
        else: return root
```

复杂度

时间 O(h)，空间迭代 O(1)。

高频失分点

- 节点值可能相等/重复的 BST 定义未澄清
- 明明是 BST 却写通用 O(n) 解法

面试官追问

- 若只给数值不保证存在于树中，需要先验证吗?

60 秒口述模板

我会先给出一个可工作的基线方案：通用 LCA 递归可 O(n)，但没有利用 BST 排序。 真正的瓶颈在重复工作，因此优化的核心是：若 p、q 都小于 root，则 LCA 一定在左子树；都大于则在右子树；否则当前 root 正好是分叉点，也就是 LCA。 正确性依赖的不变量是：在到达第一个无法把 p、q 同时放入同一侧的节点之前，它们仍共享更深祖先；该分叉点即最低公共祖先。 最终复杂度为：时间 O(h)，空间迭代 O(1)。

Q053 普通二叉树最近公共祖先

难度 M	核心 Tree / 递归	类别 Binary Tree / BST

题目

在普通二叉树中找两个节点 p、q 的最低公共祖先。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

为两个节点分别记录 root-to-node 路径再比较，空间 O(h)。

关键洞察：为什么这个数据结构合适

后序递归：若当前节点就是 p/q 返回自己；分别在左右子树寻找。若左右都非空，说明 p/q 分居两侧，当前就是 LCA；否则把唯一非空结果向上传。

不变量（Invariant） 函数返回“当前子树中是否找到了 p 或 q，若找到了则返回能代表该目标/其 LCA 的节点”。第一次左右同时非空的位置就是最低分叉点。

参考实现（Python ）

```
def lowest_common_ancestor(root,p,q):
    if not root or root is p or root is q: return root
    l=lowest_common_ancestor(root.left,p,q)
    r=lowest_common_ancestor(root.right,p,q)
    if l and r: return root
    return l or r
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 题目若不保证 p/q 都存在，此简洁版本可能返回其中一个
- 把节点值相等当成节点相同

面试官追问

- 若需要验证两节点都存在，返回状态怎么扩展?
- 多个节点的 LCA 如何做?

60 秒口述模板

我会先给出一个可工作的基线方案：为两个节点分别记录 root-to-node 路径再比较，空间 O(h)。 真正的瓶颈在重复工作，因此优化的核心是：后序递归：若当前节点就是 p/q 返回自己；分别在左右子树寻找。若左右都非空，说明 p/q 分居两侧，当前就是 LCA；否则把唯一非空结果向上传。 正确性依赖的不变量是：函数返回“当前子树中是否找到了 p 或 q，若找到了则返回能代表该目标/其 LCA 的节点”。第一次左右同时非空的位置就是最低分叉点。 最终复杂度为：时间 O(n)，栈 O(h)。

Q054 Preorder + Inorder 重建二叉树

难度 M	核心 Tree / 递归	类别 Binary Tree / BST

题目

给定无重复节点值的前序和中序遍历，重建唯一二叉树。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

每层在线性中序数组中查 root 位置会造成 O(n^2)。

关键洞察：为什么这个数据结构合适

前序的第一个元素是根；根在中序中的位置将左右子树精确分开。预建 value→inorderIndex 哈希，并用 preorder 游标按根顺序消费。

> **不变量（Invariant）** 递归 build(l,r) 负责恰好构造 inorder[l:r+1] 对应的子树；preorder 游标始终指向该子树根。

参考实现（Python ）

```
def build_tree(preorder,inorder):
    idx={v:i for i,v in enumerate(inorder)}; p=0
    def build(l,r):
        nonlocal p
        if l>r: return None
        v=preorder[p]; p+=1; root=TreeNode(v); m=idx[v]
        root.left=build(l,m-1); root.right=build(m+1,r)
        return root
    return build(0,len(inorder)-1)
```

复杂度

时间 O(n)，空间 O(n) 哈希 + O(h) 栈。

高频失分点

- 存在重复值时 value→index 不再唯一
- 递归区间 off-by-one

- 没先检查两个遍历长度/元素集合是否一致

面试官追问

- Postorder + Inorder 如何重建?
- Preorder + Postorder 为什么一般不能唯一确定?

60 秒口述模板

我会先给出一个可工作的基线方案：每层在线性中序数组中查 root 位置会造成 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：前序的第一个元素是根；根在中序中的位置将左右子树精确分开。预建 value→inorderIndex 哈希，并用 preorder 游标按根顺序消费。 正确性依赖的不变量是：递归 build(l,r) 负责恰好构造 inorder[l:r+1] 对应的子树；preorder 游标始终指向该子树根。 最终复杂度为：时间 O(n)，空间 O(n) 哈希 + O(h) 栈。

Q055 二叉树序列化与反序列化

难度 H	核心 Tree / 编码	类别 Binary Tree / BST

题目

设计 serialize/deserialize，把任意二叉树转换成字符串并无损恢复。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

只保存 preorder 节点值无法区分结构不同但遍历值相同的树。

关键洞察：为什么这个数据结构合适

在 preorder 中显式写入空节点标记 #，结构就变得唯一：每个真实节点后都递归编码左右子树。反序列化按同一文法消费 token。

> **不变量（Invariant）** 编码是一个前缀可解析文法：真实节点产生“value,left,right”，空节点产生“#”；因此解码位置唯一。

参考实现（Python ）

```
def serialize(root):
    out=[]
    def dfs(node):
        if not node: out.append('#'); return
        out.append(str(node.val)); dfs(node.left); dfs(node.right)
    dfs(root); return ','.join(out)

def deserialize(data):
    it=iter(data.split(','))
    def dfs():
        x=next(it)
        if x=='#': return None
        node=TreeNode(int(x)); node.left=dfs(); node.right=dfs(); return node
    return dfs()
```

复杂度

时间 O(n)，序列化长度 O(n)，递归栈 O(h)。

高频失分点

- 不编码 null 导致结构歧义
- 值本身若能包含分隔符需要转义/长度前缀
- 深树递归风险

面试官追问

- 如何设计二进制紧凑格式?
- BST 是否可以省掉 null 标记? 为什么?

60 秒口述模板

我会先给出一个可工作的基线方案：只保存 preorder 节点值无法区分结构不同但遍历值相同的树。 真正的瓶颈在重复工作，因此优化的核心是：在 preorder 中显式写入空节点标记 #，结构就变得唯一：每个真实节点后都递归编码左右子树。反序列化按同一文法消费 token。 正确性依赖的不变量是：编码是一个前缀可解析文法：真实节点产生“value,left,right”，空节点产生“#”；因此解码位置唯一。 最终复杂度为：时间 O(n)，序列化长度 O(n)，递归栈 O(h)。

Q056 二叉树最大路径和

难度 H	核心 Tree DP / Postorder	类别 Binary Tree / BST

题目

求二叉树任意非空路径的最大节点值之和；路径中相邻节点必须由边连接，不能重复节点。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

枚举所有路径不可行。

关键洞察：为什么这个数据结构合适

后序 DFS。向父节点返回的 gain 只能选择左/右一条分支，因为继续向上时路径不能在当前节点分叉；但全局答案可以用 node+leftGain+rightGain，在当前节点形成完整拱形路径。负 gain 应截断为 0。

不变量（Invariant） 返回值始终是一条从当前节点向下的合法单链；全局 best 则允许在当前节点把两条向下链连接。

参考实现（Python ）

```
def max_path_sum(root):
    best=float('-inf')
    def gain(node):
        nonlocal best
        if not node: return 0
        l=max(0,gain(node.left)); r=max(0,gain(node.right))
        best=max(best,node.val+l+r)
        return node.val+max(l,r)
    gain(root); return best
```

复杂度

时间 O(n)，栈 O(h)。

高频失分点

- 全负树把 best 初始化为 0 会错
- 向父返回左右两支，构成非路径结构
- 混淆 root-to-leaf path 与任意 path

面试官追问

- 如何返回实际节点路径?
- 边有权而节点无权时怎么改?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举所有路径不可行。 真正的瓶颈在重复工作，因此优化的核心是：后序 DFS。向父节点返回的 gain 只能选择左/右一条分支，因为继续向上时路径不能在当前节点分叉；但全局答案可以用 node+leftGain+rightGain，在当前节点形成完整拱形路径。负 gain 应截断为 0。 正确性依赖的不变量是：返回值始终是一条从当前节点向下的合法单链；全局 best 则允许在当前节点把两条向下链连接。 最终复杂度为：时间 O(n)，栈 O(h)。

Q057 判断子树

难度 M	核心 Tree / 匹配	类别 Binary Tree / BST

题目

判断 subRoot 是否与 root 的某个子树结构和值完全相同。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

对 root 每个节点调用 sameTree，最坏 O(nm)，但面试中通常先给这个清晰解。

关键洞察：为什么这个数据结构合适

基础解是 DFS + sameTree；若规模很大可进一步讨论结构哈希/Merkle hash 或带空标记的序列化后做线性字符串匹配。

> **不变量（Invariant）** 只在 root 某节点值可能匹配时才做结构比较；sameTree 的 null 对齐保证结构一致。

参考实现（Python ）

```
def is_subtree(root, sub):
    def same(a,b):
        if not a or not b: return a is b
        return a.val==b.val and same(a.left,b.left) and same(a.right,b.right)
    if not sub: return True
    if not root: return False
    return same(root,sub) or is_subtree(root.left,sub) or is_subtree(root.right,sub)
```

复杂度

基础最坏 O(nm)，平均通常更好；哈希方案可把候选筛选接近 O(n+m)。

高频失分点

- 序列化时不加 null 标记会出现假匹配
- 把 subtree 与“包含相同值集合”混淆

面试官追问

- 如何用树哈希降低复杂度?
- 哈希碰撞如何处理?

60 秒口述模板

我会先给出一个可工作的基线方案：对 root 每个节点调用 sameTree，最坏 O(nm)，但面试中通常先给这个清晰解。真正的瓶颈在重复工作，因此优化的核心是：基础解是 DFS + sameTree；若规模很大可进一步讨论结构哈希/Merkle hash 或带空标记的序列化后做线性字符串匹配。 正确性依赖的不变量是：只在 root 某节点值可能匹配时才做结构比较；sameTree 的 null 对齐保证结构一致。 最终复杂度为：基础最坏 O(nm)，平均通常更好；哈希方案可把候选筛选接近 O(n+m)。

Q058 二叉树原地展开为链表

难度 M	核心 Tree / 指针重连	类别 Binary Tree / BST

题目

按 preorder 顺序把二叉树原地展开成只使用 right 指针的链表，所有 left 置空。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

先做 preorder 保存数组，再重连，空间 O(n)。

关键洞察：为什么这个数据结构合适

可用类似 Morris 的 O(1) 方法：若当前节点有左子树，找到左子树最右节点 predecessor，把当前 right 接到 predecessor.right，再把左子树搬到 right，left 置空。继续向右。

不变量（Invariant） 每次改造后，当前节点 right 指向其 preorder 下一节点，同时原右子树被接在左子树 preorder 的末尾，因此整体 preorder 次序保持。

参考实现（Python ）

```
def flatten(root):
    cur=root
    while cur:
        if cur.left:
            pred=cur.left
            while pred.right: pred=pred.right
            pred.right=cur.right
            cur.right=cur.left
            cur.left=None
        cur=cur.right
```

复杂度

时间 O(n)（仔细实现可线性），空间 O(1)。

高频失分点

- 把原右子树丢失

- 忘记 left=None
- 无法解释为何 predecessor 应是左子树最右节点

面试官追问

- 递归 reverse-preorder 版本如何写?
- 与 Morris traversal 有何关系?

60 秒口述模板

我会先给出一个可工作的基线方案：先做 preorder 保存数组，再重连，空间 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：可用类似 Morris 的 O(1) 方法：若当前节点有左子树，找到左子树最右节点 predecessor，把当前 right 接到 predecessor.right，再把左子树搬到 right，left 置空。继续向右。 正确性依赖的不变量是：每次改造后，当前节点 right 指向其 preorder 下一节点，同时原右子树被接在左子树 preorder 的末尾，因此整体 preorder 次序保持。 最终复杂度为：时间 O(n)（仔细实现可线性），空间 O(1)。

第六章

Heap / Priority Queue

Q059 - Q066

Top-K、动态最值、K 路归并、在线中位数与调度。

Q059 数组第 K 大元素

难度 M	核心 Heap / Quickselect	类别 Heap / Priority Queue

题目

返回未排序数组第 k 大元素（按排序后位置计，通常允许重复）。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

完整排序 O(n log n)。

关键洞察：为什么这个数据结构合适

若 k 远小于 n，维护大小为 k 的最小堆，堆顶就是当前见过的第 k 大阈值。也应主动提到 Quickselect 平均 O(n)。

不变量（Invariant） 处理前 i 个元素后，堆中恰好保存其中最大的 min(k,i) 个；因此扫描结束后堆顶为第 k 大。

参考实现（Python ）

```
import heapq

def find_kth_largest(nums,k):
    h=[]
    for x in nums:
        if len(h)<k: heapq.heappush(h,x)
        elif x>h[0]: heapq.heapreplace(h,x)
    return h[0]
```

复杂度

Heap 时间 O(n log k)、空间 O(k)；Quickselect 平均 O(n)、最坏 O(n^2)。

高频失分点

- 第 k 大与第 k 个不同值混淆
- 用 max heap 存全部元素导致 O(n) 空间
- 没有比较 Quickselect

面试官追问

- 数据流版本为什么 heap 更合适?
- 若 k 接近 n，是否可以维护 n-k+1 的 max heap?

60 秒口述模板

我会先给出一个可工作的基线方案：完整排序 O(n log n)。 真正的瓶颈在重复工作，因此优化的核心是：若 k 远小于 n，维护大小为 k 的最小堆，堆顶就是当前见过的第 k 大阈值。也应主动提到 Quickselect 平均 O(n)。 正确性依赖的不变量是：处理前 i 个元素后，堆中恰好保存其中最大的 min(k,i) 个；因此扫描结束后堆顶为第 k 大。 最终复杂度为：Heap 时间 O(n log k)、空间 O(k)；Quickselect 平均 O(n)、最坏 O(n^2)。

Q060 距离原点最近的 K 个点

难度 M	核心 Heap / Quickselect	类别 Heap / Priority Queue

题目

给定二维点，返回距离原点最近的 k 个。无需排序输出。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

对全部点按距离排序 O(n log n)。

关键洞察：为什么这个数据结构合适

维护大小 k 的“最大堆”保存当前最近 k 个点，遇到更近点替换堆顶；Python 可用负距离模拟。若允许修改输入，Quickselect 更接近线性。距离比较无需开平方。

不变量（Invariant） 堆顶是当前 k 个候选中最远者，因此任何比它更远的新点都不可能进入最终 Top-K。

参考实现（Python ）

```
import heapq

def k_closest(points,k):
    h=[]
    for x,y in points:
        d=x*x+y*y
        item=(-d,x,y)
        if len(h)<k: heapq.heappush(h,item)
        elif item>h[0]: heapq.heapreplace(h,item)
    return [[x,y] for _,x,y in h]
```

复杂度

Heap O(n log k)、空间 O(k)。

高频失分点

- 不必要地 sqrt
- 最大堆/最小堆方向搞反
- 题目若要求排序输出还需额外处理

面试官追问

- 更高维点如何泛化?
- Quickselect 的 partition invariant 是什么?

60 秒口述模板

我会先给出一个可工作的基线方案：对全部点按距离排序 O(n log n)。 真正的瓶颈在重复工作，因此优化的核心是：维护大小 k 的“最大堆”保存当前最近 k 个点，遇到更近点替换堆顶；Python 可用负距离模拟。若允许修改输入，Quickselect 更接近线性。距离比较无需开平方。 正确性依赖的不变量是：堆顶是当前 k 个候选中最远者，因此任何比它更远的新点都不可能进入最终 Top-K。 最终复杂度为：Heap O(n log k)、空间 O(k)。

Q061 日志 Top-K 活跃用户，并保留阈值并列者

难度 M/H	核心 HashMap / Heap	类别 Heap / Priority Queue

来源标记 公开候选人面经中的 Top-K/Heap 类变体；候选人自述，非官方题库。

题目

日志中每条记录包含用户。统计活跃次数，返回至少 Top-K 的用户；若第 K 名有并列，所有同频用户都要返回。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

统计后对所有用户完整排序 O(u log u)。

关键洞察：为什么这个数据结构合适

第一阶段 HashMap 计数；第二阶段大小 k 的最小堆只求第 k 大频次阈值 T；第三阶段返回 count>=T 的全部用户。这样能正确保留并列。

不变量（Invariant） 堆扫描结束时 h[0] 是第 k 大频次；任何 count>=T 的用户排名都不会低于 k 的阈值位置。

参考实现（Python ）

```
from collections import Counter
import heapq

def top_k_with_ties(users,k):
    cnt=Counter(users)
    if k>=len(cnt): return list(cnt)
    h=[]
    for c in cnt.values():
        if len(h)<k: heapq.heappush(h,c)
        elif c>h[0]: heapq.heapreplace(h,c)
    t=h[0]
    return [u for u,c in cnt.items() if c>=t]
```

复杂度

计数 O(n)，求阈值 O(u log k)，再扫描 O(u)，空间 O(u+k)。

高频失分点

- 直接弹 k 个用户会漏掉并列
- 混淆“恰好 k 个”与“至少覆盖 Top-K 阈值”

面试官追问

- 如果日志是流式且内存有限怎么办?
- Count-Min Sketch 能提供什么近似保证?

60 秒口述模板

我会先给出一个可工作的基线方案：统计后对所有用户完整排序 O(u log u)。 真正的瓶颈在重复工作，因此优化的核心是：第一阶段 HashMap 计数；第二阶段大小 k 的最小堆只求第 k 大频次阈值 T；第三阶段返回 count>=T 的全部用户。这样能正确保留并列。 正确性依赖的不变量是：堆扫描结束时 h[0] 是第 k 大频次；任何 count>=T 的用户排名都不会低于 k 的阈值位置。 最终复杂度为：计数 O(n)，求阈值 O(u log k)，再扫描 O(u)，空间 O(u+k)。

Q062 Meeting Rooms II：最少会议室

难度 M	核心 Heap / 区间	类别 Heap / Priority Queue

题目

给定会议时间区间，求同时容纳所有会议所需的最少会议室数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 区间采用闭区间还是半开区间？端点相等时是否视为相交?

基线方案与瓶颈

对每个时间点统计重叠或两两比较较慢。

关键洞察：为什么这个数据结构合适

按开始时间排序，最小堆保存当前正在占用房间的结束时间。新会议开始前，所有 end<=start 的房间都可释放；加入当前 end，堆最大大小即并发峰值。

> **不变量（Invariant）** 任一时刻堆中恰好是已经开始但尚未结束的会议，因此堆大小就是需要的房间数。

参考实现（Python ）

```
import heapq

def min_meeting_rooms(intervals):
    intervals.sort()
    h=[]; best=0
    for s,e in intervals:
        while h and h[0] <= s: heapq.heappop(h)
        heapq.heappush(h,e); best=max(best,len(h))
    return best
```

复杂度

时间 O(n log n)，空间 O(n)。

高频失分点

- 端点相等是否可复用房间要根据题意
- 只弹一个结束会议仍可正确求房间但堆会残留过期项，最好全部弹

面试官追问

- 用 start/end 两个排序数组如何 O(n log n) 完成?
- 如何实际分配 room id?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个时间点统计重叠或两两比较较慢。 真正的瓶颈在重复工作，因此优化的核心是：按开始时间排序，最小堆保存当前正在占用房间的结束时间。新会议开始前，所有 end<=start 的房间都可释放；加入当前 end，堆最大大小即并发峰值。 正确性依赖的不变量是：任一时刻堆中恰好是已经开始但尚未结束的会议，因此堆大小就是需要的房间数。 最终复杂度为：时间 O(n log n)，空间 O(n)。

Q063 K 个有序数组的归并

难度 M/H	核心 Heap / K-way Merge	类别 Heap / Priority Queue

题目

给定 k 个有序数组，输出全局有序序列。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

全部拼接后排序 O(N log N)。

关键洞察：为什么这个数据结构合适

堆中只放每个数组尚未消费的首元素。每次弹出全局最小值，再把该元素所在数组的下一个元素加入堆。

> **不变量（Invariant）** 每个数组自身有序，所以一个数组除当前首元素外的任何剩余元素都不可能先于它成为全局最小。

参考实现（Python ）

```
import heapq

def merge_k_arrays(arrs):
    h=[]; out=[]
    for i,a in enumerate(arrs):
        if a: heapq.heappush(h,(a[0],i,0))
    while h:
        x,i,j=heapq.heappop(h); out.append(x)
        if j+1<len(arrs[i]): heapq.heappush(h,(arrs[i][j+1],i,j+1))
    return out
```

复杂度

时间 O(N log k)，空间 O(k)。

高频失分点

- 堆放所有元素而非 k 个 frontier
- 值相等时 tuple 比较需要稳定字段

面试官追问

- 外排序/日志归并为什么是相同结构?
- 若数组在远程机器上，如何考虑 I/O 和批量预取?

60 秒口述模板

我会先给出一个可工作的基线方案：全部拼接后排序 O(N log N)。 真正的瓶颈在重复工作，因此优化的核心是：堆中只放每个数组尚未消费的首元素。每次弹出全局最小值，再把该元素所在数组的下一个元素加入堆。 正确性依赖的不变量是：每个数组自身有序，所以一个数组除当前首元素外的任何剩余元素都不可能先于它成为全局最小。 最终复杂度为：时间 O(N log k)，空间 O(k)。

Q064 数据流中位数

难度 H	核心 Heap / Two Heaps	类别 Heap / Priority Queue

题目

设计结构，在线插入数字并随时返回当前中位数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次查询时排序全部数据 O(n log n)；保持有序数组插入 O(n)。

关键洞察：为什么这个数据结构合适

用两个堆：low 最大堆保存较小一半，high 最小堆保存较大一半。维持 max(low)<=min(high) 且大小差 <=1。

不变量（Invariant） 两个堆分别覆盖排序后的左右两半；平衡条件保证中间元素总在堆顶。

参考实现（Python ）

```
import heapq
class MedianFinder:
    def __init__(self): self.low=[]; self.high=[]
    def addNum(self,x):
        heapq.heappush(self.low,-x)
        heapq.heappush(self.high,-heapq.heappop(self.low))
        if len(self.high)>len(self.low):
            heapq.heappush(self.low,-heapq.heappop(self.high))
    def findMedian(self):
        if len(self.low)>len(self.high): return -self.low[0]
        return (-self.low[0]+self.high[0])/2
```

复杂度

插入 O(log n)，查询中位数 O(1)，空间 O(n)。

高频失分点

- 只维护大小平衡，没维护左右有序关系
- 偶数个元素整数除法
- 最大堆用负数时符号混乱

面试官追问

- 如何支持删除任意元素？见 Q65
- 如何求在线 90% 分位数?

60 秒口述模板

我会先给出一个可工作的基线方案：每次查询时排序全部数据 O(n log n)；保持有序数组插入 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：用两个堆：low 最大堆保存较小一半，high 最小堆保存较大一半。维持 max(low)<=min(high) 且大小差<=1。 正确性依赖的不变量是：两个堆分别覆盖排序后的左右两半；平衡条件保证中间元素总在堆顶。 最终复杂度为：插入 O(log n)，查询中位数 O(1)，空间 O(n)。

Q065 滑动窗口中位数

难度 H+	核心 Two Heaps / Lazy Deletion	类别 Heap / Priority Queue

题目

对每个大小为 k 的滑动窗口输出中位数。窗口移动时既要插入新值，也要删除离开的旧值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每个窗口排序 O(n*k log k)；有序数组插入/删除 O(k)。

关键洞察：为什么这个数据结构合适

沿用双堆，但普通 heap 不支持 O(log n) 删除任意元素。使用 delayed Counter 标记待删除值；只有当它出现在堆顶时才真正 pop，并维护两个堆的“有效元素数量”。

不变量（Invariant） 逻辑大小只统计未延迟删除的元素；prune 后堆顶一定是有效边界元素，因此中位数正确。

参考实现（Python ）

```
# 面试中建议先讲清“双堆 + delayed deletion”结构，再编码。
# 完整实现通常包含 add/remove/rebalance/prune 四个 helper。
from collections import Counter
import heapq

class DualHeap:
    def __init__(self,k):
        self.small=[]; self.large=[]; self.delayed=Counter(); self.k=k
        self.ss=self.ls=0
    def prune(self,h,sign):
        while h and self.delayed[sign*h[0]]:
            x=sign*heapq.heappop(h); self.delayed[x]-=1
    # add/remove/rebalance 按有效大小维护两个堆；生产代码应补齐边界测试。
```

复杂度

每元素入堆一次、最终出堆一次，整体 O(n log k)，空间 O(k) 级别。

高频失分点

- 把 heap.remove 当 O(log n)，实际上 Python list 查找是 O(k)
- 延迟删除后仍用 len(heap) 做平衡会错
- 重复值时必须按计数延迟删除

面试官追问

- 如果语言提供 multiset/order-statistic tree，方案会更简单吗?
- 为什么这是“逻辑删除”与“物理删除”分离?

60 秒口述模板

我会先给出一个可工作的基线方案：每个窗口排序 O(n*k log k)；有序数组插入/删除 O(k)。 真正的瓶颈在重复工作，因此优化的核心是：沿用双堆，但普通 heap 不支持 O(log n) 删除任意元素。使用 delayed Counter 标记待删除值；只有当它出现在堆顶时才真正 pop，并维护两个堆的“有效元素数量”。 正确性依赖的不变量是：逻辑大小只统计未延迟删除的元素；prune 后堆顶一定是有效边界元素，因此中位数正确。 最终复杂度为：每元素入堆一次、最终出堆一次，整体 O(n log k)，空间 O(k) 级别。

Q066 多服务器任务调度

难度 H	核心 Heap / 设计	类别 Heap / Priority Queue

题目

有若干服务器（权重/编号不同）和按时间到达的任务，每个任务占用一定时长；按规则选择可用服务器，输出分配结果。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次任务到来线性扫描所有服务器可 O(tasks*servers)。

关键洞察：为什么这个数据结构合适

用两个堆：available 按选择优先级（如 weight,index）排序；busy 按释放时间排序。处理时间 t 时先把 release<=t 的服务器移回 available；若无可用服务器，把时间跳到最早 release。

不变量（Invariant） available 始终包含当前时刻所有空闲服务器且堆顶最优；busy 包含所有占用服务器且堆顶最早释放。

参考实现（Python ）

```
import heapq

def assign_tasks(servers,tasks):
    avail=[(w,i) for i,w in enumerate(servers)]; heapq.heapify(avail)
    busy=[]; ans=[]; t=0
    for j,dur in enumerate(tasks):
        t=max(t,j)
        while busy and busy[0][0]<=t:
            free,w,i=heapq.heappop(busy); heapq.heappush(avail,(w,i))
        if not avail:
            t=busy[0][0]
            while busy and busy[0][0]<=t:
                free,w,i=heapq.heappop(busy); heapq.heappush(avail,(w,i))
        w,i=heapq.heappop(avail); ans.append(i)
        heapq.heappush(busy,(t+dur,w,i))
    return ans
```

复杂度

若任务数 n、服务器 m，时间 O((n+m)log m)，空间 O(m)。

高频失分点

- 时间没有跳跃，导致空转
- 释放多个同时间服务器时只 pop 一个
- tie-breaking 没进入 heap key

面试官追问

- 加入服务器故障/动态扩容后结构怎样演化?
- 与事件驱动模拟有什么关系?

60 秒口述模板

我会先给出一个可工作的基线方案：每次任务到来线性扫描所有服务器可 O(tasks*servers)。 真正的瓶颈在重复工作，因此优化的核心是：用两个堆：available 按选择优先级（如 weight,index）排序；busy 按释放时间排序。处理时间 t 时先把 release<=t 的服务器移回 available；若无可用服务器，把时间跳到最早 release。 正确性依赖的不变量是：available 始终包含当前时刻所有空闲服务器且堆顶最优；busy 包含所有占用服务器且堆顶最早释放。 最终复杂度为：若任务数 n、服务器 m，时间 O((n+m)log m)，空间 O(m)。

第七章

Trie / Prefix Tree

Q067 - Q072

把字符串集合压缩成公共前缀图；理解空间换查询与前缀剪枝。

Q067 实现 Trie：insert/search/startsWith

难度 M	核心 Trie / 字符串	类别 Trie / Prefix Tree

题目

从零实现前缀树，支持插入单词、完整词查询、前缀查询。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

用 HashSet 保存所有完整字符串时 search 很快，但 startsWith 可能需要扫描所有词。

关键洞察：为什么这个数据结构合适

Trie 将每个字符作为一条边，共享公共前缀。节点需区分“路径存在”和“这里恰好是一个完整单词”，通常使用 terminal/end 标记。

> **不变量（Invariant）** 从根沿字符路径走到第 i 个节点时，该路径恰好表示前 i 个字符；路径不存在即可立即失败。

参考实现（Python ）

```
class TrieNode:
    def __init__(self): self.ch={}; self.end=False
class Trie:
    def __init__(self): self.root=TrieNode()
    def insert(self,w):
        p=self.root
        for c in w: p=p.ch.setdefault(c,TrieNode())
        p.end=True
    def search(self,w):
        p=self._walk(w); return bool(p and p.end)
    def startsWith(self,prefix): return self._walk(prefix) is not None
    def _walk(self,s):
        p=self.root
        for c in s:
            if c not in p.ch: return None
            p=p.ch[c]
        return p
```

复杂度

每个操作 O(L)，L 为字符串长度；空间与所有唯一前缀节点数成正比。

高频失分点

- 忘记 terminal，导致 app 与 apple 无法区分
- 固定 26 children 与 HashMap 的空间 trade-off 没讨论

面试官追问

- 如何支持 delete?
- Radix Tree/Patricia Trie 如何压缩单孩子链?

60 秒口述模板

我会先给出一个可工作的基线方案：用 HashSet 保存所有完整字符串时 search 很快，但 startsWith 可能需要扫描所有词。 真正的瓶颈在重复工作，因此优化的核心是：Trie 将每个字符作为一条边，共享公共前缀。节点需区分“路径存在”和“这里恰好是一个完整单词”，通常使用 terminal/end 标记。 正确性依赖的不变量是：从根沿字符路径走到第 i 个节点时，该路径恰好表示前 i 个字符；路径不存在即可立即失败。 最终复杂度为：每个操作 O(L)，L 为字符串长度；空间与所有唯一前缀节点数成正比。

Q068 支持通配符 . 的词典

难度 M	核心 Trie / DFS	类别 Trie / Prefix Tree

题目

设计词典，addWord 后 search(pattern)，其中 . 可匹配任意单个字符。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

若枚举词典中所有同长度词并逐字符匹配，查询 O(total words)。

关键洞察：为什么这个数据结构合适

Trie 对普通字符只走唯一边；遇到 . 时 DFS 尝试当前节点所有 child。Trie 仍能利用 pattern 中确定字符进行强剪枝。

不变量（Invariant） DFS 状态由 (node,index) 唯一确定；当 index==len(pattern) 时只有 node.end 才表示完整词匹配。

参考实现（Python ）

```
class WordDictionary:
    def __init__(self): self.root=TrieNode()
    def addWord(self,w):
        p=self.root
        for c in w: p=p.ch.setdefault(c,TrieNode())
        p.end=True
    def search(self,w):
        def dfs(p,i):
            if i==len(w): return p.end
            c=w[i]
            if c=='.': return any(dfs(q,i+1) for q in p.ch.values())
            return c in p.ch and dfs(p.ch[c],i+1)
        return dfs(self.root,0)
```

复杂度

最坏指数于通配符数量，但通常被 Trie 分支与确定字符大幅剪枝；空间为 Trie 大小。

高频失分点

- pattern 消耗完只判断节点存在，不判断 end
- 把 . 当成任意长度而非单字符

面试官追问

- 若支持 * 匹配任意长度，状态空间怎样变化?

60 秒口述模板

我会先给出一个可工作的基线方案：若枚举词典中所有同长度词并逐字符匹配，查询 O(total words)。 真正的瓶颈在重复工作，因此优化的核心是：Trie 对普通字符只走唯一边；遇到 . 时 DFS 尝试当前节点所有 child。Trie 仍能利用 pattern 中确定字符进行强剪枝。 正确性依赖的不变量是：DFS 状态由 (node,index) 唯一确定；当 index==len(pattern) 时只有 node.end 才表示完整词匹配。 最终复杂度为：最坏指数于通配符数量，但通常被 Trie 分支与确定字符大幅剪枝；空间为 Trie 大小。

Q069 Autocomplete：前缀返回 Top-K 候选

难度 H	核心 Trie / Heap	类别 Trie / Prefix Tree

题目

维护字符串及热度，输入前缀时快速返回热度最高的 k 个候选。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

找到前缀节点后 DFS 整棵子树再排序，查询可能与词典总规模同阶。

关键洞察：为什么这个数据结构合适

读多写少时，可在每个 Trie 节点预计算/缓存该前缀 Top-K；更新单词热度时沿其字符路径更新 K 大候选。查询变成 O(prefix length + k)。

不变量（Invariant） 每个节点缓存集合始终是其整棵子树所有 terminal 词中的 Top-K，因此前缀定位后无需再遍历子树。

参考实现（Python ）

```
# 结构草图：面试重点是 trade-off，而非把所有工程细节一次写满。
class AutoNode:
    def __init__(self):
        self.children = {}
        self.top = []   # 可存最多 K 个 (score, word)，更新时维护
```

复杂度

查询 O(L+k)；更新约 O(L log k)（取决于缓存结构）；空间会增加 O(节点数*k)。

高频失分点

- 只说 Trie，不解决 Top-K 排序成本
- 忽略热度更新导致缓存失效
- 大 k 时每节点缓存空间爆炸

面试官追问

- 写多读少如何改成查询时计算?
- 如何做个性化 autocomplete?
- 分布式场景如何 shard?

60 秒口述模板

我会先给出一个可工作的基线方案：找到前缀节点后 DFS 整棵子树再排序，查询可能与词典总规模同阶。 真正的瓶颈在重复工作，因此优化的核心是：读多写少时，可在每个 Trie 节点预计算/缓存该前缀 Top-K；更新单词热度时沿其字符路径更新 K 大候选。查询变成 O(prefix length + k)。 正确性依赖的不变量是：每个节点缓存集合始终是其整棵子树所有 terminal 词中的 Top-K，因此前缀定位后无需再遍历子树。 最终复杂度为：查询 O(L+k)；更新约 O(L log k)（取决于缓存结构）；空间会增加 O(节点数*k)。

Q070 Word Search II：棋盘中查找多个单词

难度 H	核心 Trie / DFS	类别 Trie / Prefix Tree

题目

给定字符棋盘和词典，找出能通过上下左右相邻格组成的所有词，每格在一个词中最多使用一次。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

对每个 word 独立从棋盘 DFS，会重复探索大量相同前缀。

关键洞察：为什么这个数据结构合适

先把所有词构建 Trie。棋盘 DFS 时同步沿 Trie 走；当前字符不在 Trie child 立即剪枝。到 terminal 节点记录答案，并可清除 terminal 防重复；若节点无 child 可进一步剪枝 Trie。

> **不变量（Invariant）** DFS 路径既是棋盘合法路径也是词典前缀；一旦前缀在 Trie 中不存在，任何继续扩展都不可能形成目标词。

参考实现（Python ）

```
def find_words(board, words):
    root=TrieNode()
    for w in words:
        p=root
        for c in w: p=p.ch.setdefault(c,TrieNode())
        p.word=w
    R,C=len(board),len(board[0]); ans=[]
    def dfs(r,c,p):
        ch=board[r][c]
        if ch not in p.ch: return
        q=p.ch[ch]
        if getattr(q,'word',None): ans.append(q.word); q.word=None
        board[r][c]='#'
        for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
            nr,nc=r+dr,c+dc
            if 0<=nr<R and 0<=nc<C and board[nr][nc]!='#': dfs(nr,nc,q)
        board[r][c]=ch
    for r in range(R):
        for c in range(C): dfs(r,c,root)
    return ans
```

复杂度

复杂度依赖棋盘与词典前缀共享，最坏仍指数，但实际剪枝显著；空间为 Trie + DFS 深度。

高频失分点

- 对每个 word 单独 DFS
- visited 不回溯
- 找到词后重复输出
- TrieNode 需允许保存 word 字段

面试官追问

- 如何加入对角线?
- 如果棋盘很大且词典变化频繁，预处理谁更合适?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个 word 独立从棋盘 DFS，会重复探索大量相同前缀。 真正的瓶颈在重复工作，因此优化的核心是：先把所有词构建 Trie。棋盘 DFS 时同步沿 Trie 走；当前字符不在 Trie child 立即剪枝。到 terminal 节点记录答案，并可清除 terminal 防重复；若节点无 child 可进一步剪枝 Trie。 正确性依赖的不变量是：DFS 路径既是棋盘合法路径也是词典前缀；一旦前缀在 Trie 中不存在，任何继续扩展都不可能形成目标词。 最终复杂度为：复杂度依赖棋盘与词典前缀共享，最坏仍指数，但实际剪枝显著；空间为 Trie + DFS 深度。

Q071 最短词根替换句子

难度 M	核心 Trie / 字符串	类别 Trie / Prefix Tree

题目

给定词根词典，把句子中每个单词替换成它最短的、存在于词典中的前缀；若无则保留原词。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

对每个单词枚举所有前缀并查 set，最坏 O(L^2) 字符处理。

关键洞察：为什么这个数据结构合适

Trie 从单词第一个字符向下走；一旦到达 terminal，就已经找到最短可行词根，可立即返回。

不变量（Invariant） 沿单词路径第一次遇到 terminal 的深度最小，因此对应最短前缀。

参考实现（Python ）

```
def replace_words(dictionary, sentence):
    trie=Trie()
    for w in dictionary: trie.insert(w)
    def repl(w):
        p=trie.root; out=[]
        for c in w:
            if c not in p.ch: return w
            out.append(c); p=p.ch[c]
            if p.end: return ''.join(out)
        return w
    return ' '.join(repl(w) for w in sentence.split())
```

复杂度

总时间 O(句子字符数) 量级，空间为 Trie。

高频失分点

- 找到更长 root 后才返回，违反“最短”
- 标点/大小写的文本规范化未澄清

面试官追问

- 如果词根有权重且要选择“最优”而非最短，节点需要存什么？

60 秒口述模板

我会先给出一个可工作的基线方案：对每个单词枚举所有前缀并查 set，最坏 O(L^2) 字符处理。 真正的瓶颈在重复工作，因此优化的核心是：Trie 从单词第一个字符向下走；一旦到达 terminal，就已经找到最短可行词根，可立即返回。 正确性依赖的不变量是：沿单词路径第一次遇到 terminal 的深度最小，因此对应最短前缀。 最终复杂度为：总时间 O(句子字符数) 量级，空间为 Trie。

Q072 层级路径/域名祖先值聚合

难度 H	核心 Trie / Tree	类别 Trie / Prefix Tree

来源标记 公开候选人面经变体：Google Trie/层级字符串问题；候选人自述，非官方题库。

题目

给定若干层级字符串（如域名或路径）及数值，每条记录的值需要贡献给自身及相关祖先/后代查询；要求高效计算每个目标节点的累计值。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

对每个目标与每条记录比较前缀/后缀关系会 O(n^2*L)。

关键洞察：为什么这个数据结构合适

将层级 component 作为 Trie 边（域名常把组件反转，使共同祖先如 com→example 自然共享前缀）。插入时把值累计到节点；随后 DFS 传递祖先累计值或对子树做聚合。

不变量（Invariant） Trie 路径与层级祖先链一一对应，因此“祖先贡献”转化为树上的前缀累计。

参考实现（Python ）

```
class PathNode:
    def __init__(self): self.ch={}; self.own=0; self.total=0

def aggregate_paths(items):
    root=PathNode()
    for path,val in items:
        p=root
        for part in reversed(path.split('.')):  # 域名示例
            p=p.ch.setdefault(part,PathNode())
        p.own += val
    def dfs(p,acc):
        p.total=acc+p.own
        for q in p.ch.values(): dfs(q,p.total)
    dfs(root,0)
    return root
```

复杂度

构建与遍历约 O(总 component 数)，空间同阶。

高频失分点

- 层级方向选错导致公共祖先不能共享路径
- 只建 Trie 不说明 value 在哪一层累计
- 路径分隔符、重复记录、根节点语义

面试官追问

- 如果查询很多但更新也频繁，如何缓存?
- 路径存在 wildcard 时结构如何改?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个目标与每条记录比较前缀/后缀关系会 O(n^2*L)。 真正的瓶颈在重复工作，因此优化的核心是：将层级 component 作为 Trie 边（域名常把组件反转，使共同祖先如 com→example 自然共享前缀）。插入时把值累计到节点；随后 DFS 传递祖先累计值或对子树做聚合。 正确性依赖的不变量是：Trie 路径与层级祖先链一一对应，因此“祖先贡献”转化为树上的前缀累计。 最终复杂度为：构建与遍历约 O(总 component 数)，空间同阶。

第八章

Graph / BFS / DFS / Union-Find

Q073 - Q086

先建模图，再决定遍历、最短路、拓扑、连通性还是最小生成树。

Q073 岛屿数量

难度 M	核心 Graph / Grid	类别 Graph / BFS / DFS / Union-Find

题目

二维网格中 1 表示陆地、0 表示水，上下左右相连陆地属于同一岛，统计岛屿数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

若没有 visited，重复从每个陆地开始搜索会大量重复。

关键洞察：为什么这个数据结构合适

网格就是隐式图。扫描每个格子，遇到尚未访问的陆地说明发现一个新连通分量，计数 +1，并 DFS/BFS 把整个分量标记掉。

不变量（Invariant） 每次外层遇到 1 时，它不可能属于此前已发现岛屿，否则早已被 flood fill 标记。

参考实现（Python ）

```
def num_islands(g):
    if not g: return 0
    R,C=len(g),len(g[0]); ans=0
    def dfs(r,c):
        if not (0<=r<R and 0<=c<C) or g[r][c]!='1': return
        g[r][c]='0'
        for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)): dfs(r+dr,c+dc)
    for r in range(R):
        for c in range(C):
            if g[r][c]=='1': ans+=1; dfs(r,c)
    return ans
```

复杂度

时间 O(RC)，空间 DFS 最坏 O(RC) 或原地标记。

高频失分点

- 访问后才标记，可能在图有环时重复入栈
- 未确认能否修改输入

- 递归深度风险

面试官追问

- 8 邻接如何改?
- 动态增加陆地并实时报告岛屿数如何用 DSU?

60 秒口述模板

我会先给出一个可工作的基线方案：若没有 visited，重复从每个陆地开始搜索会大量重复。 真正的瓶颈在重复工作，因此优化的核心是：网格就是隐式图。扫描每个格子，遇到尚未访问的陆地说明发现一个新连通分量，计数 +1，并 DFS/BFS 把整个分量标记掉。 正确性依赖的不变量是：每次外层遇到 1 时，它不可能属于此前已发现岛屿，否则早已被 flood fill 标记。 最终复杂度为：时间 O(RC)，空间 DFS 最坏 O(RC) 或原地标记。

Q074 Clone Graph

难度 M	核心 Graph / DFS/BFS	类别 Graph / BFS / DFS / Union-Find

题目

深拷贝一个可能有环的无向/有向图。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

若递归时每次遇到邻居都新建节点，会在环上无限递归并产生重复副本。

关键洞察：为什么这个数据结构合适

维护 oldNode→newNode 映射。第一次见到节点时先创建副本并立即登记，再递归复制邻居。映射既用于 memoization，也充当 visited。

> **不变量（Invariant）** 任何旧节点只会创建一个副本；所有边最终都从旧端点映射到对应新端点。

参考实现（Python ）

```
def clone_graph(node):
    mp={}
    def dfs(x):
        if x is None: return None
        if x in mp: return mp[x]
        cp=Node(x.val); mp[x]=cp
        cp.neighbors=[dfs(y) for y in x.neighbors]
        return cp
    return dfs(node)
```

复杂度

时间 O(V+E)，空间 O(V)。

高频失分点

- 创建副本后不立即写 memo，遇到环会递归爆炸
- 用节点值当 key，但图中值可能重复

面试官追问

- 如何迭代 BFS 实现?
- 若节点包含更多 mutable metadata，深拷贝边界怎么定义?

60 秒口述模板

我会先给出一个可工作的基线方案：若递归时每次遇到邻居都新建节点，会在环上无限递归并产生重复副本。 真正的瓶颈在重复工作，因此优化的核心是：维护 oldNode→newNode 映射。第一次见到节点时先创建副本并立即登记，再递归复制邻居。映射既用于 memoization，也充当 visited。 正确性依赖的不变量是：任何旧节点只会创建一个副本；所有边最终都从旧端点映射到对应新端点。 最终复杂度为：时间 O(V+E)，空间 O(V)。

Q075 判断无向图是否为树

难度 M	核心 Graph / DFS	类别 Graph / BFS / DFS / Union-Find

题目

给定 n 个节点及无向边，判断其是否构成一棵树。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

只检查连通或只检查无环都不够。

关键洞察：为什么这个数据结构合适

树的等价条件：n 个节点、n-1 条边且连通；或者 DFS 检测无环并确认全部访问；也可 DSU 插边时检测是否把同一集合再次连接。

> **不变量（Invariant）** 无向连通图若恰有 n-1 条边必为树；反之树必满足这两个条件。

参考实现（Python ）

```
def valid_tree(n, edges):
    if len(edges) != n-1: return False
    g=[[] for _ in range(n)]
    for a,b in edges: g[a].append(b); g[b].append(a)
    seen={0}; st=[0]
    while st:
        x=st.pop()
        for y in g[x]:
            if y not in seen: seen.add(y); st.append(y)
    return len(seen)==n
```

复杂度

DFS/DSU 时间近 O(V+E)，空间 O(V)。

高频失分点

- 无向 DFS 把父边误判成 cycle
- n=0 语义

- 只用 E=V-1 不检查连通

面试官追问

- 多次动态加边，哪种 DSU API 更自然?

60 秒口述模板

我会先给出一个可工作的基线方案：只检查连通或只检查无环都不够。 真正的瓶颈在重复工作，因此优化的核心是：树的等价条件：n 个节点、n-1 条边且连通；或者 DFS 检测无环并确认全部访问；也可 DSU 插边时检测是否把同一集合再次连接。 正确性依赖的不变量是：无向连通图若恰有 n-1 条边必为树；反之树必满足这两个条件。 最终复杂度为：DFS/DSU 时间近 O(V+E)，空间 O(V)。

Q076 无向图连通分量数量

难度 M	核心 Graph / DFS	类别 Graph / BFS / DFS / Union-Find

题目

给定 n 个节点和无向边，计算连通分量个数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

从每个节点分别搜索但不共享 visited 会重复工作。

关键洞察：为什么这个数据结构合适

DFS/BFS：每次从未访问节点开始一次遍历就发现一个完整分量；或者 DSU 初始 components=n，每次成功 union 两个不同 root 时 components--。

不变量（Invariant） 一次搜索会标记该起点所在整个连通分量，因此外层下一次未访问节点一定属于新分量。

参考实现（Python ）

```
def count_components(n,edges):
    parent=list(range(n)); size=[1]*n; c=n
    def find(x):
        while x!=parent[x]:
            parent[x]=parent[parent[x]]; x=parent[x]
        return x
    for a,b in edges:
        ra,rb=find(a),find(b)
        if ra!=rb:
            if size[ra]<size[rb]: ra,rb=rb,ra
            parent[rb]=ra; size[ra]+=size[rb]; c-=1
    return c
```

复杂度

DFS O(V+E)；DSU O((V+E)α(V))。

高频失分点

- union 不先 find root
- 每条边都 c--，重复边会错

面试官追问

- 为什么路径压缩 + 按秩合并近似常数?

60 秒口述模板

我会先给出一个可工作的基线方案：从每个节点分别搜索但不共享 visited 会重复工作。 真正的瓶颈在重复工作，因此优化的核心是：DFS/BFS：每次从未访问节点开始一次遍历就发现一个完整分量；或者 DSU 初始 components=n，每次成功 union 两个不同 root 时 components--。 正确性依赖的不变量是：一次搜索会标记该起点所在整个连通分量，因此外层下一次未访问节点一定属于新分量。 最终复杂度为：DFS O(V+E)；DSU O((V+E)α(V))。

Q077 Course Schedule：课程依赖是否可完成

难度 M	核心 Graph / Topological Sort	类别 Graph / BFS / DFS / Union-Find

题目

课程先修关系构成有向图，判断是否存在一种顺序能完成全部课程。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

尝试所有课程排列是阶乘级。

关键洞察：为什么这个数据结构合适

问题等价于有向图是否无环。Kahn 算法维护入度，反复删除入度为 0 的节点；若最终处理数=n，则有拓扑序，否则剩余节点位于环中。

不变量（Invariant） 队列中的节点恰是当前剩余图中没有未满足前驱的节点；删除它不会破坏任何合法约束。

参考实现（Python ）

```
from collections import deque

def can_finish(n, prereq):
    g=[[] for _ in range(n)]; indeg=[0]*n
    for course,pre in prereq: g[pre].append(course); indeg[course]+=1
    q=deque(i for i,d in enumerate(indeg) if d==0); done=0
    while q:
        x=q.popleft(); done+=1
        for y in g[x]:
            indeg[y]-=1
            if indeg[y]==0: q.append(y)
    return done==n
```

复杂度

时间 O(V+E)，空间 O(V+E)。

高频失分点

- 边方向 course→pre 与 pre→course 搞反
- 把 BFS shortest path 模板机械套用
- 重复 prerequisite 是否可能导致重复边

面试官追问

- 如何返回一条合法课程顺序?
- 如何返回一个具体 cycle?

60 秒口述模板

我会先给出一个可工作的基线方案：尝试所有课程排列是阶乘级。 真正的瓶颈在重复工作，因此优化的核心是：问题等价于有向图是否无环。Kahn 算法维护入度，反复删除入度为 0 的节点；若最终处理数=n，则有拓扑序，否则剩余节点位于环中。 正确性依赖的不变量是：队列中的节点恰是当前剩余图中没有未满足前驱的节点；删除它不会破坏任何合法约束。 最终复杂度为：时间 O(V+E)，空间 O(V+E)。

Q078 Alien Dictionary：未知字母序

难度 H	核心 Graph / Topological Sort	类别 Graph / BFS / DFS / Union-Find

题目

给定按某个未知字母顺序排好序的单词列表，推导一种可能的字母顺序；若矛盾返回无解。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

比较所有单词对没有必要，也容易产生错误约束。

关键洞察：为什么这个数据结构合适

只需比较相邻单词，第一处不同字符决定一条有向边 a→b。特殊无解：较长单词排在其严格前缀之前，如 "abc" 在 "ab" 前。然后对字符图拓扑排序。

不变量（Invariant） 字典序相邻元素提供了足以约束全局排序的最小关系；第一处差异之前字符相等，之后差异不再决定两词顺序。

参考实现（Python ）

```
from collections import deque

def alien_order(words):
    g={c:set() for w in words for c in w}; indeg={c:0 for c in g}
    for a,b in zip(words,words[1:]):
        m=min(len(a),len(b)); i=0
        while i<m and a[i]==b[i]: i+=1
        if i==m:
            if len(a)>len(b): return ''
        elif b[i] not in g[a[i]]:
            g[a[i]].add(b[i]); indeg[b[i]]+=1
    q=deque(c for c in indeg if indeg[c]==0); out=[]
    while q:
        c=q.popleft(); out.append(c)
        for d in g[c]:
            indeg[d]-=1
            if indeg[d]==0: q.append(d)
    return ''.join(out) if len(out)==len(g) else ''
```

复杂度

建图 O(总字符数)，拓扑 O(V+E)。

高频失分点

- 忽略 prefix invalid case
- 同一边重复加入导致 indegree 重复
- 结果可能不唯一，不应强求某固定字符串

面试官追问

- 如何判断拓扑序是否唯一?

60 秒口述模板

我会先给出一个可工作的基线方案：比较所有单词对没有必要，也容易产生错误约束。 真正的瓶颈在重复工作，因此优化的核心是：只需比较相邻单词，第一处不同字符决定一条有向边 a→b。特殊无解：较长单词排在其严格前缀之前，如 "abc" 在 "ab" 前。然后对字符图拓扑排序。 正确性依赖的不变量是：字典序相邻元素提供了足以约束全局排序的最小关系；第一处差异之前字符相等，之后差异不再决定两词顺序。 最终复杂度为：建图 O(总字符数)，拓扑 O(V+E)。

Q079 Word Ladder 最短转换路径

难度 H	核心 Graph / BFS	类别 Graph / BFS / DFS / Union-Find

题目

每次可改变一个字符，且中间词必须在字典中，求 begin 到 end 的最少单词数/步数。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

DFS 会探索大量非最短路径；构建所有单词两两边为 O(N^2L)。

关键洞察：为什么这个数据结构合适

这是无权图 shortest path，天然 BFS。邻居可通过逐位替换 26 字母生成，也可按 wildcard pattern 预建桶。若搜索空间大，双向 BFS 从两端较小 frontier 扩展。

不变量（Invariant） BFS 第一次到达目标时使用的边数最少，因为节点按距离层级访问。

参考实现（Python ）

```
from collections import deque

def ladder_length(begin,end,words):
    s=set(words)
    if end not in s: return 0
    q=deque([(begin,1)])
    while q:
        w,d=q.popleft()
        if w==end: return d
        a=list(w)
        for i,old in enumerate(a):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                if c==old: continue
                a[i]=c; nw=''.join(a)
                if nw in s:
                    s.remove(nw); q.append((nw,d+1))
            a[i]=old
    return 0
```

复杂度

直接生成邻居约 O(N*L*alphabet) 量级；空间 O(N)。

高频失分点

- visited 在出队时才标记，会重复入队
- BFS 距离到底按节点数还是边数
- 忘记 end 不在词典

面试官追问

- 双向 BFS 为什么通常更快?
- 如何返回所有最短路径?

60 秒口述模板

我会先给出一个可工作的基线方案：DFS 会探索大量非最短路径；构建所有单词两两边为 O(N^2L)。 真正的瓶颈在重复工作，因此优化的核心是：这是无权图 shortest path，天然 BFS。邻居可通过逐位替换 26 字母生成，也可按 wildcard pattern 预建桶。若搜索空间大，双向 BFS 从两端较小 frontier 扩展。 正确性依赖的不变量是：BFS 第一次到达目标时使用的边数最少，因为节点按距离层级访问。 最终复杂度为：直接生成邻居约 O(N*L*alphabet) 量级；空间 O(N)。

Q080 Pacific Atlantic Water Flow

难度 M/H	核心 Graph / Reverse Search	类别 Graph / BFS / DFS / Union-Find

题目

矩阵高度中，水可从高/等高流向低/等高相邻格，找能同时流到两个不同边界海洋的格子。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

从每个格子分别模拟流向两个海洋会重复搜索，最坏很慢。

关键洞察：为什么这个数据结构合适

逆向思考：分别从两个海洋边界出发，沿“高度不下降”的方向反向 BFS/DFS，得到能流到各海洋的格子集合；最后取交集。

不变量（Invariant） 若海洋反向能到达 cell，则正向水流一定可沿反向路径倒序从 cell 到海洋。

参考实现（Python ）

```
def pacific_atlantic(h):
    if not h: return []
    R,C=len(h),len(h[0])
    def reach(starts):
        seen=set(starts); st=list(starts)
        while st:
            r,c=st.pop()
            for dr,dc in ((1,0),(-1,0),(0,1),(0,-1)):
                nr,nc=r+dr,c+dc
                if 0<=nr<R and 0<=nc<C and (nr,nc) not in seen and h[nr][nc]>=h[r][c]:
                    seen.add((nr,nc)); st.append((nr,nc))
        return seen
    pac=[(0,c) for c in range(C)]+[(r,0) for r in range(R)]
    atl=[(R-1,c) for c in range(C)]+[(r,C-1) for r in range(R)]
    return list(reach(pac)&reach(atl))
```

复杂度

时间 O(RC)，空间 O(RC)。

高频失分点

- 反向条件写成 <=
- 从每个 cell 正向做两次 DFS，重复巨大

面试官追问

- 类似“边界可达性”问题还有哪些反向搜索例子?

60 秒口述模板

我会先给出一个可工作的基线方案：从每个格子分别模拟流向两个海洋会重复搜索，最坏很慢。 真正的瓶颈在重复工作，因此优化的核心是：逆向思考：分别从两个海洋边界出发，沿“高度不下降”的方向反向 BFS/DFS，得到能流到各海洋的格子集合；最后取交集。 正确性依赖的不变量是：若海洋反向能到达 cell，则正向水流一定可沿反向路径倒序从 cell 到海洋。 最终复杂度为：时间 O(RC)，空间 O(RC)。

Q081 Binary Matrix 最短路径

难度 M	核心 BFS / Grid	类别 Graph / BFS / DFS / Union-Find

题目

0/1 矩阵中允许 8 方向（或题目指定方向）移动，阻塞格不可走，求左上到右下最短路径长度。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

DFS 可以找到路径但不保证第一条最短；Dijkstra 对等权边过重。

关键洞察：为什么这个数据结构合适

把每个可走格看成图节点，每次移动成本都为 1，因此 BFS 按层搜索。入队时立即标 visited，避免同一格重复进入队列。

> **不变量（Invariant）** 队列第 d 层的所有节点与起点最短距离都是 d，所以首次到终点即全局最短。

参考实现（Python ）

```
from collections import deque

def shortest_path_binary_matrix(g):
    n=len(g)
    if not n or g[0][0] or g[n-1][n-1]: return -1
    q=deque([(0,0,1)]); g[0][0]=1
    dirs=[(dr,dc) for dr in (-1,0,1) for dc in (-1,0,1) if dr or dc]
    while q:
        r,c,d=q.popleft()
        if (r,c)==(n-1,n-1): return d
        for dr,dc in dirs:
            nr,nc=r+dr,c+dc
            if 0<=nr<n and 0<=nc<n and g[nr][nc]==0:
                g[nr][nc]=1; q.append((nr,nc,d+1))
    return -1
```

复杂度

时间 O(RC)，空间 O(RC)。

高频失分点

- visited 在出队时标导致队列膨胀
- 方向数量与题意不一致
- 起点/终点阻塞边界

面试官追问

- 不同格子成本不同后为什么要 Dijkstra/0-1 BFS?

60 秒口述模板

我会先给出一个可工作的基线方案：DFS 可以找到路径但不保证第一条最短；Dijkstra 对等权边过重。 真正的瓶颈在重复工作，因此优化的核心是：把每个可走格看成图节点，每次移动成本都为 1，因此 BFS 按层搜索。入队时立即标 visited，避免同一格重复进入队列。 正确性依赖的不变量是：队列第 d 层的所有节点与起点最短距离都是 d，所以首次到终点即全局最短。 最终复杂度为：时间 O(RC)，空间 O(RC)。

Q082 Network Delay：Dijkstra 单源最短路

难度 M	核心 Graph / Dijkstra	类别 Graph / BFS / DFS / Union-Find

题目

有向正权图中，从源点发送信号，求所有节点收到信号所需最短时间的最大值；不可达返回 -1。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

对所有路径枚举不可行；普通 BFS 只适用于等权边。

关键洞察：为什么这个数据结构合适

Dijkstra 维护当前最短 tentative distance 的最小堆。每次弹出全局最小距离节点时，其最短路已确定；对出边做 relaxation。Python 常用“允许重复入堆 + 跳过 stale entry”简化 decrease-key。

> **不变量（Invariant）** 非负权保证任何尚未处理路径再绕经其他未确定节点都不可能把当前堆顶距离变得更小。

参考实现（Python ）

```
import heapq

def network_delay(edges,n,k):
    g=[[] for _ in range(n+1)]
    for u,v,w in edges: g[u].append((v,w))
    dist=[float('inf')]*(n+1); dist[k]=0; h=[(0,k)]
    while h:
        d,u=heapq.heappop(h)
        if d!=dist[u]: continue
        for v,w in g[u]:
            nd=d+w
            if nd<dist[v]: dist[v]=nd; heapq.heappush(h,(nd,v))
    ans=max(dist[1:]); return -1 if ans==float('inf') else ans
```

复杂度

时间 O((V+E)log V)，空间 O(V+E)。

高频失分点

- 负权边仍使用 Dijkstra
- 不跳 stale entry 虽通常仍正确但会做大量冗余工作
- 混淆最短路总和与最大到达时间

面试官追问

- 有负边用什么?
- 权重只有 0/1 时如何用 deque 做 0-1 BFS?

60 秒口述模板

我会先给出一个可工作的基线方案：对所有路径枚举不可行；普通 BFS 只适用于等权边。 真正的瓶颈在重复工作，因此优化的核心是：Dijkstra 维护当前最短 tentative distance 的最小堆。每次弹出全局最小距离节点时，其最短路已确定；对出边做 relaxation。Python 常用“允许重复入堆 + 跳过 stale entry”简化 decrease-key。 正确性依赖的不变量是：非负权保证任何尚未处理路径再绕经其他未确定节点都不可能把当前堆顶距离变得更小。 最终复杂度为：时间 O((V+E)log V)，空间 O(V+E)。

Q083 连接所有点的最小代价：MST

难度 M/H	核心 Graph / MST	类别 Graph / BFS / DFS / Union-Find

题目

给定节点之间的连接成本，选择若干边使所有节点连通且总成本最小。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

枚举所有 spanning tree 组合指数级。

关键洞察：为什么这个数据结构合适

识别最小生成树。Kruskal 按边权排序，用 DSU 跳过会形成环的边；Prim 从一个点开始，不断选择跨越“已选集合/未选集合”的最便宜边。

不变量（Invariant） Kruskal 的 cut property：当前最小且连接两个不同分量的边可以安全加入某棵 MST。

参考实现（Python ）

```
def min_cost_connect(n,edges):
    edges=sorted(edges,key=lambda e:e[2]); p=list(range(n)); size=[1]*n
    def find(x):
        while x!=p[x]: p[x]=p[p[x]]; x=p[x]
        return x
    cost=used=0
    for u,v,w in edges:
        a,b=find(u),find(v)
        if a==b: continue
        if size[a]<size[b]: a,b=b,a
        p[b]=a; size[a]+=size[b]; cost+=w; used+=1
        if used==n-1: return cost
    return -1
```

复杂度

Kruskal O(E log E)，DSU 近 O(Eα(V))；空间 O(V+E)。

高频失分点

- 把最短路与 MST 混淆
- 图不连通时仍返回部分成本
- Kruskal union 前未 find

面试官追问

- 稠密完整图上 Prim O(V^2) 什么时候反而更合适?

60 秒口述模板

我会先给出一个可工作的基线方案：枚举所有 spanning tree 组合指数级。 真正的瓶颈在重复工作，因此优化的核心是：识别最小生成树。Kruskal 按边权排序，用 DSU 跳过会形成环的边；Prim 从一个点开始，不断选择跨越“已选集合/未选集合”的最便宜边。 正确性依赖的不变量是：Kruskal 的 cut property：当前最小且连接两个不同分量的边可以安全加入某棵 MST。 最终复杂度为：Kruskal O(E log E)，DSU 近 O(Eα(V))；空间 O(V+E)。

Q084 Accounts Merge

难度 M	核心 DSU / HashMap	类别 Graph / BFS / DFS / Union-Find

题目

多个账户记录可能因共享 email 属于同一人，合并所有连通账户并输出对应邮箱集合。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

两两比较账户 email 集合成本很高。

关键洞察：为什么这个数据结构合适

把 email 当节点，同一账户中的邮箱 union；或者把账户 index 做 DSU 并用 email→firstAccount 映射。最终按 root 收集所有 email。

不变量（Invariant） 共享 email 产生等价关系，DSU 正是维护等价类的结构；传递合并自然由 union 闭包完成。

参考实现（Python ）

```
from collections import defaultdict

def accounts_merge(accounts):
    p=list(range(len(accounts)))
    def find(x):
        if p[x]!=x: p[x]=find(p[x])
        return p[x]
    def union(a,b):
        a,b=find(a),find(b)
        if a!=b: p[b]=a
    owner={}
    for i,acc in enumerate(accounts):
        for email in acc[1:]:
            if email in owner: union(i,owner[email])
            else: owner[email]=i
    groups=defaultdict(list)
    for e,i in owner.items(): groups[find(i)].append(e)
    return [[accounts[r][0]]+sorted(es) for r,es in groups.items()]
```

复杂度

总 email 数 M，近 O(M α(M))，外加输出排序成本。

高频失分点

- 名字相同并不代表同一账户
- 合并后 find root 未再次压缩导致分组 key 不统一（通常 find 会处理）

面试官追问

- 如果 email 关系会被删除，DSU 为什么不擅长动态删除?

60 秒口述模板

我会先给出一个可工作的基线方案：两两比较账户 email 集合成本很高。 真正的瓶颈在重复工作，因此优化的核心是：把 email 当节点，同一账户中的邮箱 union；或者把账户 index 做 DSU 并用 email→firstAccount 映射。最终按 root 收集所有 email。 正确性依赖的不变量是：共享 email 产生等价关系，DSU 正是维护等价类的结构；传递合并自然由 union 闭包完成。 最终复杂度为：总 email 数 M，近 O(M α(M))，外加输出排序成本。

Q085 Redundant Connection：找成环边

难度 M	核心 DSU / Cycle Detection	类别 Graph / BFS / DFS / Union-Find

题目

无向图原本是一棵树，额外加入一条边，找出使图出现环的那条边。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 图是有向/无向？是否连通？是否允许重边/自环？边权是否非负?

基线方案与瓶颈

对每条边临时删除再检查是否是树会 O(E(V+E))。

关键洞察：为什么这个数据结构合适

按输入顺序使用 DSU 加边；若某条边两端已经在同一集合，说明此前已有路径连接它们，这条边正好闭合一个环。

> **不变量（Invariant）** 插入边 (u,v) 前 find(u)==find(v) 当且仅当当前图中已有 u→v 路径，因此加入该边产生环。

参考实现（Python ）

```
def find_redundant_connection(edges):
    n=len(edges); p=list(range(n+1)); rank=[0]*(n+1)
    def find(x):
        if p[x]!=x: p[x]=find(p[x])
        return p[x]
    for u,v in edges:
        a,b=find(u),find(v)
        if a==b: return [u,v]
        if rank[a]<rank[b]: a,b=b,a
        p[b]=a
        if rank[a]==rank[b]: rank[a]+=1
```

复杂度

时间 O(E α(V))，空间 O(V)。

高频失分点

- 把有向冗余连接题与无向版本混淆

- union 后才检查是否同 root

面试官追问

- 有向版本为何复杂很多?

60 秒口述模板

我会先给出一个可工作的基线方案：对每条边临时删除再检查是否是树会 O(E(V+E))。 真正的瓶颈在重复工作，因此优化的核心是：按输入顺序使用 DSU 加边；若某条边两端已经在同一集合，说明此前已有路径连接它们，这条边正好闭合一个环。 正确性依赖的不变量是：插入边 (u,v) 前 find(u)==find(v) 当且仅当当前图中已有 u→v 路径，因此加入该边产生环。 最终复杂度为：时间 O(E α(V))，空间 O(V)。

Q086 无根度≤3 树重新选根，使其成为二叉树并满足层颜色模式

难度 H+	核心 Tree / Rerooting	类别 Graph / BFS / DFS / Union-Find

来源标记 公开候选人面经变体：Google L5，2025；候选人自述，非官方题库。

题目

给一棵无根树，每节点度数不超过 3。选择一个根，使得定向后每节点最多两个孩子；进一步要求按深度满足交替颜色或固定周期颜色序列。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

对每个候选根做完整 BFS 验证，O(n^2)，虽然对小数据可先给作基线。

关键洞察：为什么这个数据结构合适

第一层结构观察：非根节点的一条边会成为 parent，因此 degree≤3 自动意味着 children≤2；真正可能违规的只有 root，所以根必须 degree≤2。加入颜色模式后，可把每个候选根的颜色/深度模周期约束与树的二分性/距离 parity 结合验证；更一般可做 reroot DP，把邻接方向的状态在边两侧复用。

不变量（Invariant） 选根只改变 parent/child 方向，不改变无向距离；深度颜色规则本质是“从 root 出发的距离 mod period”约束。

参考实现（Python ）

```
def binary_roots_by_degree(n, edges):
    deg=[0]*n
    for u,v in edges: deg[u]+=1; deg[v]+=1
    return [i for i,d in enumerate(deg) if d<=2]

# Follow-up 颜色条件：对候选 root 做一次 BFS，验证 color[node]
# 是否等于 pattern[(distance + offset) % len(pattern)]。
```

复杂度

基础筛选 O(n)；颜色模式可 O(n) 或通过 reroot 状态线性完成，取决于规则表达。

高频失分点

- 一开始就上复杂 reroot DP，忽略 degree≤3 带来的强简化
- 把“二叉树”误解为每节点 degree≤2；非根节点允许无向 degree=3
- 颜色模式未考虑起始 offset

面试官追问

- 如何把每个 root 的 BFS 从 O(n^2) 优化成 reroot DP?
- 若树边带权，层的概念如何定义?

60 秒口述模板

我会先给出一个可工作的基线方案：对每个候选根做完整 BFS 验证，O(n^2)，虽然对小数据可先给作基线。 真正的瓶颈在重复工作，因此优化的核心是：第一层结构观察：非根节点的一条边会成为 parent，因此 degree≤3 自动意味着 children≤2；真正可能违规的只有 root，所以根必须 degree≤2。加入颜色模式后，可把每个候选根的颜色/深度模周期约束与树的二分性/距离 parity 结合验证；更一般可做 reroot DP，把邻接方向的状态在边两侧复用。 正确性依赖的不变量是：选根只改变 parent/child 方向，不改变无向距离；深度颜色规则本质是“从 root 出发的距离 mod period”约束。 最终复杂度为：基础筛选 O(n)；颜色模式可 O(n) 或通过 reroot 状态线性完成，取决于规则表达。

第九章

Fenwick / Segment Tree / Range Structures

Q087 - Q092

处理“动态更新 + 区间查询”，理解数据结构如何缓存可组合的区间信息。

Q087 Fenwick Tree：动态单点修改 + 区间和

难度 M/H	核心 Fenwick Tree / 前缀和	类别 Fenwick / Segment Tree / Range Structures

题目

维护数组，支持 update(i,delta) 与 query(l,r) 区间和，要求优于 O(n) 每次。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

普通前缀和查询 O(1) 但单点更新 O(n)；直接数组更新 O(1) 但区间查询 O(n)。

关键洞察：为什么这个数据结构合适

Fenwick Tree 把下标 i 负责的块长度定义为 lowbit(i)=i&-i。prefix(i) 不断 i-=lowbit(i) 聚合若干不重叠块；update 不断 i+=lowbit(i) 更新所有覆盖该点的祖先块。

> **不变量（Invariant）** bit[i] 存某个以 i 结尾、长度 lowbit(i) 的区间和；prefix 分解会恰好覆盖 [1,i] 且不重叠。

参考实现（Python ）

```
class Fenwick:
    def __init__(self,n): self.bit=[0]*(n+1)
    def add(self,i,delta):
        i+=1
        while i<len(self.bit): self.bit[i]+=delta; i+=i&-i
    def prefix(self,i):
        s=0; i+=1
        while i: s+=self.bit[i]; i-=i&-i
        return s
    def range_sum(self,l,r): return self.prefix(r)-(self.prefix(l-1) if l else 0)
```

复杂度

update/query O(log n)，空间 O(n)。

高频失分点

- 0-based 与 1-based 下标混淆

- update 传新值还是 delta 未约定
- 不会解释 lowbit 的覆盖区间

面试官追问

- 如何做 range add + point query?
- 如何通过两个 BIT 做 range add + range sum?

60 秒口述模板

我会先给出一个可工作的基线方案：普通前缀和查询 O(1) 但单点更新 O(n)；直接数组更新 O(1) 但区间查询 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：Fenwick Tree 把下标 i 负责的块长度定义为 lowbit(i)=i&-i。prefix(i) 不断 i-=lowbit(i) 聚合若干不重叠块；update 不断 i+=lowbit(i) 更新所有覆盖该点的祖先块。 正确性依赖的不变量是：bit[i] 存某个以 i 结尾、长度 lowbit(i) 的区间和；prefix 分解会恰好覆盖 [1,i] 且不重叠。 最终复杂度为：update/query O(log n)，空间 O(n)。

Q088 Segment Tree：动态 Range Min/Max/Sum

难度 H	核心 Segment Tree / 区间查询	类别 Fenwick / Segment Tree / Range Structures

题目

维护数组并支持区间聚合查询及点更新；聚合可为 sum/min/max 等满足结合律的操作。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 区间采用闭区间还是半开区间？端点相等时是否视为相交?

基线方案与瓶颈

数组扫描每次 O(n)；前缀和只适合可逆的 sum 等，且更新昂贵。

关键洞察：为什么这个数据结构合适

Segment Tree 每节点表示一个区间聚合值，左右孩子覆盖区间二分。查询时：完全覆盖则直接返回；无交集返回单位元；部分交集递归合并。点更新只影响根到叶的一条路径。

不变量（Invariant） 每个节点值始终等于其两个孩子值的 merge；因此根与任意查询分解都能由 O(log n) 个相关节点组合。

参考实现（Python ）

```
class SegTree:
    def __init__(self,a):
        n=1
        while n<len(a): n*=2
        self.n=n; self.t=[float('inf')]*(2*n)
        self.t[n:n+len(a)]=a
        for i in range(n-1,0,-1): self.t[i]=min(self.t[2*i],self.t[2*i+1])
    def update(self,i,x):
        p=self.n+i; self.t[p]=x; p//=2
        while p: self.t[p]=min(self.t[2*p],self.t[2*p+1]); p//=2
    def query(self,l,r):  # [l,r)
        l+=self.n; r+=self.n; ans=float('inf')
        while l<r:
            if l&1: ans=min(ans,self.t[l]); l+=1
            if r&1: r-=1; ans=min(ans,self.t[r])
            l//=2; r//=2
        return ans
```

复杂度

build O(n)，point update O(log n)，range query O(log n) 常见情形，空间 O(n)。

高频失分点

- 区间是 [l,r] 还是 [l,r) 模板混淆
- 单位元选错
- 树数组分配大小不足

面试官追问

- Range update 如何引出 Lazy Propagation?
- 为何 min 无法像 sum 一样简单用前缀差?

60 秒口述模板

我会先给出一个可工作的基线方案：数组扫描每次 O(n)；前缀和只适合可逆的 sum 等，且更新昂贵。 真正的瓶颈在重复工作，因此优化的核心是：Segment Tree 每节点表示一个区间聚合值，左右孩子覆盖区间二分。查询时：完全覆盖则直接返回；无交集返回单位元；部分交集递归合并。点更新只影响根到叶的一条路径。 正确性依赖的不变量是：每个节点值始终等于其两个孩子值的 merge；因此根与任意查询分解都能由 O(log n) 个相关节点组合。 最终复杂度为：build O(n)，point update O(log n)，range query O(log n) 常见情形，空间 O(n)。

Q089 Range Module：动态添加/删除/查询区间

难度 H+	核心 Interval Tree / Segment Tree	类别 Fenwick / Segment Tree / Range Structures

来源标记 公开候选人面经变体：Google L5 Range Module，2025；候选人自述，非官方题库。

题目

维护实数/整数线上已覆盖区间，支持 addRange、removeRange、queryRange 判断某范围是否被完整覆盖。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 区间采用闭区间还是半开区间？端点相等时是否视为相交?

基线方案与瓶颈

若坐标范围小可布尔数组，但大坐标会不可行；每次扫描所有历史区间也会逐渐变慢。

关键洞察：为什么这个数据结构合适

两条主线：1）动态 Segment Tree + lazy coverage，适合超大坐标和大量操作；2）维护互不重叠有序区间，add/remove 时找到相交邻居并 split/merge。面试应先根据坐标范围、操作数量和 API 选择。

不变量（Invariant） 有序区间方案始终维护“集合内区间两两不重叠且不可再合并”；因此 query 只需找到 start 之前最近区间并检查其 end。

参考实现（Python ）

```
# Python 标准库没有 TreeMap；面试中可先写基于排序 list 的清晰版本，
# 再说明生产环境用有序映射/平衡树把定位从 O(n) 降到 O(log n)。
class RangeModuleList:
    def __init__(self): self.r=[]
    def addRange(self,l,r):
        out=[]; placed=False
        for a,b in self.r:
            if b<l: out.append((a,b))
            elif r<a:
                if not placed: out.append((l,r)); placed=True
                out.append((a,b))
            else: l=min(l,a); r=max(r,b)
        if not placed: out.append((l,r))
        self.r=out
```

复杂度

Ordered map 操作约 O((k+1)log n)，k 为被合并/切分区间数；动态线段树约 O(log C) 每次（带常数）。

高频失分点

- 没先确认端点语义与坐标范围
- 在 Python list 版本里声称所有操作 O(log n)
- removeRange 的 split 情况遗漏

面试官追问

- query 单点与 query 区间的 API 差异
- 隐式动态 Segment Tree 如何只创建访问到的节点?

60 秒口述模板

我会先给出一个可工作的基线方案：若坐标范围小可布尔数组，但大坐标会不可行；每次扫描所有历史区间也会逐渐变慢。 真正的瓶颈在重复工作，因此优化的核心是：两条主线：1）动态 Segment Tree + lazy coverage，适合超大坐标和大量操作；2）维护互不重叠有序区间，add/remove 时找到相交邻居并 split/merge。面试应先根据坐标范围、操作数量和 API 选择。 正确性依赖的不变量是：有序区间方案始终维护“集合内区间两两不重叠且不可再合并”；因此 query 只需找到 start 之前最近区间并检查其 end。 最终复杂度为：Ordered map 操作约 O((k+1)log n)，k 为被合并/切分区间数；动态线段树约 O(log C) 每次（带常数）。

Q090 逆序对数量

难度 H	核心 Fenwick Tree / Merge Sort	类别 Fenwick / Segment Tree / Range Structures

题目

统计 i<j 且 nums[i]>nums[j] 的二元组数量。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

双重循环 O(n^2)。

关键洞察：为什么这个数据结构合适

经典有两种 O(n log n)：归并排序时统计跨左右半的逆序；或从右向左扫描，用 Fenwick Tree 统计“已见元素中小于当前值的数量”。数值范围大时先坐标压缩。

不变量（Invariant） 从右向左处理 x 时，BIT 中恰好是原数组右侧元素频次，query(rank(x)-1) 就是满足 nums[j]<x 的数量。

参考实现（Python ）

```
def count_inversions(nums):
    vals=sorted(set(nums)); rank={x:i for i,x in enumerate(vals)}
    bit=Fenwick(len(vals)); ans=0
    for x in reversed(nums):
        r=rank[x]
        ans += bit.prefix(r-1) if r else 0
        bit.add(r,1)
    return ans
```

复杂度

时间 O(n log n)，空间 O(n)。

高频失分点

- 重复元素不算逆序，query 必须严格小于
- 坐标压缩改变的是 rank，不应改变比较关系

面试官追问

- 归并排序计数如何证明?
- Count Smaller After Self 与它是什么关系?

60 秒口述模板

我会先给出一个可工作的基线方案：双重循环 O(n^2)。 真正的瓶颈在重复工作，因此优化的核心是：经典有两种 O(n log n)：归并排序时统计跨左右半的逆序；或从右向左扫描，用 Fenwick Tree 统计“已见元素中小于当前值的数量”。 数值范围大时先坐标压缩。 正确性依赖的不变量是：从右向左处理 x 时，BIT 中恰好是原数组右侧元素频次，query(rank(x)-1) 就是满足 nums[j]<x 的数量。 最终复杂度为：时间 O(n log n)，空间 O(n)。

Q091 动态第 K 小：Order Statistic Tree

难度 H+	核心 Segment Tree / Order Statistics	类别 Fenwick / Segment Tree / Range Structures

题目

数据动态插入/删除，支持查询当前第 k 小值。假设值域可离线压缩。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 区间采用闭区间还是半开区间？端点相等时是否视为相交?

基线方案与瓶颈

维护排序数组每次插入/删除 O(n)；Heap 只方便极值，不方便任意 k。

关键洞察：为什么这个数据结构合适

在线段树叶子存每个值的频次，内部存子树总频次。找第 k 小时从根开始：若左子树 count>=k 进入左边，否则 k-=leftCount 进入右边。

不变量（Invariant） 每个节点的 count 是其值域内元素总数，因此左子树数量足以判断第 k 个元素落在左半还是右半。

参考实现（Python ）

```
# 假设 seg[node] 存区间频次和。
def kth(seg, node, l, r, k):
    if l==r: return l
    m=(l+r)//2; left_count=seg[node*2]
    if k<=left_count: return kth(seg,node*2,l,m,k)
    return kth(seg,node*2+1,m+1,r,k-left_count)
```

复杂度

update O(log M)，kth O(log M)，M 为压缩后不同值数；空间 O(M)。

高频失分点

- k 是 0-based 还是 1-based
- 删除后频次可变负数必须防御
- 在线出现新值时离线压缩方案失效

面试官追问

- 若键动态出现，平衡 BST 的 subtree_size 怎么维护?

60 秒口述模板

我会先给出一个可工作的基线方案：维护排序数组每次插入/删除 O(n)；Heap 只方便极值，不方便任意 k。 真正的瓶颈在重复工作，因此优化的核心是：在线段树叶子存每个值的频次，内部存子树总频次。找第 k 小时从根开始：若左子树 count>=k 进入左边，否则 k-=leftCount 进入右边。 正确性依赖的不变量是：每个节点的 count 是其值域内元素总数，因此左子树数量足以判断第 k 个元素落在左半还是右半。 最终复杂度为：update O(log M)，kth O(log M)，M 为压缩后不同值数；空间 O(M)。

Q092 SummaryRanges：数据流维护不相交区间

难度 H	核心 Ordered Map / Interval	类别 Fenwick / Segment Tree / Range Structures

题目

连续接收整数，随时返回当前所有已出现值组成的最小不相交闭区间集合。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

把所有值存 set，每次 query 排序重建区间，查询 O(n log n)。

关键洞察：为什么这个数据结构合适

插入 x 时只需看按 start 排序的左右相邻区间：x 可能独立成段、并入左、并入右、或把左右两段桥接成一段。平衡树可 O(log n) 找邻居。

不变量（Invariant） 区间集合始终两两不相交且不相邻；新点只可能影响其直接左右邻居，其他区间与 x 距离更远。

参考实现（Python ）

```
# 标准库简化版：用 list 展示核心四种合并情况；真正 O(log n) 需有序映射。
def add_num(intervals,x):
    out=[]; cur=[x,x]
    for a,b in intervals:
        if b+1 < cur[0]: out.append([a,b])
        elif cur[1]+1 < a:
            out.append(cur); cur=[a,b]
        else:
            cur=[min(cur[0],a),max(cur[1],b)]
    out.append(cur)
    return out
```

复杂度

使用 TreeMap 时 insert 约 O(log n)，输出 O(number of intervals)。

高频失分点

- 忽略“相邻”也应合并

- 用普通 list 却宣称 O(log n)
- 重复插入 x 的幂等性

面试官追问

- 如果支持删除一个数，区间可能 split，如何处理?

60 秒口述模板

我会先给出一个可工作的基线方案：把所有值存 set，每次 query 排序重建区间，查询 O(n log n)。 真正的瓶颈在重复工作，因此优化的核心是：插入 x 时只需看按 start 排序的左右相邻区间：x 可能独立成段、并入左、并入右、或把左右两段桥接成一段。平衡树可 O(log n) 找邻居。 正确性依赖的不变量是：区间集合始终两两不相交且不相邻；新点只可能影响其直接左右邻居，其他区间与 x 距离更远。 最终复杂度为：使用 TreeMap 时 insert 约 O(log n)，输出 O(number of intervals)。

第十章

复合数据结构设计

Q093 - Q100

把单一结构的短板通过组合补齐，并用 invariant 驱动 API 复杂度设计。

Q093 LRU Cache

难度 M/H	核心 HashMap / Doubly Linked List	类别 复合数据结构设计

来源标记 公开面经常以“最近搜索/最近使用记录”包装 LRU 思想；候选人自述题型仅作趋势证据。

题目

设计固定容量缓存，get/put 平均 O(1)；超过容量时淘汰最近最少使用的键。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

只用 HashMap 不知道谁最旧；只用链表能维护顺序，但 lookup O(n)。

关键洞察：为什么这个数据结构合适

HashMap key→node 提供 O(1) 定位；双向链表按新旧顺序排列，支持 O(1) 删除任意已知节点与移动到 MRU 端。使用头尾哨兵可消除大量边界分支。

不变量（Invariant） 链表从 LRU→MRU（或反向）始终表示访问时间顺序；哈希与链节点一一对应。任何 get/put 命中都把节点移动到 MRU 端。

参考实现（Python ）

```
class DNode:
    def __init__(self,k=0,v=0): self.k=k; self.v=v; self.prev=self.next=None
class LRUCache:
    def __init__(self,cap):
        self.cap=cap; self.mp={}; self.h=DNode(); self.t=DNode()
        self.h.next=self.t; self.t.prev=self.h
    def _remove(self,n): n.prev.next=n.next; n.next.prev=n.prev
    def _append(self,n):
        p=self.t.prev; p.next=n; n.prev=p; n.next=self.t; self.t.prev=n
    def get(self,k):
        if k not in self.mp: return -1
        n=self.mp[k]; self._remove(n); self._append(n); return n.v
    def put(self,k,v):
        if k in self.mp:
            n=self.mp[k]; n.v=v; self._remove(n); self._append(n); return
        n=DNode(k,v); self.mp[k]=n; self._append(n)
        if len(self.mp)>self.cap:
            old=self.h.next; self._remove(old); del self.mp[old.k]
```

复杂度

get/put 平均 O(1)，空间 O(capacity)。

高频失分点

- 单链表无法 O(1) 删除任意已知节点的前驱关系
- 更新已有 key 时重复创建节点
- capacity=0/1 边界
- 链表与 map 删除不同步

面试官追问

- 并发 LRU 如何加锁?
- 加入 TTL 怎么办?
- LFU 为什么需要额外频率维度?

60 秒口述模板

我会先给出一个可工作的基线方案：只用 HashMap 不知道谁最旧；只用链表能维护顺序，但 lookup O(n)。 真正的瓶颈在重复工作，因此优化的核心是：HashMap key→node 提供 O(1) 定位；双向链表按新旧顺序排列，支持 O(1) 删除任意已知节点与移动到 MRU 端。使用头尾哨兵可消除大量边界分支。 正确性依赖的不变量是：链表从 LRU→MRU（或反向）始终表示访问时间顺序；哈希与链节点一一对应。任何 get/put 命中都把节点移动到 MRU 端。 最终复杂度为：get/put 平均 O(1)，空间 O(capacity)。

Q094 LFU Cache

难度 H	核心 HashMap / Frequency Buckets	类别 复合数据结构设计

题目

固定容量缓存，淘汰使用频率最低的键；频率相同时按 LRU 淘汰。要求 get/put 平均 O(1)。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次淘汰扫描全部键找最低频会 O(n)。

关键洞察：为什么这个数据结构合适

维护 key→node；freq→双向链表（同频节点按 LRU 排列）；minFreq 指向当前最低频率。访问节点时从旧频率链移除并加入 freq+1 链尾；若旧链为空且等于 minFreq，则 minFreq++。

不变量（Invariant） 每个节点只存在于与其 freq 对应的一条链；minFreq 始终是非空频率桶最小值，因此淘汰可直接从该桶 LRU 端取。

参考实现（Python ）

```
# 面试代码建议复用带哨兵的 DoublyLinkedList。
# 状态核心：
# nodes[key] -> node(key,value,freq)
# buckets[freq] -> ordered nodes of that freq
# min_freq -> smallest non-empty frequency
```

复杂度

平均 get/put O(1)，空间 O(capacity)。

高频失分点

- 只维护频率，不处理同频 LRU
- 桶空后忘记更新 minFreq
- put 新键 minFreq 必须重置为 1

面试官追问

- 如果频率无限增长如何 aging?
- 为什么“频率+时间”两个排序维度不适合一个简单 heap?

60 秒口述模板

我会先给出一个可工作的基线方案：每次淘汰扫描全部键找最低频会 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：维护 key→node；freq→双向链表（同频节点按 LRU 排列）；minFreq 指向当前最低频率。访问节点时从旧频率链移除并加入 freq+1 链尾；若旧链为空且等于 minFreq，则 minFreq++。 正确性依赖的不变量是：每个节点只存在于与其 freq 对应的一条链；minFreq 始终是非空频率桶最小值，因此淘汰可直接从该桶 LRU 端取。 最终复杂度为：平均 get/put O(1)，空间 O(capacity)。

Q095 Time Based Key-Value Store

难度 M	核心 HashMap / Binary Search	类别 复合数据结构设计

题目

支持 set(key,value,timestamp)；get(key,t) 返回 timestamp<=t 的最新值。通常 timestamp 单调写入。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次 get 扫描该 key 所有历史版本 O(n)。

关键洞察：为什么这个数据结构合适

HashMap key→按 timestamp 递增的 (t,value) 列表；set 直接 append；get 在该列表二分找最后一个 t_i<=query_t。

不变量（Invariant） 每个 key 的版本列表有序，因此“最后一个不大于 t”是标准 upper_bound-1。

参考实现（Python ）

```
from bisect import bisect_right
class TimeMap:
    def __init__(self): self.d={}
    def set(self,k,v,t): self.d.setdefault(k,[]).append((t,v))
    def get(self,k,t):
        a=self.d.get(k,[])
        i=bisect_right(a,(t,chr(0x10ffff)))-1
        return '' if i<0 else a[i][1]
```

复杂度

set O(1)（时间戳单调前提），get O(log versions)，空间 O(总版本数)。

高频失分点

- timestamp 不单调时仍 append
- 二分目标是 <=t 的最后一个，不是 ==t
- 使用 tuple 二分时 value 比较细节

面试官追问

- 如果版本可删除/乱序写入，需要什么结构?
- 如何做 MVCC/TTL?

60 秒口述模板

我会先给出一个可工作的基线方案：每次 get 扫描该 key 所有历史版本 O(n)。 真正的瓶颈在重复工作，因此优化的核心是：HashMap key→按 timestamp 递增的 (t,value) 列表；set 直接 append；get 在该列表二分找最后一个 t_i<=query_t。 正确性依赖的不变量是：每个 key 的版本列表有序，因此“最后一个不大于 t”是标准 upper_bound-1。 最终复杂度为：set O(1)（时间戳单调前提），get O(log versions)，空间 O(总版本数)。

Q096 All O(1) Data Structure

难度 H+	核心 HashMap / Bucket DLL	类别 复合数据结构设计

题目

维护字符串计数，支持 inc(key)、dec(key)、getMaxKey、getMinKey，全部平均 O(1)。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

HashMap key→count 后，max/min 每次扫描所有 key 是 O(n)；heap 会遇到更新与 stale entry。

关键洞察：为什么这个数据结构合适

用按 count 递增排列的双向“频率桶”链表，每桶存一组具有该 count 的 key；另有 key→bucket 映射。inc/dec 只把 key 移到相邻 count 桶，必要时创建/删除桶；头尾哨兵旁边直接给 min/max。

> **不变量（Invariant）** 桶链始终严格按 count 排序且不存在空桶；key 只会在当前 count 的唯一桶中，所以每次修改只影响局部相邻位置。

参考实现（Python ）

```
# 核心节点结构：Bucket(count, keys:set, prev, next)
# inc：当前 bucket c -> c+1 的相邻 bucket（不存在则插入）
# dec：当前 bucket c -> c-1；计数降到 0 则删除 key
# 删除空 bucket；head.next 为最小，tail.prev 为最大。
```

复杂度

每个操作平均 O(1)，空间 O(keys)。

高频失分点

- 用 TreeMap 会 O(log n)，不满足题目
- 桶为空后没删导致 min/max 错
- 同 count 多 key 需要集合而非单值

面试官追问

- 与 LFU 的 bucket 结构有什么共性?

60 秒口述模板

我会先给出一个可工作的基线方案：HashMap key→count 后，max/min 每次扫描所有 key 是 O(n)；heap 会遇到更新与 stale entry。 真正的瓶颈在重复工作，因此优化的核心是：用按 count 递增排列的双向“频率桶”链表，每桶存一组具有该 count 的 key；另有 key→bucket 映射。inc/dec 只把 key 移到相邻 count 桶，必要时创建/删除桶；头尾哨兵旁边直接给 min/max。 正确性依赖的不变量是：桶链始终严格按 count 排序且不存在空桶；key 只会在当前 count 的唯一桶中，所以每次修改只影响局部相邻位置。 最终复杂度为：每个操作平均 O(1)，空间 O(keys)。

Q097 RandomizedCollection：允许重复的 O(1) 随机集合

难度 H	核心 Array / HashMap	类别 复合数据结构设计

题目

支持 insert/remove/getRandom，允许重复值；getRandom 应按元素出现次数均匀抽样。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

Q20 的 value→单 index 不再够用，因为同值有多个位置。

关键洞察：为什么这个数据结构合适

数组仍保存每一次出现；HashMap 改为 value→set(indices)。删除 x 时任选一个 index，与数组尾元素交换，再同步更新两个 value 的 index set，最后 pop。随机从数组选位置天然按 multiplicity 加权。

不变量（Invariant） 索引集合始终精确等于数组中该 value 的全部位置；swap 时必须先把旧尾 index 从 lastValue 集合移除，再加入新位置。

参考实现（Python ）

```
from collections import defaultdict
import random
class RandomizedCollection:
    def __init__(self): self.a=[]; self.pos=defaultdict(set)
    def insert(self,x):
        fresh=not self.pos[x]; self.pos[x].add(len(self.a)); self.a.append(x); return fresh
    def remove(self,x):
        if not self.pos[x]: return False
        i=self.pos[x].pop(); last=self.a[-1]; last_i=len(self.a)-1
        if i!=last_i:
            self.a[i]=last
            self.pos[last].remove(last_i); self.pos[last].add(i)
        self.a.pop()
        return True
    def getRandom(self): return random.choice(self.a)
```

复杂度

平均 insert/remove/getRandom O(1)，空间 O(n)。

高频失分点

- x==last 时集合更新顺序把同一 index 删除两次
- 空 set 是否从 map 删除不是 correctness 必需但影响内存
- 随机应按 occurrence 而不是 distinct value

面试官追问

- 若希望每个 distinct value 等概率，结构如何变化?

60 秒口述模板

我会先给出一个可工作的基线方案：Q20 的 value→单 index 不再够用，因为同值有多个位置。 真正的瓶颈在重复工作，因此优化的核心是：数组仍保存每一次出现；HashMap 改为 value→set(indices)。删除 x 时任选一个 index，与数组尾元素交换，再同步更新两个 value 的 index set，最后 pop。随机从数组选位置天然按 multiplicity 加权。 正确性依赖的不变量是：索引集合始终精确等于数组中该 value 的全部位置；swap 时必须先把旧尾 index 从 lastValue 集合移除，再加入新位置。 最终复杂度为：平均 insert/remove/getRandom O(1)，空间 O(n)。

Q098 简化 Twitter News Feed

难度 H	核心 HashMap / Heap	类别 复合数据结构设计

题目

实现 postTweet、follow/unfollow、getNewsFeed(user) 返回该用户及关注者最近的 10 条 tweet。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

每次 get feed 扫描系统所有 tweet 再过滤，无法扩展。

关键洞察：为什么这个数据结构合适

每个用户单独保存按时间追加的 tweet 列表。查询时对本人+关注者的各自最新 tweet 做 K-way merge：堆中每人只放当前最新候选，弹出一个后再推进该用户到上一条。最多取 10 条。

不变量（Invariant） 每个用户 tweet 列表内部已按时间有序，因此其未消费部分的最大时间就是当前候选；所有候选最大者即全局下一条 feed。

参考实现（Python ）

```
import heapq
from collections import defaultdict
class Twitter:
    def __init__(self):
        self.t=0; self.tw=defaultdict(list); self.f=defaultdict(set)
    def postTweet(self,u,tid): self.t+=1; self.tw[u].append((self.t,tid))
    def follow(self,u,v): self.f[u].add(v)
    def unfollow(self,u,v): self.f[u].discard(v)
    def getNewsFeed(self,u):
        users=set(self.f[u]); users.add(u); h=[]
        for v in users:
            if self.tw[v]:
                i=len(self.tw[v])-1; ts,tid=self.tw[v][i]; heapq.heappush(h,(-ts,tid,v,i))
        out=[]
        while h and len(out)<10:
            nts,tid,v,i=heapq.heappop(h); out.append(tid)
            if i:
                ts,tid2=self.tw[v][i-1]; heapq.heappush(h,(-ts,tid2,v,i-1))
        return out
```

复杂度

若关注数 F，查询大致 O((F+10)log F)，发布 O(1)。

高频失分点

- 把所有关注者全部历史 tweet 放 heap
- 自己是否自动 follow 自己的语义
- 真实系统 fan-out on write/read 的 trade-off 没区分

面试官追问

- 大 V 用户（百万粉丝）怎么扩展?
- feed 缓存失效策略?

60 秒口述模板

我会先给出一个可工作的基线方案：每次 get feed 扫描系统所有 tweet 再过滤，无法扩展。 真正的瓶颈在重复工作，因此优化的核心是：每个用户单独保存按时间追加的 tweet 列表。查询时对本人+关注者的各自最新 tweet 做 K-way merge：堆中每人只放当前最新候选，弹出一个后再推进该用户到上一条。最多取 10 条。 正确性依赖的不变量是：每个用户 tweet 列表内部已按时间有序，因此其未消费部分的最大时间就是当前候选；所有候选最大者即全局下一条 feed。最终复杂度为：若关注数 F，查询大致 O((F+10)log F)，发布 O(1)。

Q099 设计内存文件系统

难度 H	核心 Trie / Tree	类别 复合数据结构设计

来源标记 公开 Google 2025 面经出现过“用 Trie 实现目录结构并扩展文件大小聚合”的开放题；候选人自述，非官方题库。

题目

设计 mkdir(path)、addContentToFile(path,content)、readContentFromFile(path)、ls(path)。路径按 / 分层。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?
- 空树、重复键、深度定义（节点数/边数）以及是否保证节点存在。

基线方案与瓶颈

把完整 path 作为 HashMap key 可以做部分 API，但目录 ls、层级创建与子树操作会越来越别扭。

关键洞察：为什么这个数据结构合适

文件系统天然是树/Trie：目录节点 children: name→node；文件节点额外保存 content/is_file。沿 path component 逐层查找/创建。ls(file) 返回文件名，ls(dir) 返回排序子节点名。

不变量（Invariant） 根到节点路径与绝对路径 component 一一对应；children map 保证每个目录下同名 child 唯一。

参考实现（Python ）

```
class FSNode:
    def __init__(self): self.ch={}; self.file=False; self.content=[]
class FileSystem:
    def __init__(self): self.root=FSNode()
    def _walk(self,path,create=False):
        p=self.root
        for part in [x for x in path.split('/') if x]:
            if create: p=p.ch.setdefault(part,FSNode())
            else: p=p.ch[part]
        return p
    def mkdir(self,path): self._walk(path,True)
    def addContentToFile(self,path,content):
        p=self._walk(path,True); p.file=True; p.content.append(content)
```

```
    def readContentFromFile(self,path): return ''.join(self._walk(path).content)
    def ls(self,path):
        p=self._walk(path)
        if p.file: return [path.rstrip('/').split('/')[-1]]
        return sorted(p.ch)
```

复杂度

单次路径操作 O(L) 查找（再加 ls 排序 O(d log d)），L 为 component 数。

高频失分点

- 目录与文件节点不区分
- root/尾斜杠/连续斜杠规范化
- 同一路径既文件又目录是否允许

面试官追问

- 如何增加 rm/mv/symlink?
- 如何维护目录总大小并支持增量更新?

60 秒口述模板

我会先给出一个可工作的基线方案：把完整 path 作为 HashMap key 可以做部分 API，但目录 ls、层级创建与子树操作会越来越别扭。 真正的瓶颈在重复工作，因此优化的核心是：文件系统天然是树/Trie：目录节点 children: name→node；文件节点额外保存 content/is_file。沿 path component 逐层查找/创建。ls(file) 返回文件名，ls(dir) 返回排序子节点名。 正确性依赖的不变量是：根到节点路径与绝对路径 component 一一对应；children map 保证每个目录下同名 child 唯一。 最终复杂度为：单次路径操作 O(L) 查找（再加 ls 排序 O(d log d)），L 为 component 数。

Q100 Hit Counter：最近 5 分钟请求计数

难度 M/H	核心 Queue / Circular Buffer	类别 复合数据结构设计

题目

记录 hit(timestamp)，查询 getHits(timestamp) 返回最近固定窗口（如 300 秒）内的 hit 数；时间戳通常单调。

面试开始先澄清

- 输入规模与合法性：是否允许空输入、重复值、负数，是否保证答案存在?
- 是否允许修改输入？输出是否要求稳定顺序或去重?
- 复杂度目标：题目是否明确要求 O(n)、O(log n) 或 O(1) 额外空间?

基线方案与瓶颈

保存所有 timestamp 并每次扫描全部历史，内存与查询成本不断增长。

关键洞察：为什么这个数据结构合适

若请求频率普通，可 deque 保存 (timestamp,count)，相同秒聚合；查询时弹出窗口外记录。若窗口长度固定且 timestamp 是整数秒，可用长度 300 的 circular array，以 timestamp%300 复用槽位。

不变量（Invariant） 队列中始终只保留当前查询仍可能落入窗口的 timestamp；过期项一旦弹出，未来也永远不会重新有效。

参考实现（Python ）

```
from collections import deque
class HitCounter:
    def __init__(self,window=300): self.q=deque(); self.total=0; self.w=window
    def hit(self,t):
        if self.q and self.q[-1][0]==t:
            ts,c=self.q.pop(); self.q.append((ts,c+1))
        else: self.q.append((t,1))
        self.total+=1
    def getHits(self,t):
        while self.q and self.q[0][0] <= t-self.w:
            _,c=self.q.popleft(); self.total-=c
        return self.total
```

复杂度

Deque 方案摊销 hit/get O(1)，空间 O(窗口内不同 timestamp 数)；环形数组固定 O(W)。

高频失分点

- 窗口边界 <= t-W 还是 < t-W 没按定义统一
- 每个 hit 单独入队，高 QPS 内存浪费
- 时间戳非单调时 deque 前提失效

面试官追问

- 分布式计数器如何处理时钟偏差?
- 近似统计可否使用时间桶?
- 若窗口从固定变成任意长度怎么办?

60 秒口述模板

我会先给出一个可工作的基线方案：保存所有 timestamp 并每次扫描全部历史，内存与查询成本不断增长。 真正的瓶颈在重复工作，因此优化的核心是：若请求频率普通，可 deque 保存 (timestamp,count)，相同秒聚合；查询时弹出窗口外记录。若窗口长度固定且 timestamp 是整数秒，可用长度 300 的 circular array，以 timestamp%300 复用槽位。正确性依赖的不变量是：队列中始终只保留当前查询仍可能落入窗口的 timestamp；过期项一旦弹出，未来也永远不会重新有效。 最终复杂度为：Deque 方案摊销 hit/get O(1)，空间 O(窗口内不同 timestamp 数)；环形数组固定 O(W)。

附录 A：复杂度速查表

结构	优势	短板	面试识别信号
Array	随机访问 O(1)	中间插入/删除 O(n)	连续内存、cache 友好
HashMap/Set	平均查找/插入/删除 O(1)	最坏 O(n)	依赖 hash 与负载因子
Linked List	已知节点后插删 O(1)	按位置访问 O(n)	指针不变量
Stack/Queue	push/pop/enqueue/dequeue O(1)	-	LIFO/FIFO
Binary Heap	push/pop O(log n)，peek O(1)	任意删除不方便	Top-K / 调度
BST(平衡)	查找/插删 O(log n)	退化树 O(n)	有序、范围查询
Trie	字符串操作 O(L)	空间可能很大	前缀查询
DSU	近 O(1) union/find	不擅长删除边	连通分量
Fenwick	update/query O(log n)	操作类型受限	前缀聚合
Segment Tree	query/update O(log n)	实现与常数较大	通用区间结构

附录 B：7 天 / 21 天 / 6 周复习路线

7 天冲刺：只覆盖最常出现的结构，目标是形成识别反射。

- Day 1：Q001-Q015 数组/字符串
- Day 2：Q016-Q033 Hash + 链表
- Day 3：Q034-Q043 栈队列 + Q059-Q066 Heap
- Day 4：Q044-Q058 Tree/BST
- Day 5：Q073-Q086 Graph/DSU
- Day 6：Q067-Q072 Trie + Q093-Q100 设计
- Day 7：随机抽 12 题做完整 45 分钟模拟，强制讲复杂度与 follow-up

21 天：每个章节安排 1-3 天，第一天看 Pattern 与基础题，第二天手写中高难，第三天只做变体与追问。

6 周：第 1-4 周按章节学习；第 5 周按 Pattern 混刷，打破“章节提示”；第 6 周做 45 分钟模拟面试，重点记录卡顿点、边界 bug 与表达问题。

附录 C：公开资料与趋势证据

Tech Interview Handbook - Data structures and algorithms study cheatsheets — 截至 2026 年的主题优先级与常见面试准备方法。

https://www.techinterviewhandbook.org/algorithms/study-cheatsheet/

Tech Interview Handbook - Coding interview techniques — Hash、Graph、Stack/Queue、Heap、Tree/Trie 以及常见算法模式的识别建议。

https://www.techinterviewhandbook.org/coding-interview-techniques/

Tech Interview Handbook - Linked List / Heap / Tree / Trie guides — 经典结构的操作、边界与核心练习题。

https://www.techinterviewhandbook.org/algorithms/

LeetCode Discuss - Google L5 Interview, January 2025 — 候选人自述：Shift sweep-line、无根树重新选根、Range Module 等变体；非官方。

https://leetcode.com/discuss/post/6522571/

LeetCode Discuss - Google L3 interview experience, 2025 — 候选人自述：开放式内存文件系统/Trie 设计；非官方。

https://leetcode.com/discuss/post/6982323/

LeetCode Discuss - Google SWE II Early Careers, 2026 — 候选人自述：Trie+HashMap、Graph+DSU 等组合题趋势；非官方。

https://leetcode.com/discuss/post/8449337/

LeetCode Discuss - Google Trie Question — 候选人自述：层级字符串/路径 Trie 变体；非官方。

https://leetcode.com/discuss/post/7101066/

结语：面试数据结构的真正目标

不要背 100 份代码 目标是看到约束后，能解释为什么某个结构减少了重复工作；能用不变量证明它正确；能在 follow-up 改变约束后替换结构，而不是重新从零猜答案。

如果只能记住一句话：先把问题写成“我需要反复查询/维护什么信息”，再问哪种数据结构能以最低成本维护这个信息。数据结构不是答案，它是为某个不变量服务的工具。